# CoCoA SONIC: Simulated Observables for Numerical Inference in Cosmology
# A Tensor-by-Tensor Guide from Generated Cosmologies to Inference

V. Miranda

*Code documentation manuscript*

(Dated: Working draft)

This manuscript follows one cosmological sample through CoCoA SONIC, the unified CoCoA emulator library. SONIC abbreviates **S**imulated **O**bservables for **N**umerical **I**nference in **C**osmology. The program begins with generated parameter tables and data-vector dumps, then explains staging, coordinate transforms, neural designs, loss construction, device-aware batching, training, family-specific physics, artifact reconstruction, Cobaya adapters and executable acceptance gates. Cobaya is the Bayesian-inference framework that asks the saved emulator for theory predictions while sampling cosmological parameters. The emphasis is operational: every tensor is assigned a shape, dtype, device and physical meaning and every non-obvious Python or PyTorch mechanism is defined where it first changes the calculation.

**CoCoA SONIC as a physical instrument.** A Doctor-inspired science traveler uses a sonic tool to assemble a cosmological surrogate from measured coordinates, transformations and checks. The image is a frontispiece and naming illustration. Every numbered scientific figure below is generated by the reproducible Python script stored with this manuscript. The artwork is included at its original aspect ratio. LaTeX changes only its displayed size.

## I. HOW TO READ THE PROGRAM

The emulator is a sequence of ownership boundaries. A generated row begins as numbers in files, becomes a NumPy array, becomes a PyTorch tensor on a selected device, becomes a network prediction in transformed coordinates, returns to physical units and is finally published in a saved artifact. The value is scientifically useful only if its row identity, column names, units, axis coordinates, dtype and transformation law survive every boundary.

This manuscript follows the executable order. Each section covers the same five checks:

1. Identify the Python objects that exist before the operation.

2. Name the function or method that owns the operation.

3. Identify the new objects that exist afterward.
4. Record which operation copied data, returned a view or deferred work.
5. State the numerical identity that verifies preservation of physical meaning.

### A. The notation used throughout

Uppercase letters denote a collection of rows. A lowercase symbol with row subscript $i$, such as $r_i$, denotes one row. A leading dimension $B$ is a minibatch size. The remaining dimensions describe one sample.

During training, $1 \leq B \leq N$. The configured batch size is an upper bound: the last minibatch can be shorter when the selected row count is not divisible by it. Axis order is part of the contract. Shapes $(B, K)$ and $(K, B)$ contain the same number of values but assign different meanings to each index. A plain model has no $T_{\mathrm{tmpl}}$ axis. A factored intrinsic-alignment model adds it between batch and output coordinates.

### B. The Python object vocabulary

Several Python words have precise meanings in the code:

**Array.:** A NumPy object containing numerical values on the CPU. An `ndarray` is normally resident in host memory. An `np.memmap` presents a file as an array and reads requested pages on demand.

**Tensor.:** A PyTorch numerical object. It has a shape, dtype and device. A tensor can also carry the information PyTorch needs to compute gradients.

**Module.:** A subclass of `torch.nn.Module`. A module owns parameters, registered buffers, child modules and a `forward` method.

**Parameter.:** A tensor registered for optimization. When `requires_grad=True`, backward differentiation accumulates its derivative in `parameter.grad`.

**Buffer.:** A tensor that belongs to a module and moves with `module.to(device)`. Only parameters are updated by the optimizer. Fixed basis-change matrices are buffers.

**Callable.:** Any object invoked with parentheses. A function, a class and an `nn.Module` instance are all callable, but the parentheses do different work: a class call constructs an instance. A module call runs its forward path.

**Iterator.:** An object that produces one item at a time and remembers its current position. A `for` loop consumes it. It is not necessarily a materialized list.

**Closure.:** A function that retains values from the surrounding scope where it was created. Loader closures remember where their data live.

**View.:** A new array or tensor description that shares underlying storage with another object. Mutating a writable view can change the original.

**Copy.:** A new storage allocation containing duplicated values. Later mutation does not change the source storage.

**Artifact.:** The saved predictor product: a PyTorch weight file with suffix `.emul` and an HDF5 recipe/geometry file with suffix `.h5`. Both are needed to reconstruct a physical predictor.

**Schema.:** The declared structure of an input or saved record: allowed keys, value types, shapes and relations between values. Validation checks a schema before a downstream API can reinterpret malformed input.

**Payload.:** The numerical values carried by a file, process result or record, as distinct from the metadata that explains their meaning. A finite payload can still be wrong if its axis or units are mislabeled.

**Provider.:** An external physics component queried for a result, such as CAMB inside Cobaya. The emulator code controls the request and validates the returned payload. It does not own the provider's internal algorithm.

**State.:** Mutable values an object retains between method calls. Model weights, optimizer moments, current Cobaya results and experiment slots are different kinds of state with different lifetimes.

**Provenance.:** Facts identifying how a result was produced: source files or digests, resolved settings, coordinates, dtype, device and implementation identity.

**Boundary.:** A function, method, file format or external interface at which responsibility changes. Validation belongs at the earliest boundary that has enough information to name an invalid value.

### C. Python mechanics that the later code uses

**Dictionary or mapping.:** A set of key-value associations. Looking up `cfg["data"]` requires the key and raises if it is absent. `cfg.get("data")` returns `None` when absent unless a default is supplied. That hidden default is part of control flow and must not stand in for validation.

**List and tuple.:** Both are ordered sequences. A list is mutable: an item can be replaced or appended. A

| Symbol | Typical shape | Physical meaning | Program owner |
|---|---|---|---|
| $C$ | $(N, P)$ | Raw cosmological parameters, one cosmology per row | `load_source` and the source dictionary |
| $V$ | $(N, D)$ | Raw physical data vectors | `load_source` |
| $X$ | $(B, P_{\mathrm{enc}})$ | Encoded model inputs | Parameter geometry and `load_C` |
| $T$ | $(B, K)$ or $(B, T_{\mathrm{tmpl}}, K)$ | Encoded training targets or template stack | Output geometry, loss wrapper and `load_dv` |
| $\widehat{T}$ | Same target shape | Model prediction in network coordinates | `nn.Module.forward` |
| $R$ | $(B, K)$ | Prediction-minus-truth residuals in physical kept coordinates | Loss wrapper |
| $\Delta\chi^2$ | $(B,)$ | One covariance-weighted error per cosmology | Loss wrapper and evaluation code |

TABLE I. Symbols and shapes used throughout the manuscript. A shape lists the length of each array or tensor axis in parentheses. $N$ is the number of cosmologies stored in a complete table or dump. $B$ is the smaller number of cosmologies processed together in one minibatch. $P$ is the number of raw cosmological parameter columns and $P_{\mathrm{enc}}$ is the width after the parameter geometry has centered, rotated or otherwise encoded those columns. $D$ is the width of the complete physical data vector on disk. $K$ is the number of kept output coordinates predicted by the network after masks or cuts have removed unused coordinates. The optional $T_{\mathrm{tmpl}}$ axis counts intrinsic-alignment templates. It exists only for a factored template model. A one-dimensional shape $(B,)$ therefore means one scalar value for each cosmology in the minibatch. The minibatch therefore contains $B$ scalar values.

tuple is immutable after construction and is commonly used for a fixed shape or fixed return record. Both are eager, with all contained object references already present.

`None`.: Python's one "no object" sentinel. It is distinct from zero, an empty list, an empty mapping and `False`. Code tests it with `is None` when those other values have their own meanings.

**Truthiness.:** Conditions such as `if value:` convert an object to a boolean. Zero and empty containers are false. Nonempty strings are true, including the string `"false"`. Public configuration therefore requires an actual boolean. Truthiness cannot validate that contract.

**Positional and keyword arguments.:** In `f(x, scale=2)`, `x` fills the first parameter by position and `2` fills the parameter named `scale`. Keyword-heavy construction is used where several values share a type or shape, because order alone would not explain their roles.

**Generator.:** A function containing `yield` returns a lazy iterator. It pauses at each yielded item and resumes on the next request. It does not construct the full list first. Converting it with `list` deliberately materializes every remaining item in memory.

**Context manager.:** A `with` statement calls an object's entry and exit protocol. File and no-gradient contexts use it so cleanup or gradient restoration occurs even when the body raises.

**Class method.:** An alternative constructor marked `@classmethod` receives the selected class as `cls`. Returning `cls(...)` preserves subclass behavior. The ordinary `__init__` stores already prepared fields. Named constructors such as `from_state` own loading and validation branches.

**Scope.:** A local name belongs to one function call. `nonlocal` lets an inner closure rebind a name in its enclosing function. A module global outlives all calls. Data arrays, devices, fitted geometries and configuration state enter through explicit arguments, so a function's signature states its real inputs without silent globals.

The definitions above become useful only when they are attached to an executed expression. The next examples unpack the mechanics that recur in the staging and training code.

### *1. Names refer to objects. Assignment creates another reference*

Consider

```
source = {"idx": selected_rows}
alias = source
alias["idx"] = replacement_rows
```

The first line constructs one dictionary object. The second line binds the name `alias` to that same dictionary. The third line therefore changes what both `alias["idx"]` and `source["idx"]` return. A shallow dictionary copy, `alias = dict(source)`, creates a new outer dictionary but still shares the array stored under `"idx"`. A deep copy recursively duplicates nested mutable objects. This distinction matters in sweep drivers:

one run must not change the nested recipe later runs will consume.

An in-place operation changes an existing object. NumPy's `array *= scale` and PyTorch's methods ending in `_`, such as `tensor.zero_()`, are in-place. The expression `array = array * scale` computes a new result and rebinds the local name. In-place tensor operations require special care when autograd has saved the old value for a derivative.

### *2. Basic indexing can be a view. Advanced indexing is a copy*

For a two-dimensional NumPy array `C`, the slice `C[1:3, :]` uses basic slicing. It normally returns a view: a new shape-and-stride description of the same memory. The integer-array expression `C[[1, 3], :]` uses *advanced indexing*. NumPy gathers the requested rows into a new allocation, so the result is a copy. The order of the integer array is preserved. `C[[3, 1]]` returns row 3 before row 1.

This mechanic is why a row-coordinate array is not bookkeeping decoration. If a disk-backed loader retains `C` and later evaluates `C[idx]`, the gather happens when the batch is requested. If a resident staging branch evaluates `C[idx]` immediately, it has already made the compact copy. The compact array must still carry a local index map that reproduces the same selected-row order.

### *3. Lazy and eager operations move work to different times*

`np.load(path, mmap_mode="r")` is eager about the small file header but lazy about the large numerical payload. It creates a memory-map object whose pages are read when indexing requests them. `np.loadtxt(path)` parses the complete text file immediately and returns a resident array. Both choices preserve the mathematical values while assigning memory and I/O costs to different times.

A Python generator is lazy for a different reason: its function body pauses at `yield`. A `for` loop asks it for the next item. A list comprehension is eager and stores all results. When reviewing a large-data path, determine whether constructing the iterable already materializes all rows. The iterable label alone leaves its construction cost unspecified.

### *4. One host-to-device chain, read operation by operation*

The training code often uses the following readable chain:

```
t_tr = (
    torch.from_numpy(C_tr)
    .float()
    .to(device)
)
```

It performs three distinct transformations.

1. `torch.from_numpy(C_tr)` creates a CPU tensor that initially shares the NumPy array's storage. Changing a writable shared value from either side can change the other side.

2. `.float()` requests PyTorch `float32`. A text loader can produce NumPy `float64`, in which case this conversion allocates new float32 storage and ends the sharing. The primary `load_source` path already requests NumPy `float32`. In that case `.float()` may return the same tensor and CPU storage sharing can continue. The method returns a tensor.

3. `.to(device)` returns a tensor on the requested device. CUDA is PyTorch's interface to supported NVIDIA GPUs. MPS is its interface to Apple's Metal accelerator on a Mac. Either destination copies values from ordinary host memory to accelerator memory. A CPU destination may return the same tensor when dtype and device already match, so `to` should not be described as an unconditional copy.

After the chain, `t_tr` has the same two axis lengths as `C_tr`, dtype `torch.float32` and device `device`. Device movement preserves its gradient setting. Model parameters are registered with `requires_grad=True`. Ordinary input batches normally use `requires_grad=False`.

### *5. Module calls, registration and gradient accumulation*

Calling `model(x)` invokes `nn.Module.__call__`, which manages PyTorch hooks and then calls `model.forward(x)`. Code should call the module so those hooks run before `forward`. Parameters are found by walking registered attributes such as `nn.Linear`, `nn.Sequential` and `nn.ModuleList`. A bare Python list of modules is invisible to that walk: its weights will be missing from `model.parameters()`, device movement and the saved state dictionary.

The backward call adds derivatives into each existing `parameter.grad`. The optimizer therefore clears gradients before the next minibatch. This is *gradient accumulation*: omitting the clear step deliberately or accidentally sums contributions from several backward calls. The manuscript will name that clear, forward, loss, backward, step order again when it reaches the epoch loop.

### *6. Factories and anonymous callables*

A normalization or activation factory is a callable stored for later use. For example,

```
act_factory = lambda size: nn.Tanh()
activation = act_factory(width)
```

The anonymous function receives `width` in its first and only parameter, named `size`. Tanh has no width-dependent parameters, so the body ignores that value. The parameter is retained so this factory has the same interface as a learnable activation whose constructor does need the width. Each call constructs a fresh module. Reusing one already-constructed `nn.Tanh()` instance risks accidental sharing when the module family later gains buffers or parameters.

### *7. Method binding and the meaning of `self`*

A method definition such as

```
def encode(self, raw):
    return (raw - self.center) / self.scale
```

has two parameters. In the call `geometry.encode(batch)`, Python binds `self` to the object named `geometry` automatically and binds `batch` to `raw`. The stored arrays `geometry.center` and `geometry.scale` are therefore explicit object state. Module globals have no role in them. Calling the underlying class function as `Geometry.encode(geometry, batch)` supplies the same two arguments manually, but ordinary code uses the bound-method form.

### *8. Imports bind names in the importing module*

`import numpy as np` binds the module object to the local module name `np`. `np.asarray` then looks up the function on that module. `from package.helpers import resolve` binds the current `resolve` object directly. That local name keeps the object bound at import time. A test monkeypatch must replace the name read by the production function at its live binding site.

A leading `_` in a name such as `_build_loaders_one` is a convention meaning "internal to this module or package." Python still allows direct access. The convention tells readers that the supported public path is its caller and that direct use may bypass validation or lifecycle rules.

A dynamic import chooses the module while the program is running. For example,

```
module_name = "emulator.geometries.parameter"
module = importlib.import_module(module_name)
geometry_class = getattr(
    module, "ParamGeometry")
```

The first line stores an ordinary string. The second line asks Python to load the module named by that string and returns the module object. The third line looks up the class attribute on that module. It does not construct an instance yet. A later `geometry_class(...)` call constructs the object. A static scanner that searches only literal `import` statements cannot discover a name assembled in this way. Such a scanner therefore needs a reviewed declaration for dynamic imports. Its report must identify the blind spot and classify the import manifest as incomplete.

### *9. Return unpacking and ignored values*

If a function returns a two-item tuple,

```
center, scale = param_stats(values)
```

assigns the first item to `center` and the second to `scale`. The number of names must match the number of returned items unless starred unpacking is used. Writing `center, _ = ...` still evaluates and returns the second item. The `_` merely tells a human reader that this call site intentionally does not use it.

### *10. Starred arguments expand a container at the call boundary*

In `f(*items)`, the asterisk supplies sequence entries as positional arguments. In `cls(**options)`, two asterisks supply mapping entries as keyword arguments: the key `"width"` fills the constructor parameter named `width`. Dictionary expansion is concise, but it must occur only after an allowlist has rejected unknown keys. Otherwise a misspelling can be silently swallowed by a constructor that accepts arbitrary `**kwargs`. The resulting object would hide which values it consumed.

### *11. Exceptions and `try/finally`*

Raising an exception stops the ordinary sequence of statements and searches outward for a matching handler. A `finally` block runs whether the body returns normally or raises:

```
resource = acquire()
try:
    use(resource)
finally:
    release(resource)
```

The release call therefore owns cleanup. The protected body and its exception path own the success verdict. Temporary memory maps, worker processes and staged files need this structure so a failed training point does not leak resources into the next point. Catching a broad `Exception` and then reporting success would destroy the failure verdict. Cleanup and verdict are separate responsibilities.

#### 12. Broadcasting aligns trailing axes without materializing copies

Suppose `H.shape == (B, W)`, `scale.shape == (W,)` and `bias.shape == (W,)`. In `H * scale + bias`, PyTorch aligns the length-$W$ axis with the last axis of `H` and applies the same scale and bias to every row. Conceptually the one-dimensional arrays repeat $B$ times, but broadcasting normally uses a stride-based view that avoids allocating $B$ copies. A shape $(B,)$ would not mean the same thing: its trailing length must equal $W$ or be one, otherwise the operation raises.

#### 13. Reshaping changes coordinates while preserving numerical order

`reshape` requests a new axis description containing the same number of elements. NumPy may return a view when the existing strides permit it and a copy otherwise. PyTorch `view` requires a compatible contiguous layout. `reshape` may make a contiguous copy when necessary. Under the row-major layout used here, both operations preserve the element sequence while changing the axis description. Flattening a surface of shape $(N_z, N_k)$ in row-major order places all $k$ values for redshift index zero first, then all $k$ values for redshift index one. Rebuilding the surface must use the identical order. The same element count cannot detect a swapped axis convention.

Mutable defaults deserve a separate rule. Python creates a default object once when the function is defined and shares it across calls. Therefore `def f(opts={})` shares one dictionary across calls. The safe form is `def f(opts=None)` followed by construction of a new dictionary when `opts is None`.

### D. A running numerical example

The small example below will reappear at each boundary. It has four cosmologies, three parameters and five data-vector entries:

$$C = \begin{bmatrix} 0.29 & 67 & 0.048 \\ 0.31 & 70 & 0.050 \\ 0.33 & 73 & 0.052 \\ 0.30 & 69 & 0.049 \end{bmatrix}, \qquad V = \begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 11 & 22 & 33 & 44 & 55 \\ 12 & 24 & 36 & 48 & 60 \\ 10.5 & 21 & 31.5 & 42 & 52.5 \end{bmatrix}.$$

The parameter names are

$$[\texttt{omegam}, \texttt{H0}, \texttt{omegab}].$$

Here `omegam` is the dimensionless matter-density fraction $\Omega_m$, `H0` is the Hubble constant in $\mathrm{km\,s^{-1}\,Mpc^{-1}}$ and `omegab` is the dimensionless baryon-density fraction $\Omega_b$. The five illustrative target columns are named $v_0, \ldots, v_4$. They use common toy units so the example can expose masking and covariance operations without introducing a family-specific unit conversion. A production artifact persists the physical spectrum, redshift, multipole or angular coordinate attached to each target column. Original disk row 1 (the second displayed row because Python starts counting at zero) means that the cosmology $(0.31, 70, 0.050)$ produced the physical data vector $(11, 22, 33, 44, 55)$. The later seeded training selection will not include this row, so Steps 3 through 5 use it as a held-out validation cosmology transformed with statistics fitted only on selected training rows. "Local row 1" means the second position in a compact selected array. Original disk row 1 is a separate coordinate. The original row number is part of the meaning. If staging pairs that parameter row with row 2 of $V$, every later tensor can be finite, every training loss can decrease and the learned function is still wrong.

Assume the likelihood keeps output columns $[0, 2, 4]$. Then $D = 5$ and $K = 3$. The network predicts only those three columns. The geometry retains the global positions of columns 1 and 3 so it can scatter a kept residual back into a full vector.

#### 1. Step 1: read shapes as concrete counts

The expression $C \in \mathbb{R}^{4\times 3}$ states that `C.shape` is `(4, 3)`. The first axis has $N = 4$ cosmologies and the second has $P = 3$ named parameters. Likewise, `V.shape` is `(4, 5)`: the same four cosmologies and $D = 5$ physical output coordinates. The equality of the first axis is a first check because row $i$ in each file must describe the same simulation. Row-identity evidence supplies the second check, since two independently permuted files can both have four rows.

In NumPy, `C[1]` removes the row axis and returns shape `(3,)`. The comma in a one-dimensional mathematical shape such as $(3,)$ matters: it distinguishes a vector from a scalar. To preserve a length-one row axis, use `C[1:2]`, whose shape is `(1, 3)`. A model expects the latter two-dimensional convention because its first axis always counts samples.

### 2. Step 2: preserve the selected order

Suppose a seeded selection produces original disk-row coordinates

$$i_{\text{selected}} = [3, 0, 2].$$

The corresponding parameter copy is not sorted:

$$C[i_{\text{selected}}] = \begin{bmatrix} 0.30 & 69 & 0.049 \\ 0.29 & 67 & 0.048 \\ 0.33 & 73 & 0.052 \end{bmatrix}.$$

The target gather must use the identical index sequence, so its first rows are $(10.5, 21, 31.5, 42, 52.5)$, then $(10, 20, 30, 40, 50)$, then $(12, 24, 36, 48, 60)$. Sorting the coordinates to $[0, 2, 3]$ preserves the set but changes which cosmology occupies each local row. A later seeded minibatch permutation acts on local row positions, so that order difference changes the minibatches even when all three cosmologies are eventually seen.

For batch size $B = 2$, taking the first two local rows gives

$$C_B = C[i_{\text{selected}}[0:2]], \qquad V_B = V[i_{\text{selected}}[0:2]],$$

both with a leading axis of length two. The slice `0:2` includes local positions 0 and 1 but excludes position 2. The nested gather is written here to expose the two coordinate systems. Production staging may first build a compact resident array and later index that compact array with an explicit local-coordinate map.

### 3. Step 3: encode one parameter row

The parameter geometry gets two facts from two different owners. It computes the center from selected training rows 3, 0 and 2:

$$\mu_C \simeq (0.306667,\ 69.666667,\ 0.049667),$$

The parameter covariance file supplies the rotation and whitening scales. For this worked example, declare the diagonal covariance

$$\Sigma_C = \text{diag}(0.01^2,\ 2^2,\ 0.001^2).$$

Although the covariance is diagonal, `np.linalg.eigh` returns its eigenvalues in ascending order. The variances appear in original parameter order as $(10^{-4}, 4, 10^{-6})$, so the eigenspace column order is (`omegab`, `omegam`, `H0`) and differs from the input order. The eigenvector matrix $Q$ is therefore a permutation matrix and the square-root eigenvalues in eigenspace order are $s_C = (0.001, 0.01, 2)$. A nondiagonal file would also rotate the directions as well as permute them. Encoding held-out original disk row 1 means subtracting the training mean, multiplying by $Q$ and then dividing in eigenspace order:

$$\begin{aligned} x_1 &= [(C_1 - \mu_C)Q] \oslash s_C \\ &\simeq (0.3333,\ 0.3333,\ 0.1667). \end{aligned}$$

The symbol $\oslash$ denotes elementwise division. Numerator and denominator both have shape $(3,)$ and both are now in eigenspace order. Decoding reverses the operations in reverse order: $C_1 = (x_1 \odot s_C)Q^{\mathsf{T}} + \mu_C$, where $\odot$ is elementwise multiplication. Centering changes the coordinate origin but cannot by itself change a prediction-minus-truth residual: the same mean is subtracted from both sides and cancels.

### 4. Step 4: keep, center and whiten one target

The kept global columns are $[0, 2, 4]$. Their mean over selected training rows 3, 0 and 2 is

$$\mu_V \simeq (10.833333,\ 32.5,\ 54.166667).$$

Use the illustrative diagonal physical covariance

$$\Sigma_{\text{kept}} = \text{diag}(1^2, 3^2, 5^2).$$

Its square-root scales are $s_V = (1, 3, 5)$. Held-out original disk row 1 has kept physical values $(11, 33, 55)$, so its encoded target is

$$\begin{aligned} t_1 &= \frac{(11, 33, 55) - \mu_V}{s_V} \\ &\simeq (0.1667,\ 0.1667,\ 0.1667). \end{aligned}$$

The target has shape $(3,)$ for one row and $(B, 3)$ for a minibatch. The network predicts these dimensionless coordinates. Conversion back to physical units belongs to the output geometry.

### 5. Step 5: convert a network error into a scientific error

Suppose the prediction for that row is

$$\widehat{t_1} \simeq (0.2667,\ 0.0667,\ 0.2167).$$

The encoded residual is

$$\widehat{t_1} - t_1 = (0.100,\ -0.100,\ 0.050).$$

Multiplication by $s_V$ decodes it into the kept physical residual

$$r_1 = (0.100,\ -0.300,\ 0.250).$$

Scattering those entries back to their original positions produces the full five-coordinate residual

$$r_{1,\text{full}} = (0.100,\ 0,\ -0.300,\ 0,\ 0.250).$$

The zeros mark coordinates excluded by this likelihood mask. The physical error at those coordinates remains unmeasured by this residual. With the diagonal covariance above,

$$\begin{aligned} \Delta\chi_1^2 &= r_1 \Sigma_{\text{kept}}^{-1} r_1^{\mathsf{T}} \\ &= \frac{0.100^2}{1^2} + \frac{(-0.300)^2}{3^2} + \frac{0.250^2}{5^2} \\ &= 0.0225. \end{aligned}$$

The same result is the squared Euclidean norm of the encoded residual because this illustrative geometry whitens with the covariance square root. Later sections generalize this calculation to dense covariance matrices, masks, template axes and nonlinear target laws. The ownership rule does not change: the geometry defines coordinates, the model predicts in those coordinates and the loss reconstructs the physical statistic.

### E. The complete ownership chain

Figure 1 separates a program action, printed above an arrow, from the object that exists after that action, printed inside a box. The number in each box is execution order. A box is therefore a state of the run. The arrow above it names the function call.

Each arrow has one owner. If two modules independently reimplement the same physical transformation, they can drift while their local tests remain green. The design therefore reuses geometry and loss decoding at inference through one prediction formula.

### F. How this manuscript states software limits

The repository identifies itself as alpha software. This manuscript describes the behavior a reader can execute in the current tree. When that behavior has a known limit, the relevant section states the consequence and a safe way to use the present implementation. Design proposals, repair queues and landing records live in `ai/notes/`. Keeping them out of the teaching text lets a reader learn one current program instead of reconstructing its development history.

The executable gate log at a named commit remains the status authority. A warning is removed only after the corresponding behavior and gate have been audited. The explanation of the resulting mechanism remains in the guide.

## II. STARTING A RUN: COMMANDS, PATHS AND CONFIGURATION

The numerical story starts with a shell command. The first tensor appears later. This section follows that command until the moment the program is ready to open the first data file. The distinction matters because configuration errors should be rejected before a large array is staged or a GPU allocation is made.

### A. The five kinds of driver

A *driver* is a small Python program that chooses how many complete training runs to perform. It does not contain a second implementation of the emulator. Every driver eventually asks `EmulatorExperiment` to perform the same staging, geometry construction, model construction and training operations.

For example, the ordinary cosmic-shear command has the form

```
python cosmic_shear_train_emulator.py \
  --root projects/lsst_y1/ \
  --fileroot emulators/training_scripts/ \
  --yaml cosmic_shear_train_emulator.yaml \
  --diagnostic diagnostic
```

The continuation character at the end of the first three lines tells the shell that the command continues on the next line. It is shell syntax. It is consumed before Python starts. The optional `-diagnostic` value asks the driver to create a diagnostic report after training. The training objective stays unchanged.

The family-specific programs are deliberately thin. Scalar, cosmic microwave background (CMB), background and matter-power drivers select the appropriate family contract and then reuse the same experiment engine. This is important to the reader: a behavior found in the generic experiment or training module normally affects all five families. A behavior in a family geometry affects only that physical output.

### B. Four path concepts form one file location

The command uses three different path concepts.

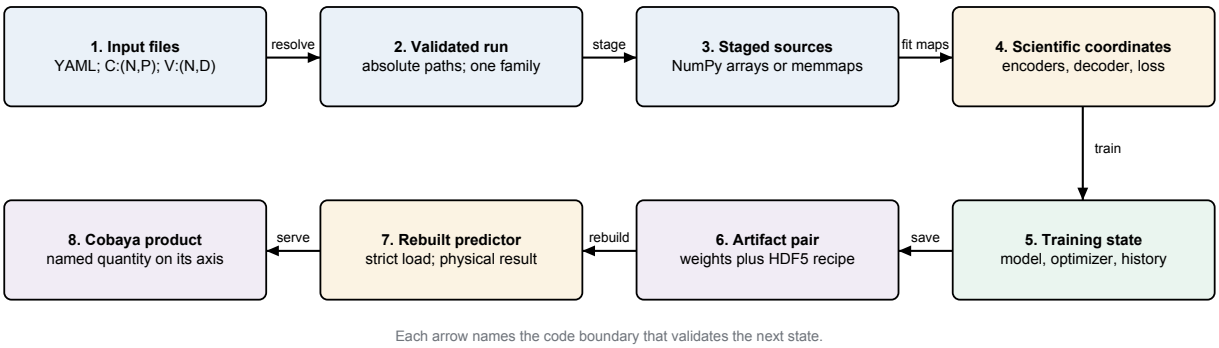
1. `ROOTDIR` is an environment variable exported by Cocoa. An environment variable is a string owned by the shell and inherited by the Python process.
2. `-root` is the project directory relative to `ROOTDIR`. Its `chains` child owns generated data and trained artifacts.
3. `-fileroot` is the directory, under the project, that owns the training YAML and driver reports.
4. `-yaml` is a bare file name inside `-fileroot`. It is not interpreted relative to the current terminal directory.

The configuration file in the example is conceptually

`ROOTDIR/-root/-fileroot/-yaml`.

Training arrays named by the YAML resolve under `ROOTDIR/-root/chains`. External likelihood files use a different declared root under Cocoa's external-data directory. A path resolver performs these joins. The numerical code receives resolved absolute paths and remains independent of the process's current working directory.

This separation prevents a common reuse bug. A relative path such as `chains/train.npy` means different files after `os.chdir`. A resolved path has one meaning throughout the run and can be written into the artifact provenance.




| Move | Code owner | Meaning of the move |
|---|---|---|
| $1 \to 2$ | `resolve_cocoa_config`, `from_config` | Join paths, parse the mapping, reject invalid values and select one physical family. |
| $2 \to 3$ | `stage_train`, `stage_val` | Select matched rows and choose a host-RAM or disk-backed representation. |
| $3 \to 4$ | `build_geometry` | Fit the reversible input/output coordinate maps and construct the scientific error function. |
| $4 \to 5$ | `build_loaders`, `run_emulator` | Convert batches to device tensors, construct trainable objects and execute optimization and validation. |
| $5 \to 6$ | `save_emulator` | Move the selected numerical state into the two persistent files. |
| $6 \to 7$ | `rebuild_emulator`, `EmulatorPredictor` | Recreate the recorded classes and load the exact saved tensors. |
| $7 \to 8$ | Cobaya adapter | Translate one physical prediction into the quantity and coordinate sequence requested by the likelihood. |

FIG. 1. The complete ownership chain. Blue boxes contain file- or host-side data, gold boxes own reversible scientific coordinates, green owns optimization state and purple marks a published interface. An arrow names the code that must validate the object crossing that boundary.

| Driver kind | Unit of work | Result established |
|---|---|---|
| Train | One model from one resolved YAML file | Artifact produced by this exact configuration |
| Training-size sweep | One independent model for each requested row count | Accuracy trend as more simulations are supplied |
| One-knob sweep | One model for each value of one named configuration leaf | Isolated effect of that control |
| Hyperparameter search | A sequence of proposed configurations recorded in a study | Configuration that minimizes the declared validation objective |
| Activation bake-off | One training-size curve per activation family | Relative learning rate across the full data-volume range |

TABLE II. Driver kinds and the scientific question each campaign answers. Every row delegates one complete training run to the same experiment engine. The driver changes how many recipes are proposed and compared. The experiment engine retains the definition of one optimizer step.

### C. The YAML is a construction recipe

YAML is a text representation of nested mappings and sequences. A mapping is a collection of named key-value pairs. A sequence is an ordered list. At the top level, an ordinary training file contains a `data` mapping and a `train_args` mapping:

```
data:
  train_params: train.1.txt
  train_dv: train.npy
  train_covmat: train.covmat
  val_params: validation.1.txt
  val_dv: validation.npy
  n_train: 25000
  n_val: 5000
  split_seed: 0
  ram_frac: 0.7

train_args:
```

```
  nepochs: 1600
  bs: 256
  loss:
    mode: sqrt
  model:
    name: resmlp
    mlp:
      width: 128
      n_blocks: 4
```

Indentation creates ownership. For example, `width` belongs to `train_args.model.mlp`, a nested setting. A dotted name is documentation shorthand for walking nested mappings, while the YAML parser stores each leaf key inside its containing dictionary.

The `data` block answers where the rows come from, which physical family they represent, how many rows to keep and how much host memory staging may use. The `train_args` block answers which objective, architecture, optimizer controls, schedules and run length to construct. Keeping these questions separate allows the same model recipe to be evaluated on different data volumes without moving file-system facts into a neural-network class.

### D. Building a training file, block by block

Never start from an empty page. The `example_yamls/` folder ships an annotated, runnable training file for every family and, for cosmic shear, the fine-tune, transfer, sweep and search campaigns as well. The safe route is to copy the file whose family and campaign match, then edit keys. This subsection builds the smallest of them, `example_yamls/scalar_emulator.yaml`, so the reader sees which decisions a file encodes and in which order to make them.

The first decision is the data: where the rows live and how many to use. A scalar run reads inputs and outputs from one parameter chain, so its complete `data` block is

```
data:
  train_params: chain_thetastar_lcdm.1.txt
  train_covmat: chain_thetastar_lcdm.covmat
  val_params:   chain_thetastar_lcdm_val.1.txt
  outputs:
    - H0
    - omegam
  n_train:    100000
  n_val:      20000
  split_seed: 0
  ram_frac:   0.7
```

The two `.txt` files hold the training and validation rows, and the `.covmat` header fixes the sampled-parameter names and their whitening order. `outputs` lists the derived-parameter columns to emulate, by the names their sidecar declares. `n_train` and `n_val` are absolute row counts, `split_seed` fixes the row shuffle, and `ram_frac` bounds how much host memory staging may use.

The same position in the file is where the family is chosen, by presence: `outputs` selects the scalar family, a `data.cmb` sub-block the CMB family, `data.grid` the background family, `data.grid2d` the matter-power family, and the CosmoLike file keys (`train_dv`, `val_dv`, `cosmolike_data_dir`, `cosmolike_dataset`) the cosmic-shear family. Exactly one of these may appear; the validators refuse a file that mixes two. A family sub-block carries that family's scientific declarations. For example, the background file `example_yamls/baosn_hubble_emulator.yaml` declares

```
data:
  grid:
    quantity: Hubble
    units:    km/s/Mpc
    law:      log_offset
    offset:   0.0
```

naming the emulated function, its units and the target law the artifact will persist.

The second decision is the model and its objective, inside `train_args`:

```
train_args:
  nepochs: 300
  bs:      256
  model:
    name: resmlp
    mlp:
      width:    64
      n_blocks: 3
    activation:
      type:    H
      n_gates: 3
  loss:
    mode: sqrt
```

`model.name` selects the design class, the `mlp` sub-block sizes the trunk, `activation` selects the learnable activation family and `loss.mode` the objective transform. A scalar output has no ordered coordinate axis, so this family accepts only the trunk-only `resmlp`; the conv and transformer heads are refused with a message naming the rule.

The third decision is the optimizer schedule, in the same block:

```
  optimizer:
    weight_decay: 0.0
  lr:
    lr_base:       0.001
    bs_base:       256.0
    warmup_epochs: 5
  scheduler:
    mode:     min
    patience: 25
    factor:   0.8
  trim:
```

```
    start:          0.0
    end:            0.0
    hold_epochs:    0
    anneal_epochs:  1
    shape:          cosine
  focus:
    start:          0.0
    end:            0.0
    hold_epochs:    0
    anneal_epochs:  1
    shape:          linear
    kappa:          0.15
```

`lr_base` is the learning rate at the reference batch size `bs_base`; a different `bs` rescales it by the square-root rule. The scheduler block configures the plateau scheduler. The `trim` and `focus` blocks are required even when both are off (every value `0.0`, as here), because the resolver persists the values it consumed and refuses to invent a code default for a missing block.

A sweep file is the same training file plus one top-level block naming the single setting to vary and the values to try. The shipped `example_yamls/cosmic_shear_sweep_hyperparam_emulator.yaml` ends with

```
sweep:
  parameter: lr.lr_base
  values:
    - 0.0010
    - 0.0025
    - 0.0063
    - 0.0160
```

where `parameter` is a dotted path into `train_args`. Exactly one `sweep` block may be active; YAML silently keeps only the last copy of a duplicated key, so a second block does not error, it wins.

Each shipped file annotates every key it uses, and the example-YAML guide (`example_yamls/README.md`) is the complete per-key reference, including the optional guard blocks (`clip`, `rewind`, `ema`) and the fine-tune, transfer and search variants this subsection does not repeat.

### E. Raw configuration and resolved configuration

The dictionary returned by the YAML parser is the *raw configuration*, which precedes execution. Resolution performs explicit transformations:

1. verify that allowed keys are spelled correctly and required keys are present.
2. reject values with the wrong type or physical domain.
3. replace a search range by its declared default in a normal training run.
4. select class objects for the requested geometry, model, activation, loss, optimizer and scheduler.
5. compute dependent facts such as input width, output width and the batch-scaled learning rate.
6. record the values that will actually be consumed.

A search leaf can be written as

```
lr_base: [0.0025, 0.0001, 0.01, log]
```

The four entries mean, in order, default value, lower bound, upper bound and proposal scale. In a normal run, the resolver chooses `0.0025`. In a hyperparameter search, the study proposes a value between the two bounds on a logarithmic scale. The anonymous positions are therefore not interchangeable. Each has a named meaning supplied by the schema.

Resolved configuration is persisted because a saved emulator must be rebuilt from the facts that created it. Defaults in a future checkout have no authority over reconstruction. Operational facts such as the number of GPUs or whether progress printing was quiet can be reported separately. They do not change the mathematical model.

*a. Safe use of the current configuration surface.* Family validators reject unknown keys inside blocks such as `data.grid` and `data.cmb`. The top level and the root of `train_args` do not yet have complete key censuses. A misspelled feature key can select a default branch, and some root controls still pass through Python truth-value rules. Start from a shipped YAML file, change only keys documented for that driver and inspect the resolved run record before an expensive campaign. Native booleans must remain unquoted. The implemented numeric validators already reject nonfinite optimizer controls, but that protection does not make every configuration namespace total.

### F. Factories turn specifications into live objects

After validation, constructible components are represented by specification dictionaries. Their `cls` entry is a Python class object and their other entries are constructor keyword arguments. A factory performs three steps:

1. copy the specification so the caller's dictionary is not changed in place.
2. remove `cls` and inject values known only at run time.
3. call the class once to create a fresh instance.

For a model, injected values include encoded input width, target width, dtype and device. For an optimizer they include the model parameters and the batch-scaled learning rate. For a scheduler they include the optimizer it will control.

The specification contains a class object. Each construction produces a fresh module instance, which prevents two training runs from sharing parameters or state. Similarly, activation and normalization entries are factories called once per layer. Two layers may use the same class and constructor values while still owning distinct learnable tensors.

### G. Precedence is part of the public behavior

A value can appear at more than one level. The code therefore needs an explicit precedence rule: a statement of which value wins. The important cases are:

- a phase-specific trunk or head block replaces the corresponding top-level optimizer, learning-rate, scheduler or EMA block.
- a head-specific activation overrides the model-wide activation only for that head.
- a one-knob sweep deep-copies the base configuration and changes the one named leaf for that run.
- a hyperparameter search proposes only leaves written in search-range form. All scalar leaves remain fixed.

Replacing a mapping and merging a mapping are different mechanics. A replace operation discards unspecified fields from the parent. A merge inherits them. The resolver chooses one behavior per block and records the result so a reader does not need to reconstruct inheritance from several YAML locations.

### H. Validation precedes expensive work

Validation is a program boundary that converts untrusted text into a smaller set of values the numerical code promises to understand, with typed errors for values outside that set. If a value has not passed that boundary, later code must not infer its meaning through Python coercion, framework defaults or truthiness.

The required safe execution order has five named states. Table III names the Python object that should be present at each state and the expensive action that remains forbidden until that state succeeds. The table is the total-boundary contract. The current alpha implementation still performs some checks later, as described directly after the table.

*a. Validation order in the current constructor.* The constructor selects a device and reads part of the covariance header before the whole recipe reaches one total validator. The ordinary `run` path also stages sources and builds geometries before `train` calls `build_specs`, where some model and optimizer keys are rejected. The program therefore has many real validators but does not yet provide the full early barrier in Table III. Review a recipe with the family validator and a dry, small-data construction before committing a large staging or accelerator run. A late configuration error can occur after file I/O or device selection.

#### *1. A type error that looks superficially reasonable*

Suppose a YAML file contains

```
train_args:
  bs: "256"
  rewind: "false"
```

Both values are strings because quotation marks request YAML text. The batch size must be an integer and `rewind` must be a boolean. Calling `int("256")` would make the first value usable, but it would also hide a schema error and make the public type contract depend on a coercion. The second value is more dangerous: `bool("false")` is `True` because every nonempty Python string is truthy. Coercion would therefore reverse the user's apparent intent.

A total boundary rejects both values while it still holds only a small mapping. It should not open a multi-gigabyte target dump merely to discover later that the batch size was text. The error names the dotted key, received type, required type and accepted domain. Correct input is

```
train_args:
  bs: 256
  rewind: false
```

where YAML constructs an integer and a boolean respectively.

#### *2. Finite and ordered are separate questions*

A floating-point value can have the correct Python type and still be invalid. For a learning rate $\eta$, validation applies at least three checks in order:

1. Confirm that the object is a real number and exclude booleans. Python booleans are a subclass of integers, so `isinstance(True, int)` alone is insufficient.
2. Confirm that the value is finite. NaN and positive or negative infinity must be rejected before comparisons are used as control flow.
3. Confirm that it lies in the declared physical or algorithmic domain, for example $\eta > 0$.

NaN makes the comparison `eta <= 0` false, so that comparison alone accepts it. A separate finiteness test must run before the domain test.

| State | Object before the step | Operation owned by the step | Object after the step | Work that must not have happened yet |
|---|---|---|---|---|
| 1 | Shell character strings | Strict command-line parsing | A namespace of named strings and flags | No YAML read. No array opened |
| 2 | Parsed path components | Absolute-path resolution and YAML parsing | A raw nested mapping of Python strings, numbers, booleans, lists and mappings | No training row selected. No CUDA or MPS query needed |
| 3 | Raw mapping | Key census, required-field checks, type checks, domain checks and class lookup | A validated construction recipe | No data-vector payload opened. No model allocated |
| 4 | Validated recipe | Resolution of defaults, inheritance, dependent widths and explicit paths | A resolved record containing the values that will be consumed | No fitted geometry and no optimizer state |
| 5 | Resolved record | File opening, row-identity checks, cuts and staging | Host-side source dictionaries | No device minibatch and no parameter gradient |

TABLE III. The required validation ladder. Each row is one state transition. The operations run in the stated row order. "Raw mapping" means the direct YAML parse. "validated recipe" means every accepted key and value has an assigned meaning. "resolved record" means defaults and precedence have been replaced by the exact values later constructors consume. Host-side means ordinary CPU memory or a disk-backed memory map. This table states the target ordering against which the current alpha implementation is audited.

#### *3. Validation and resolution perform different operations*

Validation decides whether a supplied value is permitted. Resolution selects the permitted value that this run will consume. Consider a phase-specific learning-rate block. Validation checks both the top-level and phase-specific mappings independently. Resolution then applies the declared precedence rule and stores one resulting learning rate for that phase. The artifact records the result. A future reader receives one resolved value with no pair left to adjudicate.

Likewise, a specification mapping can contain a Python class object under `cls`. Schema validation records that class and postpones construction. The factory waits until the encoded input and output widths are known. This separation prevents a constructor from allocating parameters before an unrelated invalid scheduler or file declaration has been refused.

#### *4. A valid recipe, stopped at the boundary*

For the running example, assume validation accepts $n_{\text{train}} = 3$, $n_{\text{val}} = 1$, $B = 2$, three declared parameter names, a target-file path and requested kept columns $[0, 2, 4]$. Before opening those files, resolution can establish several recipe facts:

- the final minibatch may contain one row because three is not divisible by two.
- every relative file name has become one absolute path.

The requested name and column lists suggest $P = 3$ and $K = 3$, but only file headers and row checks can prove that the actual parameter table has three usable columns, the dump width is $D = 5$ and every kept column exists. The numerical means $\mu_C$ and $\mu_V$ are also unknown because computing them requires selected training rows. Under the required ordering, no object yet has dtype `torch.float32`. No `nn.Parameter` exists. No optimizer has moment buffers and no GPU memory has been allocated. This "nothing numerical has happened" statement is the positive invariant the total boundary must eventually enforce.

At the end of the boundary, the program has a validated and resolved recipe. It can now open the named files, prove that paired files share row identity, select the seeded rows and only then fit the geometries introduced in the next sections.

## III. TRAINING CAMPAIGNS: ONE RUN, SWEEPS, SEARCHES AND BAKE-OFFS

The experiment engine trains one resolved configuration. Campaign drivers call that engine repeatedly to answer comparative questions. Every point is a complete independent training unless the driver explicitly documents shared study state.

### A. One training run

A single-run driver is the simplest scientific unit. It resolves one YAML, selects one seeded data subset, constructs one model and writes one artifact pair plus histories. Its output records the function learned by this exact recipe.

The optional diagnostic flag adds reporting after training. It does not change the objective or choose a different best epoch. Expensive family diagnostics may have their own bounded-data policy so creating a report does materialize tensors only up to the width used in training.

### B. Training-size learning curve

An $N_{\mathrm{train}}$ sweep trains the same resolved model at several absolute row counts. Each point applies physical cuts first and then chooses its requested number of rows. The horizontal axis is data volume. The vertical axis is a fixed validation failure statistic such as the fraction with $\Delta\chi^2 > 0.2$.

A curve still falling at the largest point says that additional simulations are likely useful. A flat tail suggests that architecture, objective or optimization is limiting accuracy. A rising point may indicate optimization instability, a subset-identity error or ordinary statistical noise. Repeated seeds distinguish them.

Every point fits its own data-dependent geometry and analytic base from exactly its selected rows. Borrowing geometry or a PCE fit from the largest point leaks information and invalidates the training-size effect measurement.

*a. Safe use of the training-size wrappers.* The scalar, CMB, background and matter-power wrappers accept either an active `param_cuts` block or no block at all. With no cuts, the available pool is the full named parameter table. With active cuts, the pool counter and staging use the same named columns and physical bounds. Consequently the reported pool is the exact largest legal training size, and a request for one row more is refused before training starts.

### C. One-knob sweep

A one-knob sweep deep-copies the resolved base recipe and changes one named leaf for each run. Examples include learning rate, batch size, convolution kernel width or FiLM. Deep copy means nested mappings are duplicated. A shallow copy would let one point mutate later points.

The dotted path identifies the leaf. Intermediate blocks may be constructed only where the schema defines that behavior. An unknown first segment or a phase-specific path on an architecture without that phase is refused. The results table records the actual value and fixed context so its curve can be reconstructed without reading a mutable YAML later.

### D. Hyperparameter search

A search-range leaf has four named positions:

$$[\mathrm{default}, \mathrm{minimum}, \mathrm{maximum}, \mathrm{kind}].$$

Kind is integer, linear float or logarithmic float. Optuna proposes values only for leaves written in this form. All scalar leaves stay fixed and ordinary non-search drivers use the default.

The objective function returns the declared validation statistic from one complete trial. The statistic is read from the trial's published selection record — the goal-threshold fraction of exactly the model the run restored, with the record's median stashed for the tie-break report — so the score is correct even when no trained epoch improved on the incoming weights. Trial status is part of the result: a failed trial carries failure status without a finite score or completion record.

*a. Safe use of a parallel study.* The parent process can report the best historical trial without proving that a current worker exited successfully or that the current invocation added a completed trial. Its journal also lacks a complete scientific manifest. After every parallel invocation, inspect each worker's return status and the before-and-after count of completed trials. Resume an existing journal only after manually comparing the objective, fixed configuration, search ranges, data identities, family laws and implementation. Start a new journal when any of those facts changes.

### E. Activation bake-off

The activation bake-off nests two comparisons. For each activation family it trains several $N_{\mathrm{train}}$ points, then overlays the learning curves. The architecture, seed policy, data, optimizer and evaluation statistic stay fixed.

A family can win at small data and tie later or cross another family. The complete curves are more informative than ranking one final point. The bake-off also applies head-liveness checks so a strong trunk cannot make an inactive activation/head combination appear competitive.

### F. Parallel execution and GPU packing

Campaign points are independent jobs. Each worker owns one complete model training on one selected device. Training-size points can be scheduled largest-first to reduce the long tail. Equal-cost one-knob points can use round-robin assignment.

GPU packing allows several small jobs to share a large card. A preflight estimate converts model, batch, activation-storage and I/O needs into a fraction of device memory. Tokens limit concurrent jobs so their total estimate stays inside policy. Packing improves throughput but makes per-epoch timing noisier, so timing comparisons run without it.

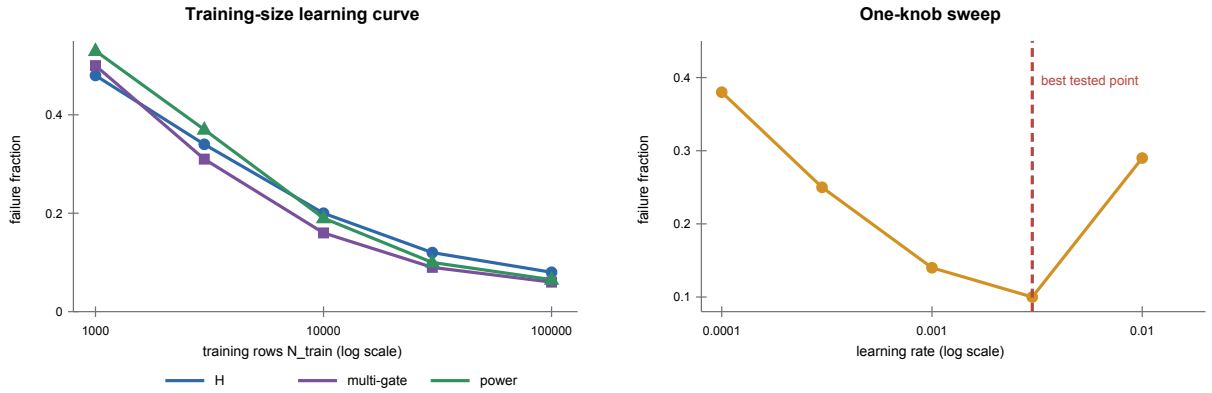



FIG. 2. How campaign curves are read. The illustrative activation curves compare data efficiency under fixed context. The one-knob curve changes only the learning rate. Its minimum does not become a universal default outside that recorded context.

*a. Safe use of pooled GPU campaigns.* The shared GPU pool can wait indefinitely when one live worker never produces its result, and it does not yet validate every token or lane assignment before launch. The activation bake-off has a second exposure: setup and staging run outside its worker exception handler, while the parent waits for a fixed count of queue results. A failure before the first epoch can therefore leave either parent waiting. Run pooled campaigns under an external timeout, retain every worker log and require one explicit success record per requested point before using the result table.

## IV. GENERATING THE COSMOLOGIES AND TARGETS

Training cannot begin until an external physics calculation has produced paired cosmologies and targets. The programs under `compute_data_vectors` own that earlier stage. A shared generator core owns sampling, MPI work distribution, checkpointing and parameter sidecars. Each thin family driver states which physical quantities to request and which array files to write.

### A. The training distribution is wider than the posterior

An emulator used inside inference must remain accurate where the sampler may walk, including the edges around the posterior. Training only on the posterior center would teach the network densely where calls are common but leave no support just outside that region. The generator therefore offers a tempered distribution

$$\log \widetilde{p}_{T_{\mathrm{temp}}}(\boldsymbol{\theta}) = \frac{1}{T_{\mathrm{temp}}}\Big[-\frac{1}{2}(\boldsymbol{\theta}-\boldsymbol{\theta}_0)^{\mathsf{T}} \widetilde{\Sigma}^{-1}(\boldsymbol{\theta}-\boldsymbol{\theta}_0) + \log \pi(\boldsymbol{\theta})\Big]. \tag{1}$$

$\boldsymbol{\theta}_0$ is the fiducial cosmology, $\widetilde{\Sigma}$ is the proposal covariance after the declared correlation treatment, $\pi$ is the prior and $T_{\mathrm{temp}}$ is the dimensionless tempering factor. The tilde over $p$ says that the right-hand side is an unnormalized density: an additive constant needed to make its integral one is omitted because the sampler needs only probability ratios. The implementation divides both the Gaussian term and the log prior by $T_{\mathrm{temp}}$. Where the prior is locally flat, this changes the Gaussian factor as if its covariance were $T_{\mathrm{temp}}\widetilde{\Sigma}$. A larger factor then makes a wider cloud. That covariance statement is exact only where the prior is locally flat. The subscript distinguishes this scalar from the target tensor $T$ used later.

The maximum-correlation control clips very strong off-diagonal correlations in the proposal covariance. Geometrically, an unmodified Fisher covariance can form a thin tilted sheet. Filling only that sheet gives poor coverage in directions perpendicular to the degeneracy. Reducing the correlation fattens the cloud so the training set occupies a volume with support away from the nearly one-dimensional ridge.

A uniform mode draws throughout a temperature-expanded hard box. It supports coverage stress tests. Gaussian or otherwise unbounded priors still need a declared rule for turning their scale into finite box boundaries.

### B. Sampling and MPI work distribution

In the tempered mode, an ensemble sampler proposes cosmologies. An *ensemble* is a set of walkers whose proposal uses the positions of other walkers. Differential-evolution and snooker moves explore elongated correlations without requiring a separate hand-tuned step size for every parameter. The resulting chain is reduced to the requested distinct rows, and its autocorrelation estimate is reported.

MPI, the Message Passing Interface, distributes expen-

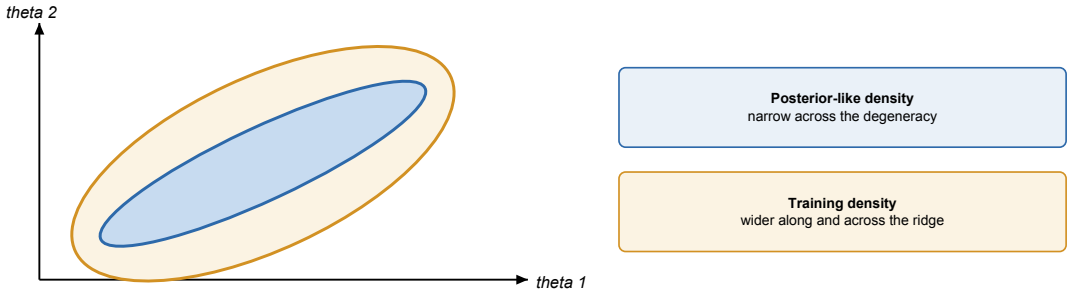



FIG. 3. Conceptual two-parameter coverage. Temperature widens the proposal. Correlation control fills directions across the posterior degeneracy. Validation belongs inside the trained support so its score measures interpolation within that support.

sive physics calls across processes. One master process owns the row queue and output coordinates. A worker receives one complete cosmology, asks the family truth code for its outputs and returns a payload plus an acceptance verdict. The master process is the sole writer of the final array. It places each returned payload into the slot belonging to the parameter row that was sent.

The family truth codes differ:

- cosmic shear calls CosmoLike, the external cosmological-likelihood code that evaluates the survey's two-point data vector and writes one masked data-vector dump.
- CMB calls the Code for Anisotropies in the Microwave Background (CAMB) and writes TT, TE, EE and lensing-potential spectra.
- background calls CAMB for $H(z)$ and distance quantities on a named redshift axis.
- matter power calls CAMB for linear power and boost surfaces on named $z$ and $k$ axes, with any analytic-base companions.

### C. A checkpoint is a coordinated file set

Generation is expensive, so the master periodically checkpoints. A valid checkpoint is a coordinated set containing the parameter rows, parameter names, covariance and ranges, family payloads, coordinate sidecars and failure flags. Resume must prove that these files describe the same generation identity and completed-row set.

For a rejected physics evaluation, the verdict must remain false until the payload has been validated in its storage dtype and written successfully. Writing zeros or stale values and clearing the failure flag would turn an external failure into a training example. Before training, the file-set contract therefore requires every row to have a current successful verdict, the payload shape expected by its family and finite values after conversion to the on-disk dtype.

Publication of several files is a transaction in the conceptual sense: readers must see either the complete old checkpoint or the complete new one, and a mixed checkpoint is invalid. A manifest records file identities, coordinate axes, row counts, dtypes and digests. A mismatch blocks resume and preserves the existing roots. A load exception cannot authorize a fresh run at those roots.

*a. Safe use of generated file sets.* The generators do not yet certify this complete relation. A run can retain failed-row flags and still return success, while training does not treat the failure sidecar as a mandatory readiness verdict. Some checkpoint sets omit parameter-name or coordinate sidecars. A checkpoint-load exception can also restart generation at the same output roots. Before training, require an empty failure set, compare row counts across every sibling file and verify the parameter names and physical axes directly. Preserve the existing roots when checkpoint loading fails. File existence alone supplies no readiness verdict.

### D. Sidecars carry scientific meaning

The parameter text file contains bookkeeping columns as well as sampled parameters. Its parameter-name sidecar states which columns are physical inputs. The covariance header provides an independent order declaration. The trainer must resolve columns by name. A fixed slice between two bookkeeping columns has no naming authority.

Axis sidecars are equally important. A CMB array of width 999 could represent $\ell = 2, \ldots, 1000$ or $\ell = 10, \ldots, 1008$. Width equality does not distinguish them. Background and matter-power files likewise need their actual redshift and wavenumber values. The artifact can then state that its axis was verified against the generated dump. That dump is the axis source, while the unrelated covariance has no authority over it.

*a. Safe use of parameter display labels.* Cobaya permits a sampled parameter without a `latex` display label, but the generator currently reads that key unconditionally after sampling and chain publication. A missing la-

bel can therefore raise `KeyError` late in an otherwise valid run. Give every sampled parameter an explicit `latex` value. The numerical parameter name is a clear fallback for a plain-text label, but the writer does not yet apply that fallback itself.

### E. The handoff to training

Generation ends with an immutable row relation:

$$\text{parameter row } i \longleftrightarrow \text{accepted physics payload } i.$$

The training YAML names the completed files. Its `train_args` block belongs to the trainer. The generator's block of the same name configures the external physics model, while the trainer configures a neural approximation. The next section begins where the generator stops. It opens the file set, revalidates names, axes, rows and failure state and selects a reproducible training subset without breaking the row relation.

## V. FROM COSMOLOGIES AND DATA VECTORS TO A STAGED TRAINING SOURCE

Suppose the data-generation calculation has finished. Its output is a table of cosmologies, a list that names the cosmological parameters and one data vector for each cosmology. The emulator begins with separate control and numerical paths. The path. The control path first reads the YAML file, resolves its file names, checks which emulator family was requested and chooses the corresponding model class. No cosmological row has been transformed yet. The first numerical operation is *staging*: the code aligns the parameter table with the data-vector dump, selects a reproducible subset of valid rows and decides whether that subset lives in ordinary memory or remains backed by a file on disk.

The execution order below follows that process and stops just before the code constructs the whitening geometries. Section VI begins with the staged arrays and derives the parameter-space and data-vector transforms.

### A. The four files that must agree

For an ordinary data-vector emulator, one training source is described by four files. The validation source has the same structure but different rows.

Let $N$ be the number of generated cosmologies, $P$ the number of modeled parameters and $D$ the length of one physical data vector. The two arrays that matter numerically are

$$C = \begin{bmatrix} \boldsymbol{\theta}_0^{\mathsf{T}} \\ \boldsymbol{\theta}_1^{\mathsf{T}} \\ \vdots \\ \boldsymbol{\theta}_{N-1}^{\mathsf{T}} \end{bmatrix} \in \mathbb{R}^{N\times P}, \qquad V = \begin{bmatrix} \mathbf{d}_0^{\mathsf{T}} \\ \mathbf{d}_1^{\mathsf{T}} \\ \vdots \\ \mathbf{d}_{N-1}^{\mathsf{T}} \end{bmatrix} \in \mathbb{R}^{N\times D}. \tag{2}$$

Row $i$ of $C$ and row $i$ of $V$ must describe the same cosmology. This row identity is the central invariant of staging. A network trained on $C_i$ paired with $V_j$ for $i \neq j$ learns the wrong function.

*a. The current parameter-table layout.* The non-scalar loader currently reads the complete text table and then uses the positional slice `[:, 2:-1]`. In other words, it assumes that the first two columns are the sample weight and minus log posterior, the modeled parameters occupy the middle columns and the last column holds bookkeeping. The `.paramnames` file is checked against the covariance header when it is present, but the current slice still follows fixed positions. This rule belongs to the current loader. The mathematical construction requires row identity and named columns. A table with extra derived columns in the middle can pass the name comparison and still violate the positional assumption.

The scalar-family loader is different. Its inputs and targets are both columns of the parameter table, so it resolves the requested input and output names explicitly through the `.paramnames` sidecar.

### B. Control path: resolving a run before touching the rows

The single-run driver is `cosmic_shear_train_emulator.py`. Its family siblings call the same package layer. Table V shows the startup chain as changes of program state. The left object in a row already exists. The owner in the middle validates or constructs the object on the right. A row does not silently perform work assigned to a later row.

The path resolver reads the environment variable `ROOTDIR`. It joins the project name and the `chains` directory to each relative data-file name. An absolute path remains absolute. This step changes strings in a Python dictionary. It does not load the large arrays.

For example, suppose the shell supplies

```
ROOTDIR=/data/cocoa
--root projects/lsst_y1
--fileroot emulators/training_scripts
--yaml cosmic_shear_train_emulator.yaml
```

The YAML may name `train_dv: train.npy`. The resolver constructs the training-vector path under the project's `chains` directory. The resolver leaves the process working directory unchanged. That directory cannot change the meaning of `train.npy` halfway through a

| File | Contents | Why the code needs it |
|---|---|---|
| Parameter table, `*.txt` | One row per cosmology | Supplies the input parameters $\boldsymbol{\theta}_i$. |
| Parameter names, `*.paramnames` | One declaration per named sampled or derived parameter | States physical parameter identity. It does not name leading weight and log-posterior bookkeeping columns in the numerical chain table. |
| Parameter covariance, `*.covmat` | A square covariance matrix with a name header | Supplies a second declaration of the input order and later defines the input whitening transform. |
| Data-vector dump, `*.npy` | One vector $\mathbf{d}_i$ per cosmology | Supplies the targets that the network will learn. |

TABLE IV. The four files forming one ordinary training source. Their row and column declarations must agree before selection. Equal array sizes establish shape compatibility alone. Sidecars establish whether parameter column $j$ or data-vector row $i$ names the same physical quantity in every file.

| Step | Object before the call | Code owner | Object after the call |
|---|---|---|---|
| 1 | Shell strings such as `-root`, `-fileroot` and `-yaml` | The family driver and its strict argument parser | Named Python strings. Unknown command-line options have already raised. No YAML or array is open. |
| 2 | One YAML file name plus the Cocoa roots | `resolve_cocoa_config` | A nested Python mapping whose declared file paths are absolute. Values remain ordinary Python values. This step does not create tensors. |
| 3 | The resolved mapping | `EmulatorExperiment.from_config` | One `EmulatorExperiment` with exactly one output family, one model recipe, a device policy, validated run controls and empty slots for arrays, geometries and a model. |
| 4 | The experiment plus resolved training and validation file names | `stage_train`, then `stage_val` | Two host-side source dictionaries. Their arrays are NumPy objects or read-only file mappings. They are not yet CUDA or MPS tensors. |
| 5 | The staged training source and the declared covariance or target law | `build_geometry` | Reversible parameter and output transforms plus the physical loss. No optimizer step has occurred. |
| 6 | Staged sources, geometries and validated model/training recipes | `train`, reached through `exp.run()` | Loaders, model, optimizer, scheduler and histories. The experiment now owns the trained state that `save_emulator` may publish. |

TABLE V. Startup control path. `exp.run()` calls the named stages in order, with path parsing, array opening, geometry fitting and training owned by separate stages.

run. The resolved mapping is a copy of the run description with path values replaced. The file contents remain unopened until staging.

The split gives an error a useful time and place. A misspelled configuration key fails while the objects are small. A missing data file fails before GPU allocation. A row-count or shape mismatch fails while staging the matching arrays. The loss cannot hide any of these errors because it does not exist yet.

`EmulatorExperiment.from_config` is an alternative constructor. A `classmethod` receives the class as its first argument, conventionally named `cls` and returns a newly constructed instance. It validates the configuration and chooses one branch: cosmic shear, scalar outputs, a CMB spectrum, a one-dimensional background grid or a two-dimensional matter-power grid. The ordinary constructor then stores cheap state: parameter names, the selected compute device, the logger and empty slots for objects that do not exist yet.

The important empty slots are

This delayed construction matters in parameter sweeps. A sweep can keep the validation source fixed while replacing the training subset or keep both sources fixed while rebuilding only a model. Expensive state is therefore created after construction by named methods.

### C. Numerical path: opening the source

`stage_train` creates a local random-number generator from the YAML value `split_seed`, then calls `load_source` in `emulator/data_staging.py`. The generator is local state. It is passed as an argument, so row selection does not depend on a module-level random state left behind by an earlier call.

The two large inputs are opened differently:

```
dv = np.load(dv_path,
             mmap_mode="r",
```

| Attribute | Meaning before and after the corresponding method runs |
|---|---|
| `train_set` | `None` initially. A staged training-source dictionary after `stage_train`. |
| `val_set` | `None` initially. A staged validation-source dictionary after `stage_val`. |
| `pgeom` | `None` initially. The parameter transform after `build_geometry`. |
| `geom` | `None` initially. The output transform after `build_geometry`. |
| `chi2fn` | `None` initially. The loss wrapper after `build_geometry`. |
| `model` | `None` initially. The trained PyTorch module after `train`. |

TABLE VI. State slots owned by `EmulatorExperiment`. `None` marks the absent-object state before the corresponding lifecycle method produces its object. Valid empty geometries, losses or models require concrete objects. Named transitions make object lifetime visible to drivers and sweeps.

```
            allow_pickle=False)
C_all = np.loadtxt(params_path,
                   dtype="float32")
C = C_all[:, param_cols]
```

`np.loadtxt` is eager: it parses the complete parameter table and creates a new in-memory NumPy array. The requested dtype is `float32`, four bytes per element. The slice that follows selects the modeled columns.

`np.load(..., mmap_mode="r")` is lazy with respect to the payload. It creates an `np.memmap`, an array-like object whose elements remain in the `.npy` file until a later indexing operation asks the operating system to read them. The mode `"r"` makes this mapping read-only. The code can inspect `dv.shape` without loading $ND$ floating-point values.

At this point the data still live in ordinary CPU memory or disk-backed virtual memory on the host. Device placement occurs later, after the geometry exists and the batching code knows which rows a particular minibatch needs.

#### 1. *The first row-identity check*

The loader immediately compares the row counts:

$$C.\text{shape}[0] = V.\text{shape}[0]. \tag{3}$$

If the counts differ, it raises an error that names both files and both counts. Equality proves that the files have the same number of rows. Row association requires separate identity evidence. The generator must preserve that ordering when it writes the pair.

### D. A seeded permutation, followed by physical cuts

The loader next creates a permutation of the integers $0, \ldots, N-1$:

```
order_tensor = torch.randperm(
    n,
    generator=gen)
order = order_tensor.numpy()
```

`torch.randperm` returns a one-dimensional PyTorch tensor containing every integer exactly once in pseudorandom order. It eagerly materializes all $N$ indices. The call to `.numpy()` exposes the CPU tensor as a NumPy array. Because both objects use CPU storage, this conversion normally exposes the same index memory to both objects.

The physical-window function receives $C$, the permutation and the ordered parameter names. Each active window locates its columns by name and computes one derived quantity per candidate row:

$$h \equiv \frac{H_0}{100\,\mathrm{km\,s^{-1}\,Mpc^{-1}}}, \tag{4}$$

$$\omega_b \equiv \Omega_b h^2, \tag{5}$$

$$q_{\texttt{omegam2h2}} \equiv (\Omega_m h)^2, \tag{6}$$

$$\omega_m \equiv \Omega_m h^2, \tag{7}$$

$$q_{\texttt{omegamns}} \equiv \omega_m n_s. \tag{8}$$

The symbol $\omega_m$ is the usual physical matter density $\Omega_m h^2$. The symbol $n_s$ is the dimensionless scalar spectral index: it describes the tilt of the primordial curvature-power spectrum relative to a scale-invariant spectrum. It is a cosmological parameter column. The row axis supplies the number of samples. The separately named configuration quantity `omegam2h2` is $(\Omega_m h)^2 = \Omega_m^2 h^2$ and is therefore not another spelling of $\omega_m$. Naming both prevents a factor of $\Omega_m$ from disappearing in a cut review. For an active lower bound $a$ and upper bound $b$, the comparison is strict: $a < q_i < b$. The code builds a Boolean mask for each window and intersects it with the masks already applied. A Boolean mask is an array of `True` and `False` values, one per candidate row. Indexing `order[mask]` uses NumPy advanced indexing and creates a new array of the surviving integer indices. Their order remains the shuffled order.

The result, called `phys`, is a list of row coordinates into the original files. Each coordinate identifies where one selected cosmology remains:

$$\texttt{phys} = [i_0, i_1, \ldots],$$
$$C[i_k] \text{ and } V[i_k] \text{ remain a matched pair.}$$

### E. Selecting an absolute number of rows

The YAML states an absolute number of training rows, `n_train` and an absolute number of validation rows, `n_val`. After the cuts, the loader checks that the requested number is at least one and that the physical pool is large enough. It then takes

```
idx = phys[:n_keep]
```

The slice is the first $n_{\text{keep}}$ entries of the seeded, cut permutation. Using the same seed at multiple training sizes creates nested learning-curve subsets: the 2,000-row set is the prefix of the 4,000-row set. Both sizes derive from the same seeded draw.

Training and validation use different files. Reusing the numerical seed therefore repeats the selection algorithm. The cosmologies remain the records of their separate files. The scientific assumption is that the two file sets are disjoint. That assumption must be checked from file identity or row identity. A seed does not establish disjointness.

### F. Staging: copy the subset or retain the disk mapping

The word *stage* means "put the selected rows in the storage form from which later batches will read them." It does not yet mean "move them to the GPU." The helper `stage_source` estimates whether the selected rows fit within a configured fraction of available host memory. The estimate adds three allocations: the compact parameter array, the compact data-vector array and the integer reindex array. Each numerical term uses the dtype and column count of the stored array it describes. The branch decision uses all three terms. The current status line prints the parameter, data-vector and index contributions separately, followed by their total, the available-memory budget, the exact comparison and the chosen resident or disk branch. The worked calculation below derives those same three contributions so the banner exposes its inputs and branch decision for an independent check.

There are two outcomes.

#### 1. Resident host-memory outcome

If the pair fits, the helper gathers the sorted unique selected rows into compact arrays. Advanced integer indexing is eager and returns a copy. The compact arrays have shapes

$$C_{\text{src}} \in \mathbb{R}^{n_u \times P}, \qquad V_{\text{src}} \in \mathbb{R}^{n_u \times D}, \tag{9}$$

where $n_u$ is the number of distinct selected rows. The source's `idx` is then expressed in the compact arrays' local row coordinates. If original dump row 91 becomes compact row 3, later code asks the resident array for row 3. Row 91 remains its original disk coordinate.

#### 2. Disk-backed outcome

If the data vectors do not fit, the helper retains the original read-only `np.memmap`. The source's `idx` remains in global on-disk row coordinates. A later batch read such as `dv[idx]` asks the operating system for only those rows. This advanced indexing operation materializes only the requested batch. The complete dump remains a disk-backed memory map.

The parameter table was already loaded eagerly by `np.loadtxt`. Thus, "disk-backed source" describes the data vectors. Other objects in the source dictionary retain their own storage policies.

### G. The source dictionary and its coordinate systems

`load_source` returns a small dictionary:

```
dump_rows = sorted_unique_original_rows
src = {"C":         C_src,
       "dv":        dv_src,
       "idx":       idx_src,
       "dump_rows": dump_rows}
```

The fields have distinct jobs.

`dump_rows` is needed by the two-dimensional matter-power path. That path has a second, row-aligned base dump. Even when staging has converted the primary arrays to local coordinates, the sibling file still needs the original disk row numbers. Sorting makes its reads sequential. Applying `np.unique` removes duplicates and also sorts. It returns a new array.

Figure 4 shows the branch with a concrete selection. The selected rows are unique because they are a prefix of a permutation. The order $[9, 2, 5]$ is deliberately not sorted: it lets the figure distinguish training-selection order from efficient disk-read order.

### H. Training statistics are owned by the training source

For the training source, `with_means=True`. The loader computes the mean of every parameter column and every staged data-vector column:

$$\bar{C}_j = \frac{1}{N_{\text{tr}}} \sum_{i=1}^{N_{\text{tr}}} C_{ij}, \qquad \bar{V}_k = \frac{1}{N_{\text{tr}}} \sum_{i=1}^{N_{\text{tr}}} V_{ik}. \tag{10}$$

Here $N_{\text{tr}}$ is the number of selected training cosmologies. The parameter index $j$ runs over raw parameter columns. The target index $k$ runs over data-vector columns. Validation rows are not included in either sum: using them would let the held-out set influence the coordinate system used to judge it. These arrays are stored as `C_mean` and `dv_mean`. The validation source uses `with_means=False`.

| Key | Coordinate system | Meaning |
|---|---|---|
| `C` | Matches `idx` | Parameter rows. It is compact in the resident outcome and the full eager table in the disk-backed outcome. |
| `dv` | Matches `idx` | Data-vector rows. It is compact in the resident outcome and a file mapping in the disk-backed outcome. |
| `idx` | Local or global | The row coordinates that consumers must use with this particular `C`/`dv` pair. |
| `dump_rows` | Always global | Sorted unique row numbers in the original files. A sibling dump uses these coordinates to retrieve the same cosmologies. |

TABLE VII. Keys in a staged source dictionary. "Local" coordinates address a compact resident copy. "global" coordinates address original disk rows. Consumers index `C` and `dv` only with the accompanying `idx`. `dump_rows` exists for a separate sibling file that still uses the original coordinate system.

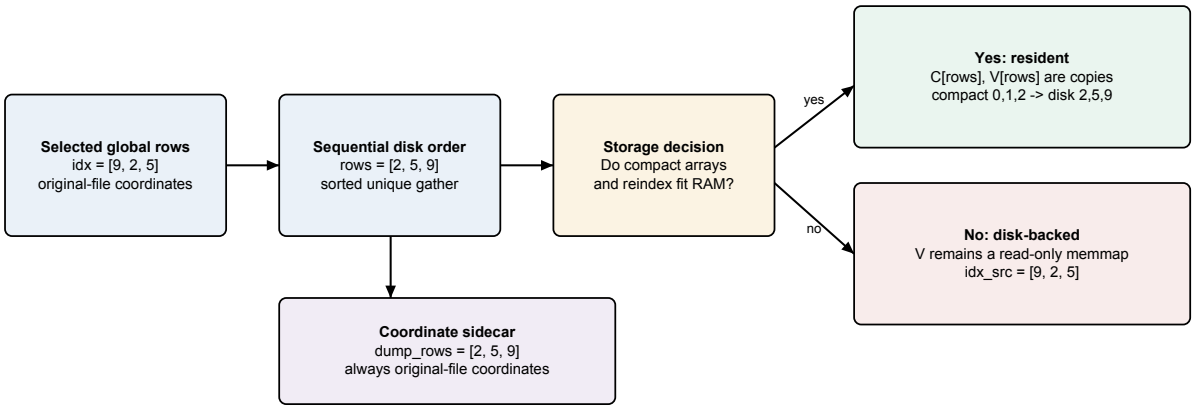



FIG. 4. One selection, two storage representations and two coordinate systems. A global row addresses the original files. A local row addresses a compact copy. `idx_src` must be interpreted with the returned `C`/`dv` pair. `dump_rows` never becomes local because the grid2d base file is addressed in original-file coordinates.

It does not estimate a second zero point. Training and validation must be expressed in the same coordinate system, so the training-derived geometry transforms both.

Subtracting a center does not change a residual-based loss. If prediction and truth are both centered by $\bar{V}$, then

$$(V_{\rm pred} - \bar{V}) - (V_{\rm true} - \bar{V}) = V_{\rm pred} - V_{\rm true}. \qquad (11)$$

Centering changes the zero point the network learns. The physical prediction-minus-truth residual remains invariant.

The matter-power path is again special. Its raw surface is transformed into "law space," such as $\log(P/P_{\rm base})$ and may be thinned along the wavenumber axis. Its final output mean must therefore be computed after that transformation. The generic loader skips the raw `dv_mean`. The bounded matter-power staging code streams the transformed mean and variance over row chunks.

## I. Worked staging example: the same rows in two coordinate systems

Return to the four-row example in Sec. I. Suppose the seeded permutation is

$$\texttt{order} = [2, 0, 3, 1]$$

and a physical cut rejects original row 0. The surviving pool is

$$\texttt{phys} = [2, 3, 1].$$

If `n_keep=2`, the selected global rows are $[2, 3]$. Before staging, the following expressions all refer to the original files:

$$C[2] = (0.33, 73, 0.052),$$
$$C[3] = (0.30, 69, 0.049),$$
$$V[2] = (12, 24, 36, 48, 60),$$
$$V[3] = (10.5, 21, 31.5, 42, 52.5).$$

If the subset fits in host RAM, `stage_source` makes compact copies over the distinct rows in sorted order:

$$C_{\rm src} = C[[2, 3]], \qquad V_{\rm src} = V[[2, 3]].$$

The new arrays have two rows. The returned `idx` is the local coordinate of each selected row, in the original selected order, inside that compact copy: `searchsorted`$([2,3],[2,3]) = [0,1]$. This selection is already sorted, so the map happens to be the identity. The next paragraph shows the general case. `dump_rows` remains $[2,3]$ because a separately opened base dump still uses original file coordinates.

If the subset does not fit, the program returns the original $C$ and memmapped $V$ with `idx=[2,3]`. No compact copy was made. A consumer must always apply the `idx` returned with its particular arrays. Reusing the resident branch's local coordinates against the file-backed branch would silently select the wrong cosmologies.

An ordinary selection is unsorted and the two branches must still present its rows in one canonical order. For $[9,2,5]$, the resident copy is stored sorted as $[2,5,9]$, but the returned local index is `searchsorted`$([2,5,9],[9,2,5]) = [2,0,1]$. A bare $[0,1,2]$ would select the wrong order. Reading the compact copy at $[2,0,1]$ walks disk rows $[9,2,5]$, the original selected order, exactly the order the disk branch keeps. The training loop applies its shared epoch permutation to whichever index the branch returned, so it trains the same cosmology at the same step whether host memory chose the resident or the disk branch. The `stage_source` routine refuses a selection with a repeated row, since the seeded selection is a unique permutation prefix and a repeat is upstream corruption. The `stage-ram` gate drives the real per-source loader in both storage regimes, draws one shared epoch permutation and compares the executed parameters, targets and minibatch membership and order row for row, with a mutation arm restoring the bare $[0,1,2]$ that must fail.

### J. Worked memory calculation

Assume $n_u = 250{,}000$, $P = 20$, $D = 3{,}000$ and both arrays are stored as `float32`. One element occupies four bytes. The resident copies need

$$\begin{aligned} B_C &= 250{,}000 \times 20 \times 4 = 20{,}000{,}000 \text{ bytes}, \\ B_V &= 250{,}000 \times 3{,}000 \times 4 = 3{,}000{,}000{,}000 \text{ bytes}, \\ B_I &= 250{,}000 \times 8 = 2{,}000{,}000 \text{ bytes}, \\ B_{\text{total}} &= B_C + B_V + B_I = 3{,}022{,}000{,}000 \text{ bytes}. \end{aligned}$$

$B_I$ is the local index array. Its entries use NumPy's 64-bit integer dtype, so each one occupies eight bytes. The decision compares this three-term total with the configured host-memory allowance. The status line names all three terms (parameters, data vector and index), prints their exact total, the comparison operator that held, the budget and the selected branch, so the displayed arithmetic sums. The comparison is strict ($B_{\text{total}} < B_{\text{budget}}$): an exact fill streams from disk. This leaves transient working room above the copies' own bytes. The gate pins the below, equal and above cases so this equality policy cannot drift.

### K. State after staging

At the end of `stage_train` and `stage_val`, the program has selected and located the rows. The staged state consists of NumPy host arrays and the seeded permutation. Geometry, model and optimizer construction follow in later lifecycle methods.

The next method, `build_geometry`, creates two maps:

$$\begin{aligned} \text{parameter geometry: } \boldsymbol{\theta} &\longmapsto x, && \text{raw cosmology to centered, rescaled network input,} \\ \text{output geometry: } \mathbf{d} &\longmapsto t, && \text{physical data vector to centered, covariance-scaled target.} \end{aligned}$$

Only after those maps exist can the batching code convert a NumPy batch with `torch.from_numpy`, cast it to the model's `float32` dtype, move it to the selected device and encode it. That is the subject of Sec. VI.

### L. Section summary

The first numerical job is identity preservation. The staging code must keep four statements true:

1. Parameter row $i$ and data-vector row $i$ describe the same cosmology.
2. Parameter names and column order match the covariance that will transform them.
3. The seeded subset is selected after the declared physical cuts and contains the requested absolute number of rows.
4. Every row index is interpreted in the coordinate system of the array it indexes: compact local rows for a resident copy, original global rows for a file-backed dump.

Once those conditions hold, the program has a staged training source. The next problem is numerical geome-

try: how to transform correlated parameters and correlated outputs into coordinates a neural network can learn without silently changing the $\chi^2$ that defines emulator accuracy.

## VI. GEOMETRIES: CHANGING COORDINATES WHILE PRESERVING THE SCIENCE

The staged source contains physical numbers. The columns can differ by many orders of magnitude and can be strongly correlated. A neural network trains more easily when its inputs and targets have comparable scales. The geometry classes own that coordinate change.

Here, *geometry* names a reversible map between physical coordinates and network coordinates. There are two maps:

$$\boldsymbol{\theta} \xleftrightarrow[\text{decode}]{\text{encode}} x, \qquad \mathbf{d} \xleftrightarrow[\text{decode}]{\text{encode}} t.$$

$x$ is the tensor presented to the model. $t$ is the tensor the model is trained to predict. The inverse maps are part of the saved artifact because an emulator prediction is useful only after it has returned to physical units.

### A. The parameter geometry

The ordinary input transform is `ParamGeometry`. The constructor stores four facts:

The covariance is symmetric, so NumPy computes

$$\Sigma = Q\,\mathrm{diag}(\lambda_1, \ldots, \lambda_P)Q^{\mathsf{T}}. \tag{12}$$

For a batch of raw parameter rows $\Theta \in \mathbb{R}^{B\times P}$, encoding is

$$X = (\Theta - \boldsymbol{\mu})Q\,\mathrm{diag}(s_1^{-1}, \ldots, s_P^{-1}). \tag{13}$$

Subtracting the mean centers the cloud at zero. Multiplication by $Q$ rotates the correlated ellipsoid onto its principal axes. Division by $s_j$ gives each rotated direction unit scale. This complete operation is called *whitening*.

Decoding performs the inverse operations in reverse order:

$$\Theta = [X\,\mathrm{diag}(s_1, \ldots, s_P)]\,Q^{\mathsf{T}} + \boldsymbol{\mu}. \tag{14}$$

Because $Q$ is orthonormal, $QQ^{\mathsf{T}} = I$. A numerical identity test must therefore verify

$$\texttt{decode(encode(theta))} \simeq \texttt{theta} \tag{15}$$

over representative rows, with an error consistent with the stored `float32` representation.

#### 1. Worked whitening example with two parameters

Consider two input parameters with different physical units: an amplitude parameter measured in dimensionless units and an expansion rate measured in $\mathrm{km\,s^{-1}Mpc^{-1}}$. Suppose their training-set mean is

$$\boldsymbol{\mu} = (0.30, 70.0),$$

and the covariance eigenvectors and square-root eigenvalues are

$$Q = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}, \qquad (s_1, s_2) = (1, 3).$$

The numerical values are illustrative. The program obtains these arrays from the actual training covariance.

Take one raw cosmology

$$\boldsymbol{\theta} = (0.32, 73.0).$$

Encoding first subtracts the mean. The offset row is $(0.02, 3.0)$. It then multiplies that row on the right by $Q$:

$$\begin{aligned}(0.02, 3.0)Q &= \left(\frac{0.02 - 3.0}{\sqrt{2}}, \frac{0.02 + 3.0}{\sqrt{2}}\right) \\ &\simeq (-2.107, 2.135).\end{aligned}$$

This multiplication forms two new coordinates. Each new coordinate combines both raw parameters. Finally, division by $(s_1, s_2)$ gives the network input

$$x \simeq (-2.107, 0.712).$$

The encoded numbers are dimensionless. The eigenvectors combine the original units only after the covariance has supplied the corresponding conversion and scale information.

Decoding uses the stored operations in reverse. Multiplying by the scales gives $(-2.107, 2.135)$. Right-multiplying by $Q^{\mathsf{T}}$ returns the raw offset $(0.02, 3.0)$ and adding $\boldsymbol{\mu}$ returns $(0.32, 73.0)$. A decoder that applies $Q$ returns the wrong cosmology even though its shape is correct. The identity test compares values because dimensions alone miss this error.

For a batch, the same operations act on every row. NumPy and PyTorch broadcast the one-dimensional mean across the batch dimension: a stored mean with shape $(P,)$ is subtracted from an input with shape $(B, P)$ as if the same mean row had been written $B$ times. Broadcasting represents those repeated rows without allocating them. The matrix multiply then maps all $B$ rows in one operation, but its mathematical meaning is still the row-by-row calculation shown above.

*a. Alternative constructors.* The method `from_covmat` reads and diagonalizes the training covariance. The method `from_state` rebuilds the object from saved arrays. The ordinary `__init__` method only stores fields. Keeping those jobs separate means inference never needs the original covariance file.

| Field | Meaning |
|---|---|
| `names` | Parameter names in the exact column order of the covariance. |
| `center` | Training-set mean $\boldsymbol{\mu}$. |
| `evecs` | Orthonormal covariance eigenvectors $Q$. |
| `sqrt_ev` | Square roots of the covariance eigenvalues, $s_j = \sqrt{\lambda_j}$. |

TABLE VIII. Persisted fields of the ordinary parameter geometry. Together they define both directions of the reversible map in Eqs. (13) through (14). None may be inferred later from a current default or filename.

### B. Views, copies, dtype and device

The geometry converts each array with

```
torch.as_tensor(array,
                dtype=torch.float32,
                device=device)
```

If the source already has the requested dtype and device, PyTorch may reuse its storage. If dtype or device changes, it creates new storage. Moving from a NumPy CPU array to CUDA necessarily copies the values to GPU memory.

The model uses `float32`. Covariance construction and diagnostic reductions often use `float64` on the CPU because sums and eigendecompositions lose more information than a neural-network forward pass. The program must state where a cast occurs. A usable scale must remain finite and nonzero after conversion to `float32`.

### C. The data-vector geometry

Cosmic-shear output uses `DataVectorGeometry`. Its first operation is a mask. The full physical vector has width $D$, but the likelihood keeps only $K$ entries. `dest_idx` stores their global column positions.

For a batch $V \in \mathbb{R}^{B\times D}$,

$$V_{\text{keep}} = V[:, \texttt{dest\_idx}] \in \mathbb{R}^{B\times K}.$$

This is PyTorch advanced indexing. It gathers the selected columns in the specified order and returns a new tensor. The method is named `squeeze`, but it is not `torch.squeeze`: no size-one dimension is removed.

The inverse method, `unsqueeze`, allocates a zero-filled $B \times D$ tensor and assigns the $K$ values back into `dest_idx`. Masked locations remain zero. This is a scatter operation.

After masking and centering, the dense-covariance geometry applies the same eigenvector construction as Eq. (13). If

$$\Sigma_{\text{keep}} = U \operatorname{diag}(\eta) U^{\mathsf{T}}, \tag{16}$$

then a centered kept vector $r$ is whitened as

$$t = rU \operatorname{diag}(\eta^{-1/2}). \tag{17}$$

The ordinary squared length of $t$ equals the covariance-weighted physical length of $r$:

$$\|t\|^2 = r\Sigma_{\text{keep}}^{-1} r^{\mathsf{T}}. \tag{18}$$

This identity is why training in whitened space does not redefine the scientific metric.

For example, let a two-coordinate physical residual be $r = (0.5, 3.0)$ and let the covariance be diagonal with variances $(0.25, 9.0)$. Its inverse is $\Sigma_{\text{keep}}^{-1} = \operatorname{diag}(4, 1/9)$, so

$$r\Sigma_{\text{keep}}^{-1} r^{\mathsf{T}} = 0.5^2 \times 4 + 3.0^2 \times \frac{1}{9} = 2.$$

Whitening divides the two residuals by their standard deviations $(0.5, 3)$, which gives $t = (1, 1)$. The ordinary squared length is again $1^2 + 1^2 = 2$. The direct contraction and the whitened sum use different code paths to express the same scientific distance. Comparing them gives an independent check of the geometry.

### D. Diagonal output families

Some families use one independent scale per output and omit a dense covariance rotation. They use

$$t_k = \frac{d_k - \bar{d}_k}{\sigma_k}, \qquad d_k = t_k\sigma_k + \bar{d}_k. \tag{19}$$

The axes and units are artifact facts. A consumer reads them from the saved geometry. Filenames and current code defaults have no authority to reconstruct those facts.

### E. The geometry acceptance test

A geometry is ready only when four numerical statements hold:

1. Every scale is finite and representably nonzero in its stored dtype.
2. `decode(encode(raw))` returns the raw value within the declared floating-point tolerance.
3. Saving and rebuilding the state leaves the transform unchanged.

| Geometry | Coordinate axis | Stored physical facts |
|---|---|---|
| `ScalarGeometry` | Named derived parameters | Output names, center, scale. |
| CMB diagonal | Multipole $\ell$ | Spectrum, $\ell$, units, fiducial spectrum, cosmic-variance scale, amplitude law. |
| `GridGeometry` | Redshift $z$ | Quantity, units, target law, offset, stored $z$ grid. |
| `Grid2DGeometry` | Flattened $(z, k)$ surface | Quantity, units, target law, both axes, constant-point mask. |

TABLE IX. Output families whose standardization is diagonal in their stored coordinate order. The axis column defines what output index $k$ means. The facts in the last column travel with the artifact so an equal-width but different physical axis is refused.

4. The squared whitened residual agrees with a direct physical covariance contraction.

These four statements are the acceptance contract.

*a. Safe use of current geometry constructors.* Three construction paths can violate the first statement before a round trip is attempted. One covariance path takes the square root of a numerically negative eigenvalue and produces NaN. A block-diagonal path clips a negative eigenvalue to zero and later divides by the zero scale. A diagonal family can also accept a positive `float64` scale that becomes exact zero after storage as `float32`. After fitting or rebuilding a geometry, inspect centers and stored factors in their final dtype, require every divisor to be finite and nonzero and run both the encode/decode round trip and the direct physical-score comparison before training.

## VII. THE NEURAL DESIGNS AND THEIR CONSTRUCTIBLE SPECIFICATIONS

Once the geometries exist, the program knows the model's input width and output width. It can construct a neural network without embedding data-file knowledge in the network class.

### A. Define each design before comparing it

A *design* is the rule for connecting trainable operations, fixed coordinate operations and any analytic physics component into one predictor. The names used by this library mean:

A multilayer perceptron, abbreviated MLP, is a sequence of affine maps and nonlinear activations. A dense or fully connected affine map lets every output feature depend on every input feature. "Residual" describes a learned correction added to an unchanged skip path. A physical residual appears only when the loss constructs one. The following subsections define the mechanics behind each row.

### B. A class is passed as data

The model configuration is a dictionary whose `cls` entry holds a class object and whose remaining entries are constructor arguments:

```
model_opts = {"cls":         ResMLP,
              "int_dim_res": 256,
              "n_blocks":    4,
              "block_opts":  block_opts}
```

A class object is callable. Calling `ResMLP(...)` creates a new module. `make_model` removes the bookkeeping entries, injects the data-dependent input and output dimensions, constructs the module and moves it to the selected device.

The same pattern constructs the optimizer and scheduler. Computed values such as batch-scaled learning rate and device-dependent fused behavior are injected by the factory at construction time. The configuration contains no stale copies of those facts.

### C. Factories create distinct submodules

Residual blocks receive activation and normalization factories. A factory is a callable such as `act(width)` that returns a new `nn.Module`. Calling it once per layer gives every layer its own learnable activation parameters. Passing one already-created activation instance would share its parameters across layers.

Parameter-free activations ignore the width while obeying the same interface:

```
def relu_factory(dim):
    return nn.ReLU()
```

The argument is intentionally unused. The uniform interface lets the model construction loop treat learnable and fixed activations identically.

### D. Registration: why ModuleList matters

PyTorch discovers trainable parameters by walking registered child modules. `nn.Sequential` and `nn.ModuleList` register their contents. A bare

| Design | Definition | What is learned and what is fixed |
|---|---|---|
| ResMLP | A residual multilayer perceptron: dense layers act on the complete feature vector and each block adds a learned correction to its input. | All dense weights and activation parameters are learned. Input/output geometries are fixed after fitting. |
| ResCNN | A residual model with a one-dimensional convolutional correction head. A convolution reuses one local kernel along an ordered coordinate. | The trunk, kernels and head gate are learned. Scatter/gather coordinates and validity masks are fixed. |
| ResTRF | A residual model with a transformer correction head. A transformer repeats attention, feature mixing and residual additions so each token can use information from other tokens. | Query/key/value projections, mixing layers and head gate are learned. Token/coordinate layout is fixed. |
| IA template model | A factored intrinsic-alignment model. Intrinsic alignment is the correlation of galaxy shapes with local structure. This source effect is separate from gravitational lensing. The network predicts basis templates and a closed-form law combines them with named IA amplitudes. | Template values are learned. The amplitude combination law is fixed physics. |
| PCE plus neural residual | A polynomial-chaos expansion supplies a frozen smooth base. A neural network learns what that finite polynomial basis misses. | Selected polynomial terms and coefficients are fitted once, then frozen. The residual network remains trainable. |

TABLE X. Constructible predictor designs. A design specifies tensor connectivity and the boundary between learned and fixed operations. Separate owners supply the data files, physical units, optimizer and output geometry.

Python list does not. A plain list is safe only as a temporary builder that is immediately expanded into `nn.Sequential(*layers)` or copied into `nn.ModuleList(layers)`.

Registration controls four operations:

1. inclusion in `model.parameters()`,
2. movement by `model.to(device)`,
3. inclusion in `state_dict()`.
4. switching between training and evaluation modes.

### E. ResMLP: the baseline map

Table XI separates the two rectangular projections from the width-preserving trunk. `nn.Linear(in_features,out_features)` returns exactly `out_features` values. Every dense layer inside `ResBlock` is square and therefore preserves the block width.

For a batch of $B = 256$ cosmologies, $P_{\mathrm{enc}} = 12$, $W = 128$ and $K = 780$, the entry projection reads a $(256, 12)$ tensor and returns $(256, 128)$. Every residual block reads and returns $(256, 128)$. The exit projection alone produces $(256, 780)$. Calling the entry projection a residual-block layer would erase this defining equal-width condition.

The library's residual block applies $L$ internal dense layers before the skip addition, final normalization and activation. The skip is added to the final dense layer's output. That sum then passes through the final normalization and activation. Figure 5 and the recurrence below state the exact order.

### F. Structured correction heads

`ResCNN` and `ResTRF` retain the ResMLP trunk and add a correction head. The trunk predicts every kept output. The head reshapes that output into physical bins, learns local or cross-bin structure and adds a gated correction:

$$\widehat{T} = T_{\mathrm{trunk}} + g\, T_{\mathrm{correction}}. \tag{20}$$

The convolutional head treats tomographic bins as channels and angular positions as the one-dimensional coordinate. The transformer head treats bins as tokens. A token is one row presented to attention. Attention mixes information between token rows.

Ragged bins have different lengths. The implementation scatters them into a rectangle of width equal to the longest bin. Padding positions require an explicit validity mask because convolution, affine modulation and attention can turn an initial zero into a nonzero hidden value.

| Stage | Input shape | Operation | Output shape | Membership in a `ResBlock` |
|---|---|---|---|---|
| Entry projection | $(B, P_{\mathrm{enc}})$ | `Linear(P_enc,W)` mixes the encoded parameters and changes the feature width. | $(B, W)$ | No |
| Residual trunk | $(B, W)$ | Apply $N_{\mathrm{block}}$ independent residual blocks. Every internal dense layer is `Linear(W,W)`. | $(B, W)$ | Yes |
| Exit projection | $(B, W)$ | `Linear(W,K)` changes hidden features into the $K$ encoded target coordinates. | $(B, K)$ | No |
| Final affine | $(B, K)$ | Multiply by one learned gain and add one learned bias. Broadcasting preserves the target shape. | $(B, K)$ | No |

TABLE XI. Shape contract of the baseline `ResMLP`. The rectangular entry and exit projections change feature count. A residual block begins only after the entry projection has established width $W$ and it ends before the exit projection changes $W$ to $K$.

The correction is initialized to zero, so the initial structured model equals its trunk. A nonzero outer gate lets gradients reach the zero-initialized last correction layer on the first optimizer step. An activation placed after a zero-initialized layer must have a nonzero derivative at zero or the head can remain permanently asleep.

### G. Factored intrinsic-alignment designs

The analytic combination owns the known polynomial dependence on intrinsic-alignment amplitudes. The input geometry appends the raw amplitudes after the whitened non-amplitude parameters. The model reads the non-amplitude block and emits $T_{\mathrm{tmpl}}$ templates:

$$X \longmapsto \widehat{T}_{\mathrm{tmpl}} \in \mathbb{R}^{B \times T_{\mathrm{tmpl}} \times K}.$$

For a fixed template index $j$, the slice $\widehat{T}_{\mathrm{tmpl}}[:, j, :]$ has shape $(B, K)$: it contains that template for every cosmology in the minibatch. The symbol $T_{\mathrm{tmpl}}$ counts templates, while $T$ denotes the encoded target tensor. The loss reads the appended amplitudes and combines the templates with the known coefficients. This separation keeps the exact amplitude law outside the neural approximation.

### H. One dense layer, written explicitly

The basic learned operation is an affine map. For a minibatch $H \in \mathbb{R}^{B \times W}$, a PyTorch `nn.Linear(W,W)` computes

$$Z_{bi} = \sum_{j=0}^{W-1} H_{bj} A_{ij} + a_i. \tag{21}$$

$A_{ij}$ is a learnable weight and $a_i$ is a learnable bias. The letter $W$ denotes the hidden width. $A$ denotes the weight matrix. The first index $b$ selects a cosmology in the minibatch. The index $j$ selects an incoming feature and $i$ selects an outgoing feature. PyTorch stores the weight matrix with shape $(\texttt{out\_features}, \texttt{in\_features})$ and applies the transpose convention needed for a batch whose feature axis is last.

An affine map alone cannot represent a general nonlinear physical response. Composing two affine maps still gives one affine map. The activation is therefore applied element by element after selected linear layers:

$$H'_{bi} = \phi(Z_{bi}).$$

Element by element means that the activation changes each scalar without mixing feature columns. The linear layer performs the mixing. The activation provides the bend.

The entry and exit projections are rectangular. If the encoded cosmology has $P_{\mathrm{enc}} = 12$ features, the hidden width is $W = 128$ and the target has $K = 780$ values, their shapes are

$$(\texttt{128}, \texttt{12}), \qquad (\texttt{780}, \texttt{128}).$$

All dense layers between them are square $(\texttt{128}, \texttt{128})$. This fixed width lets a residual block add its input to its correction without another shape adapter.

### I. A residual block, operation by operation

Figure 5 names the two paths. Let $h_0$ be the block input, and let the block contain $L$ internal dense layers. For every nonfinal layer $i = 1, \ldots, L-1$, the executed recurrence is

$$\begin{aligned} z_i &= h_{i-1} A_i^{\mathsf{T}} + a_i, \\ h_i &= \phi_i(N_i(z_i)). \end{aligned} \tag{22}$$

$A_i$ is a square $W \times W$ weight matrix, $a_i$ is a length-$W$ bias, $N_i$ is that layer's independently constructed normalization module and $\phi_i$ is that layer's independently

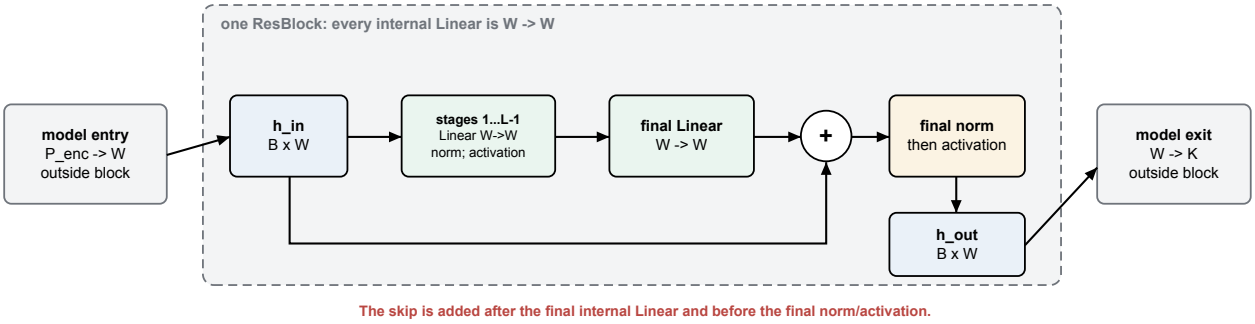



FIG. 5. The exact residual-block order used by the library. Every dense layer inside the dashed boundary maps width $W$ to the same width $W$. The skip is added after the final internal dense layer and before that layer's normalization and activation. The residual-block boundary contains only width-preserving layers. Rectangular model-entry and model-exit projections sit outside that boundary.

constructed activation module. Every array in Eq. (22) has shape $(B, W)$ after the batch matrix multiplication. The length-$W$ bias is broadcast across the $B$ minibatch rows. Broadcasting reuses one stored bias tensor across all $B$ rows.

The last layer has a different order:

$$\begin{aligned} z_L &= h_{L-1} A_L^{\mathsf{T}} + a_L, \\ s_L &= z_L + h_0, \\ h_{\text{out}} &= \phi_L(N_L(s_L)) \,. \end{aligned} \tag{23}$$

The addition therefore lands after the final internal dense layer. With the default $L = 2$, it lands after the second dense layer. It lands before the second layer's normalization and activation. The result returned by the library is $\phi_L(N_L(z_L + h_0))$. Describing it as a bare $h_0 + F(h_0)$ would omit two executed operations.

The skip path bypasses the internal dense calculation that forms $z_L$. Backward differentiation still passes through the final $N_L$ and $\phi_L$ because they follow the addition. This supplies a shorter gradient route. The complete block derivative still includes the final normalization and activation between output and input.

The sum is coordinate by coordinate. Entry $(b, j)$ of $z_L$ is added only to entry $(b, j)$ of $h_0$. Concatenation would append feature columns and return shape $(B, 2W)$. Addition returns $(B, W)$. This is why every dense layer inside the block must preserve width. A rectangular $P_{\text{enc}} \to W$ or $W \to K$ projection belongs outside the block.

Operations preserve the input and construct new tensors whose autograd records point back to their parents. *Autograd* is PyTorch's automatic-differentiation system: it records the executed tensor operations and later applies the chain rule in reverse during `backward()`. Avoiding an in-place overwrite keeps the original value available to the skip addition and to backward differentiation.

### J. Normalization controls the activation's operating range

The model's normalization slot performs affine rescaling. In its shared form,

$$\widetilde{Z} = gZ + b,$$

where $g$ and $b$ are learned scalars. In its per-feature form, $g_i$ and $b_i$ are vectors of length $W$ and are broadcast across the batch axis. *Broadcasting* means that PyTorch treats the missing batch dimension as repeated without allocating $B$ copies of the vector.

The purpose is to keep values in the region where the activation derivative is useful. If a saturating activation receives extremely large magnitudes, its output becomes nearly constant and its derivative approaches zero. A zero derivative prevents upstream weights from receiving a learning signal. The affine normalization learns a scale and shift that can return those features to the curved portion of the activation.

Prediction for one cosmology uses its own row and the learned parameters. The other rows in its minibatch leave that prediction unchanged. The same learned $g$ and $b$ are used during training and inference, and the module has no running mean or variance buffer to synchronize with an exponential moving average.

### K. Activation factories and learnable activation parameters

The activation configuration names a family. A factory creates one module per activation site. In the default smooth family, the module owns learned shape parameters such as a left-hand slope and a transition sharpness. They are ordinary `nn.Parameter` objects: they appear in `model.parameters()`, receive gradients, move with `model.to(device)` and are saved in the state dictionary.

A *state dictionary* is a mapping from stable string names to tensors. It contains parameters and registered buffers. Python source code remains in the module files. Rebuilding therefore creates the architecture first and then loads the recorded tensors into matching names and shapes.

An activation used immediately after an exactly zero-initialized correction layer needs special scrutiny. At initialization its input is exactly zero. If $\phi'(0) = 0$, the chain rule multiplies the correction layer's gradient by zero and that path cannot wake on the first update. The structured-head schema therefore considers both $\phi(0)$, which controls exact identity at initialization and $\phi'(0)$, which controls whether the head can learn.

### L. The convolutional correction head

Cosmic-shear outputs form a structured physical vector. The channel order groups values by correlation type and tomographic-bin pair. Each group varies along angular separation. The convolutional head first changes layout while preserving every value:

$$(\mathtt{B}, \mathtt{K}) \longrightarrow (\mathtt{B}, \mathtt{N}_{\mathrm{bin}}, \mathtt{N}_{\theta}).$$

The first operation is a fixed coordinate scatter. It writes each kept output into a declared rectangle position and carries no learned values.

A one-dimensional convolution slides the same kernel along the angular axis. For kernel half-width $q$, an output at angle index $u$ has the schematic form

$$y_{c,u} = \sum_{c'} \sum_{s=-q}^{q} k_{c,c',s}\, x_{c',u+s} + a_c.$$

The index $c'$ ranges over input bins, so the layer can mix bin pairs at nearby angular positions. Reusing $k$ at every $u$ is weight sharing: the number of learned values does not grow with the number of angular positions.

Ragged bins contain different numbers of valid angles. They are stored in a rectangle by padding shorter rows. The validity mask marks real positions. Zeroing once at the input is insufficient because a convolutional bias, cross-bin mixing or feature-wise modulation can make a padded cell nonzero. The mask must be applied wherever the architecture can repopulate invalid cells and again before gathering the correction.

The map stores each survivor's original angular slot, not its rank among the survivors in that bin. For example, two bins may each keep two values while one keeps angular slots zero and two and the other keeps slots one and two. Their counts are equal, but their physical layouts are not. A completely masked bin remains an empty rectangle row so that every later bin keeps its original channel number. The artifact records this map and the aligned Boolean mask, and rebuilding compares both with fixed buffers in the saved model state.

### M. The transformer correction head

The transformer head uses the same physical rectangle but a different mixing mechanism. Each bin is a token. A token is one row presented to attention. Attention constructs query, key and value representations:

$$Q_{\mathrm{qry}} = XA_Q, \qquad K_{\mathrm{key}} = XA_K, \qquad V_{\mathrm{val}} = XA_V,$$

then computes

$$\mathrm{Attention}(X) = \mathrm{softmax}\left(\frac{Q_{\mathrm{qry}} K_{\mathrm{key}}^{\mathsf{T}}}{\sqrt{d_h}}\right) V_{\mathrm{val}}. \tag{24}$$

The softmax converts each row of similarity scores into nonnegative weights that sum to one. One attention layer can couple distant bins. A convolution applies its kernel locally.

For one attention head, let $B$ be batch size, $S$ the number of tokens and $d_h$ the head-feature width. Then $X$ has shape $(B, S, d_{\mathrm{model}})$. Each projected tensor $Q_{\mathrm{qry}}$, $K_{\mathrm{key}}$ and $V_{\mathrm{val}}$ has shape $(B, S, d_h)$. The matrix product with the last two axes transposed has shape $(B, S, S)$: entry $(b, s, u)$ scores how strongly token $s$ reads token $u$ in sample $b$. Multiplication by $V_{\mathrm{val}}$ returns $(B, S, d_h)$. With $B = 2$, $S = 3$ and $d_h = 4$, the score tensor is therefore $(2, 3, 3)$. The head-feature width appears in the projected tensors and not in the two score axes. A $(2, 4, 4)$ score tensor would incorrectly put the head width on both axes. Multiple heads repeat this calculation with separate projections and concatenate their feature axes before an output projection.

The layout assigns angular positions to feature coordinates inside each bin token. A textbook token mask alone therefore cannot hide every ragged angular feature. The implementation needs the same per-block coordinate-validity mask used by the scatter. The mask's shape and meaning are persisted with the artifact because reconstructing it from counts can assign the right number of cells to the wrong coordinates. Layer normalization computes its mean and variance from physical features only. Query, key and value projections are masked again after their linear maps. A query-key pair is excluded when that attention head has no shared physical coordinate, and an entirely masked token returns an exact zero instead of a row of nonfinite softmax values. The mask is applied again after every attention projection, residual addition, MLP layer, activation and FiLM shift because each of those operations can repopulate a storage-only cell.

### N. FiLM: conditioning a head on the cosmology

Feature-wise linear modulation, abbreviated FiLM, lets a correction head depend directly on the encoded cosmology. A small parameter network computes a scale $\gamma(x)$ and shift $\beta(x)$, then applies

$$H' = \gamma(x) \odot H + \beta(x).$$

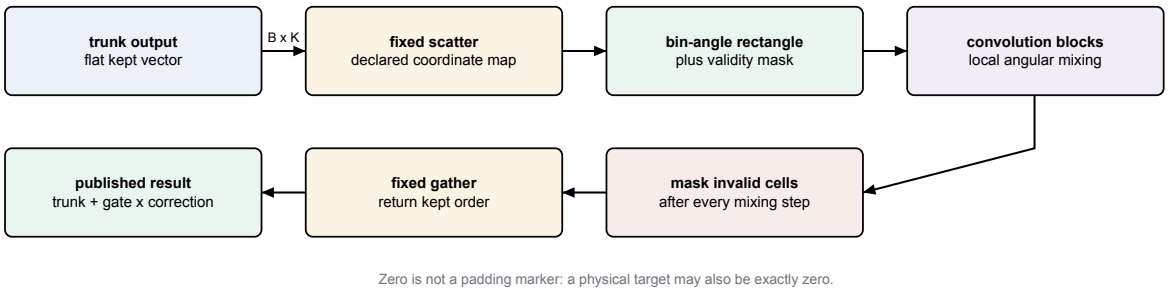



FIG. 6. The convolutional correction-head path. Reshaping and scatter/gather operations are fixed coordinate transformations. Only the convolution blocks and outer gate contain learned values.

$\odot$ denotes elementwise multiplication. The scale and shift are broadcast over the declared spatial axis. FiLM is initialized as the identity, $\gamma = 1$ and $\beta = 0$, so adding the option does not perturb the initial trunk prediction.

For example, suppose the head activation $H$ has shape $(B, C, S)$: $C$ is a feature channel and $S$ is a spatial or angular coordinate. If the parameter network emits $\gamma(x)$ and $\beta(x)$ with shape $(B, C)$, the code inserts a singleton final axis to obtain $(B, C, 1)$. Broadcasting reuses each sample-and-channel value across all $S$ coordinates without materializing $S$ stored copies. It must not reuse a value across the batch axis: each cosmology has its own modulation.

The ownership distinction is useful: the main trunk learns the full cosmology-to-output map, the convolution or transformer learns structured corrections and FiLM tells that correction which cosmology it is correcting. Those roles should remain visible in tensor names and artifact metadata.

## VIII. POLYNOMIAL-CHAOS BASES AND NEURAL RESIDUALS

### A. Definition: polynomial, basis, expansion and chaos

A *polynomial* in one variable is a finite sum of nonnegative integer powers:

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_d x^d.$$

The numbers $a_0, \ldots, a_d$ are coefficients and $d$ is the degree when $a_d \neq 0$. A multivariate polynomial also contains products such as $x_1 x_2$ or $x_1^2 x_3$.

A *basis* is a set of building-block functions whose weighted sum can represent the approximation. This library uses Legendre polynomials on $[-1, 1]$:

$$P_0(x) = 1, \qquad P_1(x) = x,$$
$$P_2(x) = \tfrac{1}{2}(3x^2 - 1), \qquad P_3(x) = \tfrac{1}{2}(5x^3 - 3x).$$

Higher degrees follow the three-term recurrence

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x).$$

The input parameters are mapped from their fitted training box to $[-1, 1]$ before these curves are evaluated.

The fit receives already-whitened parameter columns $u_j$. For each column, it finds the selected-training minimum and maximum, then pads the half-width by five percent:

$$\begin{aligned} m_j &= \tfrac{1}{2}(u_{j,\min} + u_{j,\max}), \\ h_j &= 1.05\,\tfrac{1}{2}(u_{j,\max} - u_{j,\min}) + 10^{-12}, \\ \ell_j &= m_j - h_j, \qquad r_j = m_j + h_j, \\ x_j &= 2\frac{u_j - \ell_j}{r_j - \ell_j} - 1. \end{aligned}$$

The $10^{-12}$ term prevents a zero denominator for a constant training column. If selected whitened values span $[-2, 2]$, then $(\ell, r) \simeq (-2.1, 2.1)$ and $u = 1.05$ maps to $x = 0.5$. These persisted bounds define the basis coordinate. The current inference-time clamp to the box is a later domain policy and must not be confused with the affine map.

Legendre polynomials are *orthogonal* under constant weight on that interval:

$$\int_{-1}^{1} P_m(x) P_n(x)\, dx = 0 \quad \text{when } m \neq n.$$

Orthogonality means different basis directions do not duplicate one another under this inner product. If a uniformly distributed input defines the probability measure $dx/2$, then

$$\int_{-1}^{1} \left[\sqrt{2n+1}\,P_n(x)\right]^2 \frac{dx}{2} = 1.$$

This is why the implementation multiplies degree $n$ by $\sqrt{2n+1}$. Under the unnormalized measure $dx$, the unit-norm factor is $\sqrt{(2n+1)/2}$. Naming the measure prevents a hidden factor of two.

An *expansion* is the weighted sum of selected basis terms. The word *chaos* is historical terminology from representing functions of random inputs with orthogonal polynomials. The emulator still represents deterministic cosmological dynamics. A polynomial chaos expansion, abbreviated PCE, is therefore an orthogonal-polynomial approximation to the mapping from input parameters to output quantities.

A multivariate PCE uses separable terms. Separable means that one term is a product of one-dimensional basis curves:

$$\Psi_{\boldsymbol{\alpha}}(x) = \prod_{j=1}^{P} \sqrt{2\alpha_j + 1}\, P_{\alpha_j}(x_j). \tag{25}$$

The multi-index $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_P)$ states which degree is used for each parameter. A total-degree rule keeps terms satisfying $\sum_j \alpha_j \le p_{\max}$. The implemented hybrid rule is

$$\left(\sum_j \alpha_j^q\right)^{1/q} \le p_{\max}, \qquad \#\{j : \alpha_j \neq 0\} \le r_{\max},$$

where $q \in (0, 1]$ controls hyperbolic sparsity and $r_{\max}$ is the maximum interaction order, the number of distinct parameters allowed in one product. At $q = 1$ the first rule is ordinary total degree. With $p_{\max} = 4$ and $q = 0.5$, $x_1^4$ scores 4 and is kept. $x_1^2 x_2^2$ scores $(\sqrt{2} + \sqrt{2})^2 = 8$ and is excluded. The rule expresses a sparsity-of-effects assumption: concentrated degree is preferred over a high-order interaction spread across many inputs.

The public `pce` block has eight controls. They answer different questions and therefore should not be collapsed into one generic "size" knob:

## B. From one-dimensional curves to a term matrix

For each box-mapped parameter $x_j$, the basis builder evaluates $P_0(x_j), \ldots, P_{p_{\max}}(x_j)$. Degree zero is the constant function. A multi-index selects one degree from each parameter and multiplies those values according to Eq. (25).

For two parameters and $\boldsymbol{\alpha} = (2, 1)$,

$$\Psi_{(2,1)}(x_1, x_2) = \sqrt{5}\, P_2(x_1) \sqrt{3}\, P_1(x_2).$$

This is an interaction term because both parameters affect it. A total-degree bound of two excludes the term since $2 + 1 = 3$.

Allowed multi-indices are enumerated once in deterministic order. That order identifies coefficient rows and is persisted. Rebuilding under a different enumeration can preserve every matrix shape while assigning coefficients to the wrong polynomials.

Number the candidate multi-indices $\boldsymbol{\alpha}^{(0)}, \ldots, \boldsymbol{\alpha}^{(M-1)}$. The term matrix $\Phi$ is defined entry by entry as

$$\Phi_{im} = \Psi_{\boldsymbol{\alpha}^{(m)}}(x_i).$$

It has shape $(\mathtt{N}, \mathtt{M})$: one row per cosmology and one column per candidate polynomial term. If $J$ output modes are accepted, their coefficient matrix has shape $(\mathtt{M}, \mathtt{J})$. The product has shape $(\mathtt{N}, \mathtt{J})$ mode amplitudes. Multiplying by the stored $J$ mode vectors and adding the output mean produces $(\mathtt{N}, \mathtt{K})$ target predictions, where the global symbol $K$ again means kept output coordinates. The executed selector starts with the constant column, repeatedly chooses the inactive column with largest normalized correlation to the current residual and refits ordinary least squares on the entire growing active set. This is a greedy orthogonal-matching-pursuit-like procedure. The historical function name contains `lars`, but the code does not execute the equiangular update that defines full LARS. Selection must terminate when every column is active. Re-appending an already-active column does not add a new model.

### 1. A complete two-input degree-two PCE row

Let $P = 2$ mapped inputs and keep every total-degree-two term. In deterministic order the six multi-indices are

$$(0,0), (1,0), (2,0),$$
$$(0,1), (0,2), (1,1).$$

This is the implementation's persisted enumeration: it finishes the single-variable degrees for input zero, then those for input one, then the two-variable interaction. Reordering the same set would attach coefficients to different polynomials. Choose the mapped point $x = (0.5, -0.5)$. The one-dimensional values are $P_1(0.5) = 0.5$, $P_1(-0.5) = -0.5$ and $P_2(\pm 0.5) = -0.125$. Including the normalization factors in Eq. (25), the corresponding row of $\Phi$ is

$$\Phi(x) \simeq (1,\ 0.8660,\ -0.2795,\ -0.8660,\ -0.2795,\ -0.7500).$$

For a one-mode coefficient column

$$c = \big(1,\ 0.2,\ -0.1,\ 0.05,\ 0.4,\ -0.2\big)^{\mathsf{T}},$$

the polynomial mode amplitude is the dot product

$$\Phi(x)c \simeq 1 + 0.1732 + 0.0280 - 0.0433 - 0.1118 + 0.1500 \simeq 1.1961.$$

This is the entire PCE evaluation at one point: map the physical parameters to $[-1, 1]$, evaluate named basis terms, preserve their persisted column order and multiply by fitted coefficients. The later SVD mode vectors turn one or more such amplitudes back into the $K$ output coordinates.

## C. Fitting shared output modes

Let $Y \in \mathbb{R}^{N \times K}$ be the whitened training targets. The mean $\bar{Y}$ is taken over the $N$ training rows separately for

| Configuration key | Default | Exact meaning |
|---|---|---|
| `form` | required | Composition law. `residual` adds the neural correction to the base. `ratio` uses the separately defined multiplicative/gain construction and is legal only on compatible families. |
| `p_max` | 4 | Hyperbolic degree bound on a candidate multi-index in Eq. (25). |
| `r_max` | 2 | Maximum number of distinct input parameters that may have nonzero degree in one polynomial term. |
| `q` | 0.5 | Hyperbolic sparsity exponent in $(0, 1]$. Smaller values penalize degrees spread across several inputs more strongly. |
| `k_max` | 40 | Maximum number of leading singular-vector output modes whose polynomial fits may be attempted. It limits output directions. Separate bounds above limit polynomial terms. |
| `loo_max` | 0.05 | Largest accepted relative leave-one-out error for one output mode. A smaller value demands stronger generalization evidence. |
| `max_terms` | 30 | Maximum number of active polynomial columns in the greedy fit for one output mode. The constant column counts as one active term. |
| `max_fail` | 4 | Stop visiting later singular-vector modes after this many consecutive modes fail the leave-one-out threshold. |

TABLE XII. The complete public PCE control block. `p_max`, `r_max` and `q` determine which polynomial columns exist. `k_max`, `loo_max`, `max_terms` and `max_fail` control fitting and acceptance. `form` controls how the frozen base and neural correction are composed.

each output coordinate, so it is mathematically a row vector of shape $(1, K)$. The stored PyTorch buffer has shape $(K,)$ and broadcasts across rows. The mean spans training rows separately for each coordinate. With $\mathbf{1}_N$ denoting an $N$-element column of ones, the centered matrix is $Y - \mathbf{1}_N\bar{Y}$. Its thin singular-value decomposition is

$$Y - \mathbf{1}_N\bar{Y} = USV^{\mathsf{T}}.$$

If $q \le \min(N, K)$ modes are retained by the factorization, $U$, $S$ and $V$ have shapes $(N, q)$, $(q, q)$ and $(K, q)$. SVD is a matrix factorization. Columns of $V$ are orthogonal directions in output space. The corresponding columns of $US$ are one scalar amplitude per training row. Leading modes carry the largest target variance.

For mode $j$, let $z_j = (Y - \mathbf{1}_N\bar{Y})V_{:j}$, a length-$N$ column. Sparse least squares chooses a small set of PCE columns and coefficients $c_j$ so that

$$\Phi c_j \approx z_j.$$

The implementation adds terms greedily and judges each candidate model by leave-one-out error. Conceptually, one row is omitted, the remaining rows are fit and the omitted row is predicted. The prediction residual sum of squares (PRESS) identity calculates these errors from the ordinary fit's residual and diagonal influence without performing $N$ complete refits. This influence is the diagonal of the fitted model's projection matrix. It measures how strongly a row influences its own fitted value.

For a concrete four-row fit, suppose the ordinary fit has residuals $e = (0.06, -0.04, 0.02, -0.03)$ and projection diagonals $h = (0.20, 0.25, 0.10, 0.40)$. The PRESS identity says the residual obtained by leaving out row $i$ is $e_i/(1 - h_{ii})$: Squaring and summing the final column gives the PRESS score. A high-influence row has $h_{ii}$ near one, so its denominator is small and its omitted-row error is amplified. This calculation explains why the algorithm can score a candidate term set without executing four separate fits in this example.

The threshold `loo_max` receives a dimensionless relative mean-square error,

$$\mathrm{LOO}_{\mathrm{rel}} = \frac{N^{-1}\sum_i [e_i/(1 - h_{ii})]^2}{\mathrm{Var}(z_j)},$$

where $\mathrm{Var}(z_j)$ is the population variance of the target mode amplitudes over training rows. For the four final-column values above, the mean squared omitted-row residual is approximately 0.002866. If the mode variance is 0.04, the relative LOO is $0.002866/0.04 \simeq 0.0716$. A threshold of 0.05 rejects that mode. Zero target variance is refused because a relative error is undefined there. A leverage value outside $[0, 1)$ is also refused; it is not clipped to manufacture a finite denominator.

Only modes whose relative leave-one-out error is below `loo_max` enter the base. Rejected modes are left for the neural refiner. Reconstruction is

$$\widehat{Y}_{\mathrm{PCE}} = \mathbf{1}_N\bar{Y} + \Phi C_{\mathrm{mode}} V_{\mathrm{kept}}^{\mathsf{T}}.$$

This mode-first design directs polynomial capacity toward the dominant smooth directions of the covariance-whitened target.

*a. What happens when every PCE mode fails selection.* If every attempted mode misses `loo_max`, construction raises an error that names the limit, the best attempted score and mode, and the complete set of attempted scores. No fallback mode is constructed. Because geometry construction precedes neural training and artifact publication, this refusal writes no new emulator artifact.

| Omitted row | $e_i$ | $h_{ii}$ | leave-one-out residual |
|---|---|---|---|
| 0 | 0.06 | 0.20 | 0.0750 |
| 1 | −0.04 | 0.25 | −0.0533 |
| 2 | 0.02 | 0.10 | 0.0222 |
| 3 | −0.03 | 0.40 | −0.0500 |

TABLE XIII. A four-row PRESS calculation. Each final-column entry is the residual that would be observed if the named row were omitted and the same active polynomial columns were refitted. PRESS obtains it from one ordinary fit and derives all four omitted-row residuals.

The input bounds and coefficient matrix are stored as 32-bit floating-point values. The training design is built with those stored bounds, and acceptance recomputes PRESS after each selected coefficient is rounded and the dense design–coefficient product is evaluated in the saved precision. After the retained columns are assembled, it repeats the product with the complete coefficient matrix because floating-point reduction can differ with matrix shape. A mode that crosses `loo_max` during the joint check is removed, the narrower matrix is built, and the joint check repeats. Construction fails only if no mode remains. Thus a rejected mode is left for the neural refiner even when its 64-bit precursor or one-column product was just below the limit.

Selection maintains a set of distinct active indices. When every usable candidate term is active, no candidate remains and the loop terminates. Filling unavailable scores with a sentinel and applying unconditional `argmax` can otherwise return column zero and append an already active term. The active count is also capped at $N - 1$ for $N$ training rows. A saturated $N$-coefficient interpolation would leave too little independent information for a meaningful omitted-row check.

The basis is refit independently for every training-size sweep point. Reusing a fit from the largest point leaks rows into smaller points and invalidates the learning-curve comparison.

In neural PCE, abbreviated NPCE, the PCE is a frozen base and the neural network learns its residual. The word "neural" refers to the learned correction. The polynomial base itself is a linear combination of fixed basis functions after fitting. Write the neural correction as $r_{\boldsymbol{\varphi}}(x)$, where $\boldsymbol{\varphi}$ denotes trainable network parameters. This notation is deliberately different from the cosmological parameter vector $\boldsymbol{\theta}$. In additive form,

$$\widehat{t}(x) = t_{\mathrm{PCE}}(x) + r_{\boldsymbol{\varphi}}(x).$$

In multiplicative form, the domain of the base and correction must make the product meaningful. The artifact records the allowed input domain and the combination law. Inference may clamp or refuse an out-of-domain point only according to that persisted policy. An undocumented clamp silently changes the mathematical model.

### D. Residual and ratio modes

In residual mode, the neural target is

$$\delta(x) = t(x) - t_{\mathrm{PCE}}(x).$$

In ratio mode it is based on $t/t_{\mathrm{PCE}}$ or the declared gain form, so zeros and signs of the base require an explicit policy. That policy controls any transition to addition near a small denominator.

The loss reconstructs the complete prediction before computing physical $\Delta\chi^2$. Scoring only correction coordinates can favor a base whose scale makes the residual numerically small without improving the physical prediction.

### E. Persistence and inference

The artifact stores input-domain facts, basis family, ordered multi-indices, selected terms, coefficient matrix, combination mode and correction recipe. The predictor evaluates base and correction on the same encoded input.

Out-of-domain behavior is named and persisted. If clamping is the declared approximation, its bounds are stored and reporting identifies a clamped request. If the model is valid only inside its training domain, prediction raises outside it. Coefficient values alone cannot reveal this policy.

*a. Safe use of the current NPCE domain.* The predictor clamps every mapped coordinate to $[-1, 1]$, while the artifact does not name that policy. Distinct points beyond one fitted boundary can therefore produce the same polynomial-base input. Restrict prediction to the persisted fitted box and refuse an out-of-box request in the calling code. Clamped extrapolation should not be interpreted as a measured property of the trained model.

## IX. LEARNABLE ACTIVATION FUNCTIONS AND THEIR GENERALIZATIONS

An activation function is the scalar nonlinear map placed between learned linear layers. Scalar means that the function is applied independently to every feature value. Learnable means that the curve itself contains `nn.Parameter` tensors updated by the optimizer. In this

library every hidden feature receives its own curve parameters, so two columns in the same layer can learn different left-tail slopes, transition positions or tail exponents.

The implemented families form a hierarchy. The baseline $H$ has one sigmoid transition and linear tails. Multi-gate adds movable transitions in the bulk. Power-gated $H$ gives the tails a bounded learned exponent. The gated-power family combines both extensions.

### A. The baseline $H$: an identity-to-Swish interpolation

Define the logistic sigmoid

$$\sigma(z) = \frac{1}{1+\exp(-z)}.$$

Its value lies strictly between zero and one. The baseline activation is

$$H(x) = [\gamma + (1-\gamma)\sigma(\beta x)]\, x. \tag{26}$$

All operations are elementwise when $x$, $\gamma$ and $\beta$ are feature vectors. Equation (26) can also be written

$$H(x) = \gamma x + (1-\gamma)\, x\sigma(\beta x).$$

The second term is a Swish-shaped branch. When $0 \le \gamma \le 1$, $\gamma$ interpolates between an identity branch and a Swish branch. The stored parameter may lie outside that interval. There the same formula is an affine extrapolation. Convex interpolation applies only within the stated interval. $|\beta|$ sets how quickly the sigmoid gate changes around zero, while the sign of $\beta$ chooses whether it rises or falls.

For positive $\beta$, the left limiting slope is $\gamma$ and the right is one. For negative $\beta$, those two limits swap. At $\beta = 0$, the sigmoid is one half everywhere and both tails have the constant slope $(1+\gamma)/2$. In every case the tails remain linear. This differs from `tanh`, whose output approaches a constant and whose derivative approaches zero on both tails.

The implementation initializes $\gamma = 0$ and $\beta = 0$. Since $\sigma(0) = 1/2$, every feature begins as $H(x) = x/2$. The initial map has derivative $1/2$ everywhere and therefore transmits a gradient even when its input is zero. Training then changes $\gamma$ and $\beta$ independently for each feature.

### B. Multi-gate: learn several transitions in the bulk

$H$ contains one sigmoid transition fixed at the origin and ties the positive-tail slope to one. The multi-gate generalization replaces that single transition with a sum of $G$ learned sigmoids:

$$\begin{aligned} g_G(x) &= a_0 + \sum_{g=1}^{G} w_g\, \sigma[\beta_g(x-\mu_g)]\,, \\ A_G(x) &= g_G(x)x. \end{aligned} \tag{27}$$

Here $G$ is the number of learned sigmoid transitions. It is unrelated to the global output width $K$. Each term has three roles. $w_g$ controls how much the local slope schedule changes, $|\beta_g|$ controls how sharp the change is, the sign of $\beta_g$ chooses whether the sigmoid rises or falls and $\mu_g$ moves its center away from zero. $a_0$ supplies a baseline gate value.

The output remains asymptotically linear because every sigmoid is bounded. A gate with $\beta_g > 0$ contributes zero on the left and $w_g$ on the right. A gate with $\beta_g < 0$ contributes $w_g$ on the left and zero on the right. A gate with $\beta_g = 0$ contributes $w_g/2$ to both tails. Therefore the left and right limiting gate values are

$$\begin{aligned} g_- &= a_0 + \sum_{\beta_g<0} w_g + \frac{1}{2}\sum_{\beta_g=0} w_g, \\ g_+ &= a_0 + \sum_{\beta_g>0} w_g + \frac{1}{2}\sum_{\beta_g=0} w_g, \end{aligned}$$

and $A_G(x) \sim g_- x$ on the left and $g_+ x$ on the right. The general form can therefore learn a piecewise-smooth slope schedule through the bulk without inheriting `tanh`'s flat tails.

$H$ is contained in Eq. (27): choose $G = 1$, $a_0 = \gamma$, $w_1 = 1-\gamma$, $\mu_1 = 0$ and $\beta_1 = \beta$. The implemented initialization uses a convenient common start: $a_0 = 0$, the first gate has $w_1 = 1$ and $\beta_1 = 0$ and extra gates have zero weight. The total gate is again $1/2$, so the output is $x/2$. Extra centers are spread across a finite interval and become active only if training moves their initially zero weights.

### C. Bounded power tail: learn how rapidly the tails grow

Multi-gate changes the slope schedule but retains linear tails. The power generalization changes a different property: the magnitude growth far from zero. First define the signed power transform

$$\psi_p(x) = \operatorname{sign}(x)\frac{(1+|x|)^p - 1}{p}. \tag{28}$$

The base $1+|x|$ is at least one, so a fractional real exponent does not take a power of a negative number. Dividing by $p$ gives $\psi_p'(0) = 1$ for the mathematical function at every allowed exponent. Changing $p$ reshapes the tail while preserving the analytic local first-order behavior at zero.

A naive tensor expression for this transform would not reproduce that statement. Writing it as `sign(x)` times a function of `abs(x)` looks equivalent, but PyTorch assigns zero subgradients to `sign` and `abs` at exactly zero, so automatic differentiation of that form reports a zero derivative at precisely the input a zero-initialized correction head produces everywhere. Forward values cannot reveal the defect, because $\psi_1(x) = x$ holds either way; only

the derivative of the executed expression can. The implementation therefore factors the transform as $x$ times an even magnitude ratio, $[(1+|x|)^p - 1]/(p\,|x|)$: the direct quotient away from the origin, and its quadratic Taylor series below $|x| = 10^{-3}$, whose truncation error at the boundary is smaller than `float32` resolution. Each `torch.where` branch receives a safe substituted input, so no unselected position can poison a gradient with NaN. Automatic differentiation of this form returns exactly one at the origin for every allowed exponent, so a zero-initialized correction head receives its first learning signal. When adding any new activation intended for that head position, evaluate the derivative of the executed tensor expression at zero and require a finite nonzero result before accepting it.

The exponent is trained through a bounded parameter. The stored parameter $\rho$ is mapped through a sigmoid:

$$p = p_{\min} + (p_{\max} - p_{\min})\sigma(\rho). \tag{29}$$

With the default interval $[0.5, 1.5]$, $p$ ranges between square-root-like sublinear growth and mildly superlinear growth. At $\rho = 0$, $p = 1$ and $\psi_1(x) = x$. The power activation is

$$P(x) = [\gamma + (1-\gamma)\sigma(\beta x)]\,\psi_p(x). \tag{30}$$

It therefore recovers $H$ when $p = 1$. This is a bounded generalization: it allows the tail to adapt without allowing an arbitrary exponent to run to a numerically explosive value. The word *bounded* describes the exponent. The floating-point result can still overflow. A sufficiently large finite `float32` input can still overflow when $p > 1$. The implementation therefore needs both protections: constrain $p$ at construction and reject a nonfinite activation at the numerical boundary. The two protections cover separate failure modes.

### D. Gated power: combine bulk and tail freedom

The two extensions act on orthogonal aspects of the curve. Multi-gate changes where and how the gate transitions in the central region. The analytic function $\psi_p$ changes the tail-growth law while keeping unit slope in its mathematical limit at the origin. The current `sign`/`abs` tensor expression has the zero-backward exception described above. The combined activation is

$$GP_G(x) = g_G(x)\psi_p(x). \tag{31}$$

It is the most flexible implemented family and also has the most learned shape vectors. Its additional parameters enlarge the search space seen by the optimizer, so an activation bake-off must measure whether the added freedom improves validation behavior.

### E. Parameter count and qualitative comparison

The count matters because a network creates a fresh activation module at every activation site. For width $W = 128$, $G = 3$ and $L$ sites, multi-gate adds

$$(3G+1)WL = 1280L$$

learned scalars. Those values are small compared with a $128 \times 128$ dense matrix at each site, but they are not free and their gradients enlarge the optimization problem.

### F. PyTorch shape mechanics in the multi-gate code

For an input of shape $(\mathtt{B}, \mathtt{W})$, the multi-gate parameter tensors have shape $(\mathtt{G}, \mathtt{W})$. The code calls `x.unsqueeze(-2)`, which inserts a size-one axis immediately before the feature axis:

$$(\mathtt{B}, \mathtt{W}) \longrightarrow (\mathtt{B}, \mathtt{1}, \mathtt{W}).$$

Broadcasting against $(\mathtt{G}, \mathtt{W})$ then produces $(\mathtt{B}, \mathtt{G}, \mathtt{W})$ without explicitly copying the input $G$ times. The sigmoid is evaluated for every sample, gate and feature. Summing over axis `-2`, the gate axis, returns $(\mathtt{B}, \mathtt{W})$. Finally the gate is multiplied elementwise by the original input.

This implementation is eager tensor algebra: every expression executes when `forward` is called, with no `yield` boundary. The parameter tensors live on the same device and use the same dtype as the module because they are registered `nn.Parameter` objects.

### G. How the family is selected and persisted

`make_activation(name,n_gates)` returns a factory with the interface `act(dim)` returning a new module. For `relu` and `tanh`, the factory's `dim` argument is unused because those classes have no per-feature parameters. Keeping the same callable interface lets residual-block construction remain uniform.

The resolved artifact stores the family name and gate count. Rebuilding calls the same factory before loading the state dictionary. This order is required because the state dictionary contains tensor values. The stored family name supplies the Python class that gives them meaning. A warm start inherits the source artifact's activation family. Silently rebuilding identical tensor shapes under a different formula would create a different function while making weight loading appear successful.

### H. A fair activation comparison

An activation bake-off holds the data subset, model architecture, optimizer rules, schedule, random-seed policy and evaluation statistic fixed while changing only the activation family. One final score is insufficient. The driver

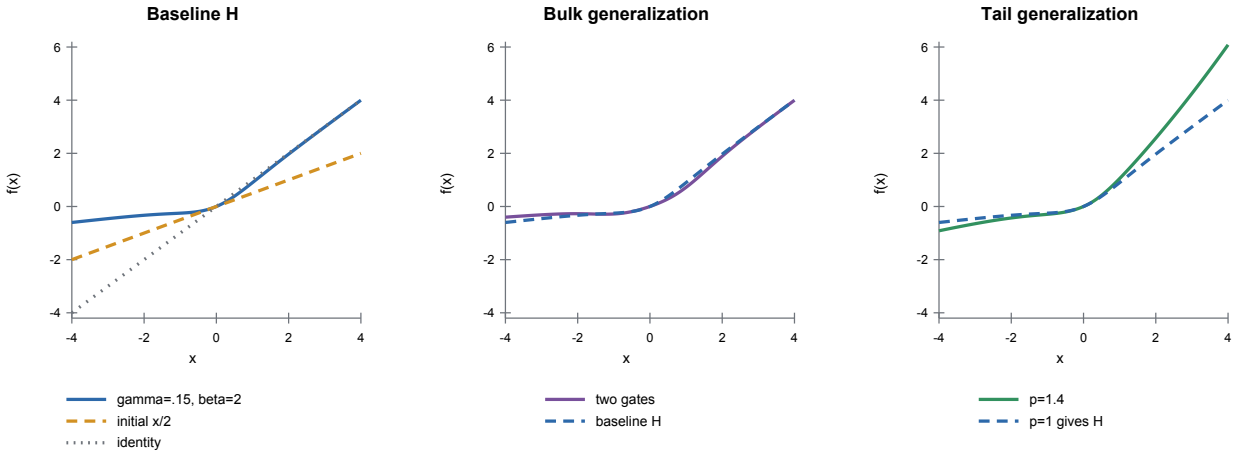



FIG. 7. Representative activation curves. Multiple gates can introduce more than one transition in the bulk. The bounded power changes the tail law while the analytic signed transform preserves its local slope at zero. The current PyTorch expression's exact-zero backward derivative equals zero, as explained in the text. The plotted forward curve cannot display that autograd exception. The plotted values illustrate mechanisms. Learned parameters are feature-specific and need not equal these examples.

| Family | Learned scalars per feature | Bulk behavior | Tail behavior | Primary comparison |
|---|---|---|---|---|
| $H$ | 2 | One signed transition centered at zero | Linear. Slopes are $(\gamma, 1)$ for $\beta > 0$, swapped for $\beta < 0$ | Sufficiency of the compact non-saturating default |
| Multi-gate | $3G + 1$ | $G$ movable transitions | Linear, both limiting slopes learned | Restriction imposed by one central bend |
| Power | 3 | The same single gate as $H$ | Signed $\lvert x\rvert^p$-like growth with bounded $p$ | Effect of tail curvature on the fit |
| Gated power | $3G + 2$ | $G$ movable transitions | Bounded learnable power growth | Joint effect of both restrictions |
| ReLU | 0 | One hard kink. Negative side exactly zero | Zero on the left, linear on the right | Comparison with a parameter-free rectifier |
| tanh | 0 | Smooth S-curve | Saturates to constants | Comparison with a classical smooth but saturating baseline |

TABLE XIV. The implemented activation families. Per feature means that a layer of width $W$ owns the listed count multiplied by $W$.

trains each family at several values of $N_{\text{train}}$ and plots the validation failure fraction against training-set size. A family whose curve is lower across the range learns more accurately at the same simulation budget. Curves that cross reveal a data-regime dependence.

The comparison also checks liveness. A structured correction head that remains equal to its trunk may still produce a respectable aggregate score. The gate must therefore verify that at least one head weight receives a nonzero finite gradient and changes after an optimizer step. This is especially important for ReLU: its derivative at exactly zero is zero, so a ReLU placed after an exactly zero-initialized final head layer can make the head permanently inactive even though the trunk continues to learn.

## X. THE LOSS: WHERE NETWORK ERROR BECOMES A SCIENTIFIC ERROR

The model emits network coordinates. The loss decides whether their error is acceptable in the physical likelihood. It is therefore not an interchangeable training convenience.

### A. Plain covariance-weighted loss

For a prediction $\widehat{T}$ and encoded truth $T$, both with batch-first network shape, the geometry can return both to physical kept coordinates. The residual matrix is

$$R = \widehat{V}_{\text{keep}} - V_{\text{keep}} \in \mathbb{R}^{B\times K}. \tag{32}$$

The slice $r_i = R[i : i+1, :]$ preserves a length-one batch axis and has shape $(1, K)$. Integer indexing $R[i, :]$ removes that axis and returns shape $(K, )$. With $\Sigma_{\text{keep}}^{-1}$ of shape $(K, K)$, its per-sample statistic is

$$\Delta\chi_i^2 = r_i \Sigma_{\text{keep}}^{-1} r_i^{\mathsf{T}}. \tag{33}$$

It must be finite and nonnegative, apart from a small negative roundoff band derived from floating-point precision and contraction width.

For a contraction executed in floating-point dtype $d$, the current domain band is

$$\delta_K = \max\left(10^{-6}, 32\,\epsilon(d)\,K\right),$$

where $\epsilon(d)$ is machine epsilon and $K$ is the kept per-row width. The bound scales linearly with $K$. The rejected tolerance mutation replaces that factor with $K^2$. Values in $[-\delta_K, 0)$ are normalized to exact zero as roundoff. A value below $-\delta_K$, NaN or infinity is refused before transformation, ranking or reporting.

When the geometry is whitened exactly, Eq. (18) reduces this to a sum of squared whitened residuals. The direct physical contraction remains the independent reference used to verify that reduction.

Correlation changes the answer even when marginal variances are unchanged. For two kept coordinates, choose

$$\Sigma_{\text{keep}} = \begin{pmatrix} 4 & 1 \\ 1 & 1 \end{pmatrix}, \qquad \Sigma_{\text{keep}}^{-1} = \frac{1}{3}\begin{pmatrix} 1 & -1 \\ -1 & 4 \end{pmatrix},$$

and residual $r_i = (1, 2)$. The complete contraction is

$$\Delta\chi_i^2 = (1,2)\frac{1}{3}\begin{pmatrix} 1 & -1 \\ -1 & 4 \end{pmatrix}\begin{pmatrix} 1 \\ 2 \end{pmatrix} = \frac{13}{3} \simeq 4.333.$$

Using only the diagonal variances would give $1^2/4 + 2^2/1 = 4.25$ and would silently discard the correlation. Whitening is allowed to make the final calculation look like a sum of squares only because the whitening matrix was constructed from the complete covariance and the direct result agrees.

## B. The training objective and the reported metric are distinct

The program may transform $\Delta\chi^2$ before averaging it over a minibatch. For example, a square-root objective reduces the influence of catastrophic rows during optimization. This transformation changes how gradients vote. The reported per-sample $\Delta\chi^2$ stays unchanged.

The training reduction must handle an exact fit without producing the undefined derivative $0/0$. A safe square-root implementation preserves the forward value and defines the exact-fit gradient as zero. A negative or nonfinite input is rejected before the transformation.

The executed construction first forms a Boolean mask `positive = (c > 0)`. It builds `c_safe = where(positive, c, 1)`, takes `sqrt(c_safe)` and then selects that square root only for positive entries. The zero branch uses `c - c`, whose value and derivative are both zero. Replacing this with `sqrt(c + eps)` would change every positive forward value and would make an exact fit nonzero. Writing only `where(c > 0, sqrt(c), 0)` is also unsafe on some backward paths because the unselected `sqrt(0)` branch can still create an infinite derivative that later combines with zero as NaN.

### 1. *Worked batch: three scientific scores, two summaries*

Suppose a minibatch contains three valid per-row scores

$$(\Delta\chi_1^2, \Delta\chi_2^2, \Delta\chi_3^2) = (0.04, 1, 9).$$

The mean physical score is $(0.04 + 1 + 9)/3 \simeq 3.347$ and the median physical score is 1. Both numbers differ from the square-root training objective. If the configured reduction averages $\sqrt{\Delta\chi^2}$, the optimizer receives

$$\frac{\sqrt{0.04} + \sqrt{1} + \sqrt{9}}{3} = \frac{0.2 + 1 + 3}{3} = 1.4.$$

The largest row supplies $3/(0.2 + 1 + 3) \simeq 71.4\%$ of the square-root numerator, compared with $9/(0.04+1+9) \simeq 89.6\%$ of the raw chi-square numerator. The transformation changes how rows compete for one optimizer step. It does not authorize reporting 1.4 as a median or as a chi-square.

The tensor holding the three row scores retains one element per sample until the reduction. That distinction matters for optional trim and focus: those features change which row values enter the scalar objective, while validation continues to publish the complete, untrimmed physical-score distribution.

## C. BerHu: square-root bulk with stronger tail pressure

BerHu means *reversed Huber*. Its input $c$ is one nonnegative per-cosmology chi-square. A signed residual is a different input type. Given a positive lower knot $t_1$ measured in the same chi-square units, the ordinary form is

$$v_{\text{BerHu}}(c) = \begin{cases} \sqrt{c}, & 0 \le c \le t_1, \\ \dfrac{c + t_1}{2\sqrt{t_1}}, & c > t_1. \end{cases}$$

At $c = t_1$, both branches have value $\sqrt{t_1}$ and derivative $1/(2\sqrt{t_1})$. The value and slope are continuous. This is the meaning of $C^1$ continuity. Below the knot, an ordinary nonzero row has the square-root derivative. Above it, the constant derivative gives hard rows stronger pressure than a square root would.

The capped form introduces a second knot $t_2 > t_1$:

$$v_{\mathrm{cap}}(c) = \begin{cases} \sqrt{c}, & 0 \le c \le t_1, \\ \dfrac{c + t_1}{2\sqrt{t_1}}, & t_1 < c \le t_2, \\ \dfrac{2\sqrt{t_2 c} + t_1 - t_2}{2\sqrt{t_1}}, & c > t_2. \end{cases}$$

The third branch matches both value and slope at $t_2$, then grows only like $\sqrt{c}$. "Capped" therefore means that the tail's gradient pressure is bounded relative to the linear middle branch. The loss value itself does not become a constant.

For an illustrative $t_1 = 0.25$ and $t_2 = 4$, the three inputs $c = (0.04, 1, 9)$ exercise the three regions:

$$v_{\mathrm{cap}}(c) = (0.2,\ 1.25,\ 8.25).$$

The first value is $\sqrt{0.04}$. The second is $(1 + 0.25)/(2\sqrt{0.25})$. The third is $[2\sqrt{4 \times 9} + 0.25 - 4]/(2\sqrt{0.25})$.

An optional schedule blends from the plain square-root objective to the selected BerHu curve:

$$v_s(c) = (1 - s)\sqrt{c} + s\, v_{\mathrm{BerHu/cap}}(c), \qquad 0 \le s \le 1.$$

Thus $s = 0$ is exactly the square-root mode and $s = 1$ is exactly the final BerHu mode. The blend is evaluated per row before trim, focus weighting and the final normalized minibatch reduction.

### D. Physics-aware loss wrappers

Several wrappers modify how a prediction is assembled before the same scientific metric is evaluated.

The rule is the same in every case: the reported $\Delta\chi^2$ must refer to the final physical prediction. A preprocessing amplitude is not allowed to multiply the metric accidentally.

### E. Frozen bases and packed targets

A frozen base can run once during staging. The loss evaluates it under `torch.no_grad()`, which omits those operations from backward differentiation. It packs the base output beside the truth:

$$T_{\mathrm{packed}} = \mathrm{concat}_{\text{last axis}}(T_{\mathrm{base}}, T_{\mathrm{truth}}).$$

For two ordinary $(B, K)$ blocks, the result has shape $(B, 2K)$. The operation does not add rows or concatenate along the batch axis. A family with a different base layout records and owns its exact split width. The minibatch loss unpacks both pieces and evaluates only the small correction network with gradients enabled. In a refine stage the base becomes live, enters the computation graph and receives its own optimizer parameter group.

### F. The finite contract

A NaN comparison with a positive threshold is false. Without an explicit guard, a diverged row can therefore be counted as below every error threshold and appear perfect. The code rejects nonfinite losses, gradients, validation rows, reductions and parity comparisons before they can rank or save a model.

The ordering is important:

1. compute the loss,
2. reject a nonfinite loss,
3. run backward,
4. inspect unscaled gradients for finiteness,
5. clip finite gradients when clipping is enabled,
6. update parameters,
7. apply any anchor.
8. update the exponential moving average.

If a step is rejected, none of the later state may advance.

## XI. OPTIMIZER, LEARNING RATE, SCHEDULER AND CLIPPING

The loss produces derivatives. The optimizer turns them into parameter updates. These are different responsibilities. Backward differentiation writes a gradient into each trainable parameter's `.grad` field. AdamW reads those fields, updates running first and second moments, applies decoupled weight decay to the declared group and changes the parameters.

### A. Batch-scaled learning rate

The configured base learning rate $\ell$ is anchored at batch size $B_0$. For actual training batch size $B$, the resolved rate is

$$\eta = \ell\sqrt{\frac{B}{B_0}}. \tag{34}$$

The square-root rule acknowledges that averaging more samples reduces gradient noise without assuming a fully linear scaling. The factory computes $\eta$ and injects it into the optimizer. Persisting only $\ell$ would leave the executed step size unspecified. The resolved record carries both the inputs and resulting value.

A warmup ramps the rate from zero to $\eta$ during the declared early epochs. The ramp is evaluated before the optimizer step it controls. An off-by-one implementation that steps first and then asks whether warmup is complete executes one unintended full-rate update.

| Wrapper | Operation before the final residual |
|---|---|
| Analytic rescaling | Divide out a known approximate cosmology dependence for training and multiply it back before scoring. |
| Factored IA | Combine neural templates with exact amplitude-polynomial coefficients. |
| CMB amplitude law | Apply the persisted amplitude and optical-depth law before comparing physical spectra. |
| NPCE residual | Evaluate a polynomial-chaos base and add or multiply the network correction. |
| Transfer loss | Combine a saved base emulator with a new correction network. |

TABLE XV. Physics-aware loss wrappers and the operation each performs before the final physical residual is formed. Every row must still report the same per-cosmology covariance-weighted $\Delta\chi^2$. A wrapper may change model coordinates or training gradients. The reported metric retains its declared meaning.

### B. AdamW state and module-role weight decay

AdamW maintains moving estimates of the gradient and squared gradient for each parameter. Those optimizer tensors are state: a complete rewind restores model weights and their matching moments.

For one scalar neural-network parameter $w$, let $g_t$ be its gradient at step $t$. AdamW updates a first moment $m_t$, a smoothed estimate of gradient direction, and a second moment $v_t$, a smoothed estimate of squared gradient magnitude:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t,$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2.$$

Because both start at zero, bias-corrected values are $\widehat{m}_t = m_t/(1-\beta_1^t)$ and $\widehat{v}_t = v_t/(1-\beta_2^t)$. Ignoring optional fused execution, one decoupled update is

$$w_t = (1-\eta\lambda) w_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}.$$

Here $\eta$ is learning rate, $\lambda$ is weight decay and $\epsilon$ keeps the adaptive denominator nonzero. Loss tolerances have a separate owner.

For a first step with $w_0 = 3$, $g_1 = 2$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.01$, $\lambda = 0.1$ and $\epsilon = 10^{-8}$, the raw moments are $m_1 = 0.2$ and $v_1 = 0.004$, while the corrected moments are $\widehat{m}_1 = 2$ and $\widehat{v}_1 = 4$. The adaptive term changes the parameter by approximately $-0.01$ and the decoupled decay changes it by $-0.003$ to yield $w_1 \simeq 2.987$. The example shows why optimizer moments must be rewound with parameters: they control the next step even when the visible parameter value was restored.

Decoupled weight decay applies

$$w \leftarrow w - \eta\lambda w$$

beside the adaptive gradient update. The factory assigns decay by module role. True weight matrices in linear, convolutional and bin-linear modules may decay. Biases, normalization gains and activation-shape parameters remain exempt from weight decay. Module role supplies ownership more reliably than tensor rank: a matrix-shaped activation parameter is still a curve-control value and a custom weight owner may not match a generic rank heuristic.

*a. Protocol refusals in the optimizer and scheduler factories.* On CUDA, the factory selects the fused kernel implementation only when the chosen class's constructor accepts a `fused` argument; a specification that explicitly sets `fused` for a class without that argument is refused, naming the class and the repair. LBFGS is refused at construction, because its `step()` requires a loss closure while the training loop always calls `step()` with no argument. The scheduler factory refuses the per-batch schedulers `OneCycleLR` and `CyclicLR`, because the loop advances its scheduler once per epoch and a per-batch schedule stepped at that cadence would be silently stretched. Each refusal names the observed choice and a working alternative. A novel optimizer class outside the shipped configurations still deserves a one-step protocol check before staging a full data set.

### C. Plateau scheduling and rewind

`ReduceLROnPlateau` is a metric-driven scheduler. At the end of an epoch, it receives the raw validation median and decides whether improvement has stalled for its patience interval. Calling `step()` without that metric follows a different scheduler protocol and is therefore not a constructible drop-in replacement.

When rewind is enabled and the scheduler cuts the rate, the program restores the best coherent snapshot while keeping the newly reduced rate. A coherent snapshot contains live model weights, optimizer moments and the EMA state when present. The purpose is to leave a bad basin while preserving the scheduler's decision to take smaller steps.

For a concrete timeline, take initial learning rate $10^{-3}$, patience one and factor 0.5. Suppose smaller validation median is better: At the epoch-3 cut, rewind restores the coherent epoch-1 state and retains the new smaller rate. The next epoch therefore retries from the best state with a smaller step size.

| Epoch | Median | Scheduler interpretation | Rate after step |
|---|---|---|---|
| 1 | 1.00 | new best. Save coherent snapshot | $10^{-3}$ |
| 2 | 1.10 | first bad epoch. Patience consumed | $10^{-3}$ |
| 3 | 1.20 | second bad epoch. Cut | $5 \times 10^{-4}$ |
| 4 | 0.95 | new best after recovery | $5 \times 10^{-4}$ |

TABLE XVI. A four-epoch plateau-scheduler example. "Patience one" allows one bad epoch after the best result. The second consecutive bad epoch cuts the learning rate. Rewind restores the coherent epoch-1 state but retains the new smaller rate.

### D. Gradient clipping

Let $g$ denote the concatenated gradient across all trainable parameters. Global-norm clipping performs

$$g \leftarrow g \min\left(1, \frac{c}{\|g\|}\right),$$

where $c > 0$ is the configured ceiling. Multiplying the whole gradient by one scalar preserves its direction. A zero ceiling means clipping is off. Negative, NaN and infinite ceilings are invalid configuration. Zero is the sole off spelling.

Under float16 loss scaling, the order is scale loss, backward, unscale gradients, reject nonfinite unscaled gradients, clip the unscaled global norm and step. Clipping the scaled values would make the effective ceiling depend on the current scaler.

## XII. TRIM: TEMPORARILY EXCLUDE THE HARDEST ROWS

Trim removes a declared fraction of the largest per-sample losses before forming the training mean. It is a hard selection: excluded rows contribute no gradient in that minibatch. Validation never trims because its purpose is to report the full held-out distribution.

For batch size $B$ and fraction $q$, the integer number of rows kept is

$$k = \max(1, \text{round}[(1-q)B]).$$

Python and PyTorch use round-half-to-even: an exact fractional part 0.5 rounds to the nearest even integer. For $B = 10$, $q = 0.35$ gives 6.5 and therefore $k = 6$. With $q = 0.25$, the value 7.5 gives $k = 8$. The minimum of one prevents a small batch from becoming empty. The implementation sorts or selects by detached hardness and averages only those rows. The discrete ranking is detached from gradient calculation. The retained loss values still carry their gradients.

The trim fraction can follow a schedule: hold the start value, anneal to the end value, then remain at the end. Cosine interpolation changes with zero slope at both endpoints. Linear interpolation changes at constant rate. Step interpolation quantizes the ramp. Constant holds the start indefinitely. An increasing schedule is valid only if the owner intends it. The validator checks start, end, durations and shape before training so a NaN cannot disable the branch through a false ordered comparison.

*a. Safe use of stepped schedules.* The shared stepped evaluator applies `max(end, floored_value)`. A decreasing ramp follows the intended staircase, while an increasing ramp jumps to its final value at the first live epoch. This helper is shared by trim, BerHu and EMA. Use the step shape only for a decreasing schedule. Use the linear or cosine shape for an increasing schedule, inspect the printed values at intermediate epochs and supply finite durations and strictly ordered step fractions.

Trim protects early optimization from a small contaminated tail. It does not justify ignoring the removed rows forever. A nonzero final floor is an explicit outlier-resistant objective choice and the untrimmed validation distribution reveals whether those hard rows remained scientifically unacceptable.

#### Worked trim example

Take a four-row minibatch with differentiable losses $(0.2, 0.4, 1.0, 8.0)$ and a trim fraction $q = 0.25$. Here $(1-q)B = 0.75 \times 4 = 3$, so the rule keeps three rows and the detached ranking excludes the row whose current loss is 8.0. The scalar loss for backward is

$$(0.2 + 0.4 + 1.0)/3 \simeq 0.533.$$

"Detached ranking" applies only to the decision that chose indices 0, 1 and 2. PyTorch retains the computation graphs attached to the selected loss values, so those three rows update the network. The fourth row receives no gradient from this minibatch. Detachment removes gradients from the ranking calculation while preserving them for the selected losses.

The excluded row remains part of validation. If its score is still 8.0 at the end of the epoch, it appears in the validation median, mean and threshold fractions. Trim changes training votes. The definition of held-out accuracy remains fixed.

## XIII. FOCUS: SMOOTHLY WEIGHT SAMPLES BY HARDNESS

Focus is a continuous alternative to hard trimming. For per-sample chi-square $c_i$, it uses

$$w_i = m_i \left( \frac{c_i}{c_i + \kappa} \right)^{\gamma_+(e)}, \qquad \gamma_+(e) = \max[\gamma(e), 0]. \tag{35}$$

$\kappa$ is the chi-square scale at which the base fraction is one half and $\gamma(e)$ is an epoch-dependent exponent. The factor $m_i$ is one for a row retained by trim and zero for an excluded row. Clamping the exponent makes a negative off sentinel behave as zero and prevents inverse weighting of easy rows. At $\gamma_+ = 0$, every retained base is raised to zero, so the reduction is the plain mean. Increasing $\gamma_+$ emphasizes harder rows.

Let $v_i$ be the per-row loss after the selected mode transform: for example, $v_i = c_i$ in chi-square mode or $v_i = \sqrt{c_i}$ in square-root mode. The actual scalar sent to backward is the normalized weighted mean

$$\mathcal{L}_{\text{batch}} = \frac{\sum_i w_i v_i}{\sum_i w_i + \epsilon_w}, \qquad \epsilon_w = 10^{-12}. \tag{36}$$

The small denominator constant protects an all-zero weight sum while leaving each individual scientific chi-square unchanged.

The weights are detached from autograd before multiplying the losses. The current errors still decide voting strength during the forward calculation. During backward, each detached weight is treated as a constant. The gradient therefore follows the fitted loss value while the error-dependent weight acts as a fixed multiplier.

Focus and BerHu can both emphasize a tail. Using them together is a deliberate composition with two named controls. The resolved configuration and banner state both effects so a sweep can distinguish a changed loss curve from a changed sample-weight curve.

### Worked focus example

Let $\kappa = 1$, $\gamma = 1$, no trimming and chi-square mode so $v = c$. Rows with scores $c = (0.1, 1, 9)$ receive detached weights

$$w = \left( \frac{0.1}{1.1}, \frac{1}{2}, \frac{9}{10} \right) \simeq (0.091, 0.5, 0.9).$$

The high-error row gets nearly ten times the voting weight of the low-error row. The loss value for a row still depends on the model output, so backward can lower it. Autograd treats the current weight as a numerical coefficient and excludes the weight's derivative from the gradient.

The numerator of Eq. (36) is

$$0.091 \times 0.1 + 0.5 \times 1 + 0.9 \times 9 \simeq 8.609,$$

and the weight sum is approximately 1.491. The scalar objective is therefore $8.609/(1.491 + 10^{-12}) \simeq 5.774$. Reporting only the three weights would leave the backward scalar unspecified. The normalization completes the algorithm.

At $\gamma_+ = 0$, each positive base is raised to zero and becomes one. The ordinary retained-row mean is recovered. For an exact-fit row $c_i = 0$, the current PyTorch path evaluates $0^0$ as one. There is no separate branch in the implementation. That executed convention must be pinned by a test if it is part of the off-mode identity, because the symbolic expression alone is indeterminate.

## XIV. EXPONENTIAL MOVING AVERAGE: A SECOND, SMOOTHED MODEL STATE

After a successful optimizer step, EMA updates an averaged parameter copy

$$\bar{w} \leftarrow \beta_{\text{ema}} \bar{w} + (1 - \beta_{\text{ema}}) w, \qquad \beta_{\text{ema}} = \max\left(0, 1 - \frac{1}{H_{\text{ema}} S}\right). \tag{37}$$

$H_{\text{ema}}$ is the requested averaging horizon in epochs and $S$ is the executed number of optimizer steps per epoch. Expressing the horizon in epochs makes its physical training duration comparable across batch sizes. When $H_{\text{ema}} S < 1$, the unclamped expression would be negative. The executed maximum sets $\beta_{\text{ema}} = 0$, so the average becomes the current live parameter and avoids extrapolation.

The live parameters produce the current batch gradient. After the live optimizer step, the averaged copy is updated under no-gradient semantics. Evaluation and best-epoch selection may use the averaged state, while the plateau scheduler intentionally watches the raw model median. The artifact must say which state produced its selected metrics.

An annealed horizon avoids carrying early poor weights. During its hold, the average remains inactive according to the resolved schedule. After the live point, the horizon grows toward its requested value. Saving, rewind and resume treat $w$, optimizer state and $\bar{w}$ as one coordinated training record.

For a concrete scale, let the horizon be $H_{\text{ema}} = 10$ epochs and let one epoch execute $S = 100$ optimizer steps. Equation (37) gives $\beta_{\text{ema}} = 1 - 1/1000 = 0.999$. If one scalar parameter has averaged value $\bar{w} = 0$ and the accepted live step changes it to $w = 2$, then the new average is

$$0.999 \times 0 + 0.001 \times 2 = 0.002.$$

The live parameter remains 2, while EMA changes the separately stored averaged copy. Repeating many accepted steps moves that copy toward the trajectory while suppressing step-to-step noise.

The step count $S$ records executed optimizer updates. A nominal count based on source rows and batch size can

diverge when ragged chunks discard data. If the storage regime changes the number of executed steps, that nominal count would change the physical EMA horizon. The resolved record therefore needs the effective rows and effective steps that actually reached the optimizer.

## XV. LOADERS, MINIBATCHES AND THE TRAINING LOOP

The staged source is a host-side description. The training loop needs fixed size tensors on the model's device. Loaders bridge those representations.

### A. A loader is a closure

A closure is a function that retains values from the scope in which it was created. `build_loaders` returns functions with the interface

$$\texttt{load(rows)} \longrightarrow \text{device tensor.}$$

The closure remembers whether its source is GPU-resident, in host RAM or file-backed. The training loop does not branch on storage.

There are two loaders per source. `load_C` returns encoded parameter rows. `load_dv` returns encoded targets. Both accept row indices and return tensors on the compute device.

A common transfer line has the form

```
t_tr = torch.from_numpy(C_tr).float().to(device)
```

Read it left to right. `torch.from_numpy(C_tr)` creates a CPU tensor that initially shares storage with the NumPy array when its dtype is already compatible. `.float()` requests PyTorch `float32`. It allocates a converted copy when the source is not already `float32`. `.to(device)` places the resulting tensor on the selected CPU, CUDA GPU or Apple MPS device and copies storage when the device changes. The final name `t_tr` is an eager tensor whose device transfer has completed before the first minibatch.

### B. Three storage regimes

On CUDA, a host tensor may be pinned before transfer. Pinned memory cannot be paged out by the operating system, which allows an asynchronous device copy. Pinning is a host-memory property. It does not move the tensor to the GPU.

### C. Fixed batch shapes

Compiled PyTorch graphs work best with stable tensor shapes. Training uses full batches. Validation pads its final short batch with copies of a valid row, runs the fixed-size graph and discards the padded scores. Padding is not allowed to enter the reported median, mean or threshold fractions.

For one batch,

$$X_b = \texttt{load\_C(rows)}, \quad T_b = \texttt{load\_dv(rows)},$$
$$\widehat{T}_b = \texttt{model}(X_b).$$

`model(X_b)` uses positional syntax because the tensor is the direct subject of the call. Configuration-heavy calls use named arguments.

### D. One minibatch across the host-to-device boundary

Assume the staged source contains $P = 12$ cosmological parameters, the encoded target width is $K = 780$ and the training batch size is $B = 256$. The epoch permutation supplies a one-dimensional integer tensor or array named `rows` with shape $(256, )$. Each value is a row coordinate in the staged source. It is an eager collection of indices. All 256 integers already exist when the loader is called.

On a disk-streaming path, indexing a memmap reads the requested target values from the file into a bounded host array. Converting that array to `float32` and moving it to CUDA or MPS creates device storage. The output geometry then transforms the device tensor. On a device-resident path, the already encoded tensor is gathered directly. Both paths must return the same rows in the same order and preserve the same training sample.

The two loaders are closures because they remember this storage policy and the geometry objects. Calling a closure is an ordinary synchronous function call unless the implementation explicitly starts an asynchronous device transfer. A returned closure executes its body when its caller invokes it. This distinction separates a closure, which retains surrounding values, from a generator, which contains `yield` and produces values lazily across repeated iteration.

### E. Optimizer and scheduler

The optimizer owns parameter updates. Parameters are divided by module role: true linear and convolutional weight matrices may receive weight decay. Biases, affine gains and activation-shape parameters remain exempt.

The scheduler changes learning rates. `ReduceLROnPlateau` requires a validation metric in `scheduler.step(metric)`. A scheduler with a different protocol cannot be substituted merely because its constructor fits the same factory.

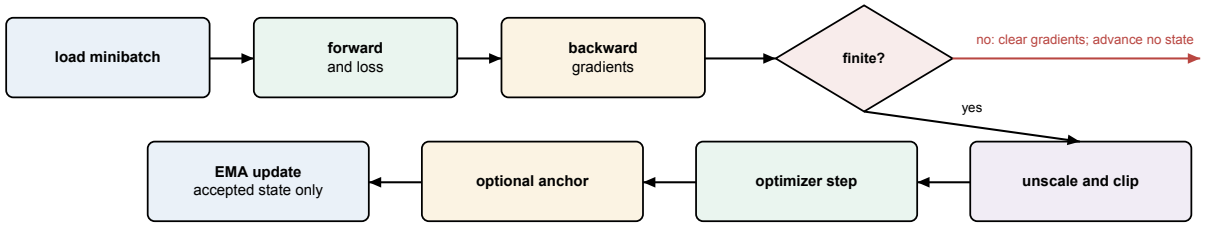



FIG. 8. One accepted training step. The order is part of the algorithm. Finiteness is checked before any persistent state advances. EMA is last because it summarizes the accepted post-step parameters.

| Regime | Persistent storage | Work performed for a minibatch |
|---|---|---|
| Device resident | Encoded parameters and targets on GPU/MPS | Gather rows by device indexing. |
| Host RAM stream | Encoded parameters resident. Raw targets in a NumPy array | Copy selected target rows to the device and encode them. |
| Disk stream | Encoded parameters resident. Raw targets in a memmap | Read a bounded host block, copy it to the device and encode it. |

TABLE XVII. The three loader storage regimes. "Persistent storage" means the representation retained between minibatches. Each returned batch is a PyTorch tensor on the model device with the same numerical and row-order contract regardless of regime.

### F. One optimizer step

The common plain-loss call names every argument. A full-precision step can be read as the following sequence:

```
prediction = model(inputs)
loss = lossfn.loss(
    pred=prediction,
    target=targets,
    mode=loss_mode,
    trim=trim_fraction,
    focus=focus_exponent,
    focus_scale=focus_scale,
    berhu_knot=berhu_knot,
    berhu_cap=berhu_cap,
    berhu_s=berhu_blend,
)
require_finite(loss)
optimizer.zero_grad(set_to_none=True)
loss.backward()
grad_norm = finite_gradient_norm(
    model.parameters(),
    clip=clip_limit,
)
require_finite(grad_norm)
optimizer.step()
require_finite_parameter_and_optimizer_state(
    model=model,
    optimizer=optimizer,
)
if anchor is not None:
    anchor.apply(optimizer)
if ema_state is not None:
    ema_state.update(model.parameters())
```

The parameter-aware loss families add `params_whitened=inputs`. The ordinary families use the prediction, target and reduction controls. The trim, focus and BerHu values are zero-dimensional tensors set once per epoch, which lets a compiled forward reuse one graph across changing epoch values.

This listing states the complete numerical contract. Some names, such as `require_finite_parameter_and_optimizer_state` and `ema_state.update`, are descriptive labels for multi-line owned operations. Each line still names its inputs and its position in the algorithm. No hidden callback fills an unexplained argument. `zero_grad` clears gradients accumulated by earlier backward calls. `backward` traverses the recorded computation graph and adds each parameter's derivative to its `.grad` field. Gradients accumulate by default, so omitting `zero_grad` changes the algorithm.

The finite checks form three distinct barriers. A nonfinite scalar loss is refused before backward can create corrupted gradients. A nonfinite gradient norm is refused before the optimizer mutates parameters. Parameters and optimizer moments are checked after the step because finite inputs to an update do not prove a finite result. Clipping, when enabled, acts on the unscaled gradient and must not turn a nonfinite norm into a finite-looking step.

| Stage | Object and shape | Storage | Operation |
|---|---|---|---|
| Selection | `rows`, $(256,)$ | CPU or device integers | Names the staged rows in the seeded epoch order. |
| Parameter gather | raw $C_b$, $(256, 12)$ | NumPy host array or device tensor | Reads the selected cosmologies without changing their order. |
| Parameter encode | $X_b$, $(256, 12)$ | Model device, `float32` | Centers, rotates and scales each row with the parameter geometry. |
| Target gather | raw $V_b$, $(256, D)$ | NumPy host array, memmap slice or device tensor | Reads physical targets. $D$ may include columns later masked. |
| Target encode | $T_b$, $(256, 780)$ | Model device, `float32` | Applies the family mask, target law, centering and whitening. |
| Forward | $\widehat{T}_b$, $(256, 780)$ | Model device, `float32` | Calls the network once for the whole batch. |
| Score | $c_b$, $(256,)$ | Model device. Reductions use declared precision | Produces one physical scientific error per row before trim, focus or mean. |

TABLE XVIII. One full minibatch. The table separates a row coordinate from the row payload and separates a physical target from its encoded network target.

Mixed-precision float16 training requires loss scaling. Scaling makes small gradients representable during backward. The gradients are unscaled before finiteness checks and clipping. Bfloat16 has a wider exponent range than float16, so its scaler policy is selected separately.

`model.train()` and `model.eval()` select a module's training or evaluation behavior. They do not enable or disable gradient recording. `torch.no_grad()` separately disables construction of the backward graph inside its context. Evaluation uses both `eval()` and `no_grad()`: the first chooses inference behavior and the second avoids retaining activations that no backward pass will consume. Returning to `train()` before the next epoch restores training behavior.

For the $(256, 780)$ example, `lossfn.loss` first returns one score for each of the 256 rows. The configured reduction produces a scalar. Calling `backward()` on that scalar uses the chain rule to accumulate derivatives through the reduction, loss and model. NumPy files, integer row indices and frozen geometry statistics remain data outside the trainable `Parameter` set.

*a. Limits of the current update chain.* The loop checks the scalar loss and gradient norm. A complete live proof of post-optimizer parameter and moment finiteness is still owed on the workstation. The anchor is applied before the EMA update, exactly as the listing above shows, so the averaged state tracks the same anchored weights the live model serves.

On Apple MPS, autocast means float16 and no gradient-scaling policy exists, so small early gradients from zero-start correction heads and soft gates could underflow to exact zero while the loss stayed finite. The mixed-precision policy resolver therefore refuses `use_amp` on an MPS device before any staging or training work starts, naming the device, the missing scaler and the corrective action. CUDA bfloat16 remains available unscaled, because its wider exponent range does not need the float16 scaler; that safety argument is specific to bfloat16 and never transfers to the refused MPS float16 branch.

### G. Epoch metrics and model selection

After every epoch, `eval_val` produces one $\Delta\chi^2$ per validation row. It then reports a median, mean and the fraction above each configured threshold. The even-sample median is the ordinary 50th percentile and averages the two central samples.

The best epoch is selected by the primary threshold fraction, with the median as the tie-breaker. The saved best record must contain the exact weights that produced the recorded metrics. If an exponential moving average is used, evaluation, selection, restoration and persistence must all refer to that same averaged state.

The threshold vector is validated once, before any model or loader work: it must be a nonempty flat sequence of finite nonnegative numbers in strictly increasing order. Strict increase rules out duplicates and makes index zero the tightest cutoff, the selection goal. Every later consumer of the vector trusts this single check.

The training loop returns its decision as an explicit selection record beside the history lists. The record names the restored candidate — the incoming weights, or a trained epoch — together with the raw-or-averaged weight identity and that candidate's own fractions, median and mean. This matters because the incoming weights are a real winner: their seeding evaluation guarantees a pass never ends worse than it started, yet that evaluation never enters the per-epoch history lists. A consumer that re-derived the winner by scanning histories would therefore name a trained epoch, and attach that epoch's statistics, even when the model on disk holds the untouched incoming weights. The run-level record adds the pass identity, the position of the winning epoch in the concatenated histories (zero when the final pass's

incoming weights won), and the threshold vector with its selection index, and it persists inside the artifact's resolved configuration. Drivers, tuning objectives and the saved run attributes all read this record; none re-derives a winner from the histories.

## XVI. TWO-PHASE TRUNK AND CORRECTION-HEAD TRAINING

A model with a correction head can run two phases. The trunk phase bypasses and freezes the head. The head phase may freeze the trunk or train trunk and head jointly, according to the resolved configuration. Each phase receives its own optimizer and learning-rate schedule. A frozen parameter has `requires_grad=False`, so PyTorch omits its gradient.

The phase boundary is a state transition whose banner reports it. Tests count trainable trunk and head parameters and verify which weights can change.

### A. Phase one: learn the complete baseline map

The trunk predicts the complete target on its own. During the trunk phase, the forward path bypasses the correction head. Only trunk parameters receive gradients and enter the phase optimizer.

This produces a scientifically meaningful intermediate model. If the head later fails to improve, the trunk still represents a full emulator. The trunk is the complete baseline and the head supplies an additional correction path from inputs to outputs.

### B. Phase two: expose the structured correction

At the phase transition, the head enters the forward graph. With a frozen trunk, its input is treated as fixed and only head parameters are updated by the optimizer. With joint training, both sets have `requires_grad=True` and appear in the new optimizer.

Each phase resolves its own learning rate, warmup, scheduler, loss transform, focus/trim controls and EMA policy according to the configuration precedence. The transition creates fresh optimizer state for the parameter groups it will update. The differently grouped head optimizer therefore starts with fresh Adam moments.

The correction's final learned layer begins at zero so phase-two epoch zero equals the trunk. The outer gate begins representably nonzero and the head activation must transmit a derivative at zero. These three facts together give exact initial identity and a live first gradient.

### C. Why model settings are checked in two stages

A YAML value can look reasonable to a reader while having the wrong Python type. For example, `rescale_kernel: false` is a Boolean switch, but `rescale_kernel: "false"` is text. Nonempty text normally acts like true in a Python condition. Silently accepting the quoted form could therefore enable the opposite of what the file says. Counts have the same problem: `n_blocks: 1` is an integer, while `n_blocks: "1"` is text.

The first validation stage examines the selected architecture before the program reads scientific files, chooses an accelerator or constructs learned layers. It requires real Booleans for switches and unquoted integers for counts. Selected CNN and transformer head depths must be positive. It also checks rules that need no output data, such as an odd CNN kernel width and the short allowed lists for normalization and compilation mode. A head block belonging to an unselected architecture remains unused, which lets one example file carry both CNN and transformer settings.

Some questions cannot be answered until the physical output has been read. The second stage therefore uses the finished output geometry. It checks, for example, that `n_heads` divides the transformer's actual padded token width and that a requested CNN grouping follows the physical output divisions. This later check is not a second interpretation of the YAML. It compares the already type-checked choice with a shape that did not exist at the first stage.

The correction head has a separate learning constraint. In a simplified notation its initial output is

$$\widehat{t} = \widehat{t}_{\text{trunk}} + g\, a(Wh), \qquad W = 0,$$

where $W$ is the zero-initialized final learned layer, $a$ is the head activation and $g$ is the outer gate. Because $a(0) = 0$, the new head changes nothing before training. Its first derivative with respect to $W$ contains

$$g\, a'(0)\, h^{\mathsf{T}}.$$

If either $g = 0$ or $a'(0) = 0$, that first learning signal vanishes. The gate must therefore be finite and must remain nonzero after conversion to the model's 32-bit format. Both `gate_init: 0` and a number such as `1e-50`, which rounds to zero in that format, are refused.

The accepted head activations are `H`, `multigate`, `tanh`, `power` and `gated_power`: each transmits a nonzero derivative at exactly zero input, the power families through the origin-exact factored transform behind Eq. (30). `relu` is refused at this zero-initialized head position because its derivative at zero is zero, which would leave the head frozen at its identity start. ReLU is still valid in the MLP trunk. A frozen two-phase run may use a ReLU trunk and explicitly pin the head to one of the accepted families.

### D. Frozen parameters remain present

A frozen trunk still executes forward to provide the head input. A no-gradient context omits its backward activations and reduces memory. Its registered buffers and evaluation mode still matter. The code must not mutate the source module in a way that changes later prediction.

Joint-mode evidence counts trainable trunk and head parameters, verifies optimizer membership, checks finite nonzero gradients after a discriminating batch and observes weight changes. Timing alone is only supporting evidence: workstation load can make a frozen epoch slow or a joint epoch fast.

### E. What is selected and saved

Validation after each phase evaluates the same final physical metric. The best trunk snapshot seeds phase two. The best final snapshot contains the exact model state that produced its recorded metrics, including EMA when enabled.

The artifact records one resolved phase configuration and final architecture. A single-phase model resolves or refuses phase-shaped keys according to its public rule. Invalid phase-shaped keys stop before a constructor with no head.

## XVII. STARTING FROM A SAVED EMULATOR: FINE-TUNING AND TRANSFER

Random initialization discards knowledge already stored in a trusted artifact. The library provides two reuse strategies. Fine-tuning adapts the source network itself. Transfer freezes the source and trains a separate correction. Both obey the same epoch-zero contract:

$$\widehat{d}_{\text{new}}(\boldsymbol{\theta}, e = 0) = \widehat{d}_{\text{source}}(\boldsymbol{\theta})$$

for every parity fixture before the first optimizer update.

### A. Fine-tuning extends the existing function

The fine-tune configuration names a source artifact and omits a new model description. Architecture, activation family, parameter order and output geometry are inherited from the source. Repeating them in YAML would create two potentially conflicting authorities.

If the new table contains additional input parameters, the input projection is expanded in a way that gives the new columns zero initial influence. The old input columns, hidden weights, output projection and output geometry retain their source values. Thus the initial function agrees exactly on the old parameter subspace. A smaller learning rate can then adapt all trainable weights to the extended data.

Output geometry is pinned because changing its center or whitening basis would change the numerical network coordinates represented by the loaded output weights. Centering cancels in a physical residual only when encode, network and decode agree on the same center. Recomputing one side from new data breaks epoch-zero parity even if the physical target family is unchanged.

An anchor, when supported by the validated public configuration, is a *decoupled post-optimizer update* outside the training loss. Let $w_j^{\text{opt}}$ be a parameter element immediately after the optimizer step, $w_{j,0}$ its persisted source value, $m_j$ a zero-or-one element mask, $\eta$ the current learning rate and $\lambda$ the anchor strength. The executed anchor operation is

$$w_j^{+} = w_j^{\text{opt}} - \eta\lambda m_j \left(w_j^{\text{opt}} - w_{j,0}\right).$$

For example, if $w_{j,0} = 2.0$, the optimizer produces $w_j^{\text{opt}} = 2.3$, $\eta = 0.01$, $\lambda = 4$ and $m_j = 1$, then

$$w_j^{+} = 2.3 - (0.01)(4)(0.3) = 2.288.$$

With $m_j = 0$, the anchor leaves the optimizer result at 2.3. Adam's first- and second-moment buffers retain the optimizer step's own values because the anchor follows AdamW. The literature calls this penalty L2-SP: an L2 (squared-distance) pull toward the Starting Point, where ordinary weight decay is the same pull toward zero. The loop applies the anchor before the EMA update, so the averaged state includes the same step's pull toward the source. The mask is created by canonical parameter names. Reporting distinguishes matched, masked, frozen and unexpected keys.

*a. Fine-tune anchor availability.* The public validator refuses `finetune.anchor` before training begins. The downstream mask and update machinery exists but is unreachable through the fine-tune configuration. Use unanchored fine-tuning, or use the separate `transfer.refine.anchor` control when transfer refinement is the intended mode. Adding `finetune.anchor` to a YAML file produces an error.

### B. Transfer keeps the base frozen

Transfer creates a new correction network that sees the complete new parameter space. The base receives only the named subset it was trained on. During ordinary correction training, base evaluation is performed once per training row under `torch.no_grad()` and cached with the target. No gradient graph or base activation storage is needed for each minibatch.

Two composition forms are supported conceptually:

$$\begin{aligned} &\text{sum:} && y = y_0 + r, \\ &\text{gain:} && y = y_0(1 + r). \end{aligned}$$

The composition space is equally important. In physical space, $y_0$ and $r$ are physical output values. In whitened space, they are geometry coordinates. Operation order matters when addition, multiplication and nonlinear output laws are combined, so the family validator accepts only combinations whose meaning is defined.

For factored intrinsic-alignment models, composition occurs per template before the exact amplitude polynomial combines the templates. Correcting only the already-combined vector would make the learned correction depend on the particular amplitude used to construct a target.

### C. Target packing is an internal storage contract

When the base is frozen, staging can concatenate cached base and truth arrays:

$$T_{\text{packed}} = [T_{\text{base}} \mid T_{\text{truth}}].$$

The vertical bar denotes concatenation along the last, feature axis. Logical or uses a separate operator. The loss owns the exact split width and unpacks the two blocks. A loader treats the packed target as an opaque tensor. Duplicating the split rule in the loader would create another authority.

Packing is eager: the cached base is computed for the selected rows during staging. The resulting array may be file-backed when its memory cost is too large. Its temporary-file lifecycle belongs to the experiment so repeated sweep points release superseded files promptly during staging.

### D. Optional joint refinement

For a supported family, refinement can unfreeze the base after the correction has converged. The optimizer then contains separate groups: correction parameters at the run learning rate and base parameters at a scaled rate. A base anchor limits drift from the original source.

*a. Safe use in a transfer-refinement sweep.* Refinement changes the shared transfer base in place. A family sweep that reuses one experiment can start a later point from weights and mode state changed by an earlier point. Construct a fresh experiment from the same source artifact for every refinement point. A forward-versus-reverse order comparison is independent only when both directions rebuild that source before each point.

The update order is optimizer, base anchor, then EMA, so the averaged state tracks the same anchored base the live model serves. The artifact records original base tensors and refined tensors. Treat its reported drift as provisional unless it was computed over the named trainable parameter set, without buffers or padding indices.

### E. Artifact self-containment

A transfer artifact embeds the base facts it needs at inference and removes dependence on an external path that may later point to different weights. The current two-file publication still lacks pair binding. A same-shaped weight file can be mixed with the wrong HDF5 recipe because neither file carries a shared payload identity. Keep the two files together under one immutable root, refuse partial copies and run a save/rebuild prediction comparison after moving them. The HDF5 record keeps the model recipe, saved geometries and analytic law, and composition mode. Retain the source commit when reproducing the exact software version matters.

## XVIII. THE FIVE OUTPUT FAMILIES

The training machinery is shared, but the physical object predicted by the network changes by family.

The word *family* identifies the meaning and shape of one target row. It does not select a separate optimizer or a separate definition of a minibatch. All five families use the same high-level order: resolve files, stage rows, build an input geometry, build an output geometry, construct a model, train, validate and save. The family hooks supply the output-specific facts that the shared order cannot infer.

Suppose two target files both have 780 columns. One may be a cosmic-shear vector ordered by probe and angular bin, while the other may be a CMB spectrum covering 780 multipoles. Their shapes agree, but no operation can interpret one as the other. The same problem occurs within a family: a CMB array for $\ell = 2, \ldots, 781$ has the same width as one for $\ell = 10, \ldots, 789$. A sidecar or artifact axis distinguishes them.

The shared trainer receives only encoded tensors after a family has established that identity. It can therefore remain ignorant of multipoles, redshift, wavenumber and probe names. A family adapter requires an explicit scientific axis in addition to the generic trainer's output width. This split is an ownership rule: generic code owns optimization mechanics. Family code owns the physical interpretation of coordinates.

## XIX. COSMIC SHEAR: A MASKED, COVARIANCE-COUPLED DATA VECTOR

The cosmic-shear geometry reads a likelihood covariance, mask, data-vector center and probe layout. A model predicts only the unmasked entries. The loss scatters their residual into the full vector and contracts the kept covariance block. Structured heads use the persisted angular/bin map.

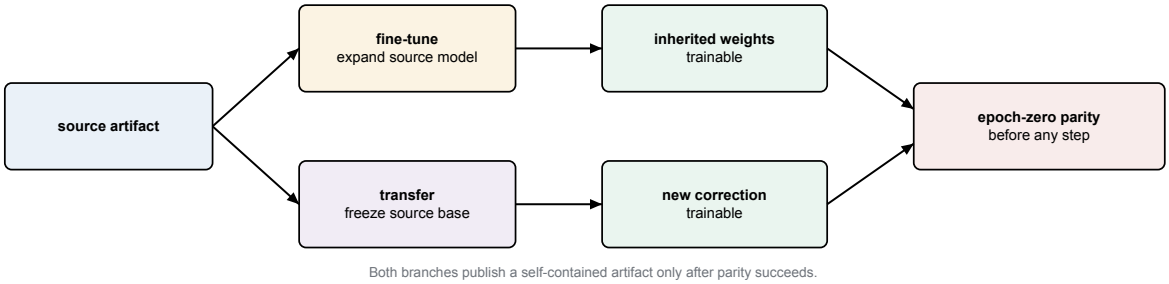



FIG. 9. The two warm-start strategies. They differ in which parameters may move, but both must reproduce the source function before training and publish a self-contained result afterward.

| Family | Raw target | Output axis | Physical reconstruction |
|---|---|---|---|
| Cosmic shear | Masked $3 \times 2$-point data vector | Probe block and angular bins | Dense covariance geometry. Optional IA templates. |
| Scalar | Named derived parameters | Output-name list | Per-output center and scale. |
| CMB | One spectrum $C_\ell$ | Multipole $\ell$ | Per-$\ell$ covariance scale and amplitude law. |
| Background | $H(z)$ or $D_M(z)$ | Redshift $z$ | Target-law inverse. Distances derived from $H(z)$. |
| Matter power | $P_{\rm lin}(z,k)$ or nonlinear boost | Stored $(z,k)$ grid | Target-law inverse and analytic-base multiplication. |

TABLE XIX. The five supported output families. One raw target row has a family-specific physical axis, but all families share the same notions of cosmology row, minibatch, training lifecycle, validation statistic and saved predictor.

### A. What one target row means

One generated row is the set of two-point correlation values produced by one cosmology and nuisance-parameter point. The entries form one ordered physical vector. Its coordinate order runs through correlation type, tomographic-bin pair and angular separation. A likelihood mask keeps the entries used in the analysis and a covariance assigns coupled uncertainty to the kept vector.

Let $\mathbf{d}_{\rm full} \in \mathbb{R}^D$ be the generated row and let $m \in \{0,1\}^D$ be the analysis mask. Explicit indices select the kept vector:

$$I = \{j : m_j = 1\}, \qquad \mathbf{d}_{\rm keep} = \mathbf{d}_{\rm full}[I].$$

Physical values can be zero, so value equality cannot identify masked positions. The sequence $I$ is part of the geometry state.

### B. Covariance geometry and the physical score

The likelihood covariance $\Sigma$ is first restricted to the kept coordinates, $\Sigma_{\rm keep} = \Sigma[I,I]$. A geometry basis transforms the target into well-scaled network coordinates. Whether the implementation uses an eigenbasis, block structure or another stored factor, the acceptance identity remains

$$\|\widehat{t} - t\|^2 = (\widehat{\mathbf{d}}_{\rm keep} - \mathbf{d}_{\rm keep})^{\mathsf{T}} \Sigma_{\rm keep}^{-1} (\widehat{\mathbf{d}}_{\rm keep} - \mathbf{d}_{\rm keep})$$

for the plain whitened form.

The data-vector center changes the zero point the network learns. A physical residual subtracts prediction and truth, so their shared center cancels. This cancellation holds only when both use the identical persisted geometry.

### C. Likelihood order and angular order

The likelihood's flat order suits covariance contraction. A structured head may require a different order. The convolutional and transformer heads use a second, fixed coordinate map that groups values by physical bin and angular position. This is a fixed permutation/scatter operation. Learned projection weights have no role in it.

The geometry stores both directions:

kept likelihood order
$\rightleftharpoons$ (scatter / gather)
bin-angle rectangle with validity mask.

The correction returns through the inverse map before it is added to the trunk result and scored. Persisting only bin lengths is insufficient when kept coordinates contain holes. The exact coordinate slots are the identity.

| Family | Example meaning of one column | Why column order matters | Decoder's last scientific job |
|---|---|---|---|
| Cosmic shear | One probe/bin/angular-bin entry | The covariance and likelihood mask refer to those exact positions | Scatter kept entries and restore the dense-covariance coordinates. |
| Scalar | One named quantity such as a derived parameter | Equal width does not prove equal output names | Restore each named center and scale. |
| CMB | One $C_\ell$ value for one spectrum and multipole | Adjacent columns represent distinct multipoles. Holes are not zeros | Undo the target transform and persisted amplitude law. |
| Background | One $H(z_j)$ or $D_M(z_j)$ value | A value belongs to a declared redshift and trained window | Undo the target law. Derive distances only on a trained, valid domain. |
| Matter power | One point on a flattened $(z_i, k_j)$ surface | Reshaping with the wrong flattening order transposes scientific coordinates | Undo the law and apply the analytic base at the same $(z, k)$ point. |

TABLE XX. The family-specific part of a target column. Every column requires a coordinate identity in addition to a numerical value.

### D. Factored intrinsic-alignment dependence

Intrinsic alignments are correlations in galaxy shapes that do not arise from gravitational lensing. When their amplitude dependence is a known polynomial, the model predicts templates and applies that polynomial analytically.

For the nonlinear alignment model, abbreviated NLA, with amplitude $A_1$, three network templates give

$$\xi(\boldsymbol{\theta}, A_1) = K_0(\boldsymbol{\theta}) + A_1 K_1(\boldsymbol{\theta}) + A_1^2 K_2(\boldsymbol{\theta}).$$

Here $K_0$ is the gravitational-lensing by gravitational-lensing contribution (GG), $K_1$ carries the gravitational-lensing by intrinsic-shape cross term (GI) and $K_2$ carries the intrinsic by intrinsic term (II).

For the more detailed tidal-alignment and tidal-torquing model, abbreviated TATT, the model emits ten templates. Their persisted coefficient order is

$$(1,\ a_1,\ a_2,\ a_1 b_{\mathrm{TA}},\ a_1^2,\ a_2^2,\\ (a_1 b_{\mathrm{TA}})^2,\ a_1 a_2,\ a_1^2 b_{\mathrm{TA}},\ a_1 a_2 b_{\mathrm{TA}}).$$

The first entry multiplies the gravitational-lensing template. The next three are linear intrinsic-alignment cross terms. The final six are quadratic pairings of the three intrinsic-alignment field contributions. This order is part of the scientific contract: the same ten numbers in another order define a different prediction. The amplitude parameters are appended to the encoded input record for the loss and pass directly to the analytic combine step. That step is exact and differentiable with respect to the emitted templates.

At inference, the same template order and polynomial must recombine the prediction. Reordering templates can preserve tensor shape while changing the physical function, so template identity belongs in artifact state and round-trip gates.

*a. Safe use of a saved TATT artifact.* The artifact stores the design label `tatt` but omits the ten ordered template labels and monomials above. Rebuild uses the current registry entry, so equal-shaped templates can be interpreted in a different order without a tensor-shape failure. Keep the training checkout with the artifact, record its source commit and compare the registry's ordered coefficient list before serving a rebuilt TATT prediction. A label match by itself does not establish implementation identity.

### E. Training and diagnostic readouts

Validation reports one $\Delta\chi^2$ per held-out cosmology. The headline failure fraction is the fraction above a declared threshold such as 0.2. A diagnostic report also examines the distribution across parameter space, local behavior and the physical coverage triangle.

The mean-predictor baseline is important. A smoke run must beat the score obtained by always returning the training center. Otherwise the network may be disconnected while every file and plot still exists. Separate structured-head liveness tests prove that head parameters receive gradients and change, because the trunk alone can hide a dead correction in aggregate metrics.

## XX. SCALAR EMULATORS: NAMED DERIVED QUANTITIES

For scalar emulation, both input and target columns come from one parameter table. Declared names locate the outputs. The prediction API returns a dictionary from output name to physical value.

### A. Why a scalar emulator is a separate family

A scalar artifact maps sampled parameters to a small set of named derived quantities such as $H_0$, $\Omega_m$ or $r_{\rm drag}$. It does not have a CosmoLike data vector, likelihood mask or angular coordinate. Treating it as a width-one cosmic-shear vector would import irrelevant covariance and layout assumptions.

The generated parameter table may contain bookkeeping columns, sampled parameters, derived outputs and diagnostic quantities. The loader resolves sampled and output columns from names. It then proves that every requested name appears exactly once and that input/output sets obey the declared relationship. A fixed slice is unsafe because adding one derived column changes every later position while the table remains rectangular.

### B. Target geometry

For output $q$, the scalar geometry stores a center $\mu_q$ and scale $s_q$:

$$t_q = \frac{q - \mu_q}{s_q}, \qquad q = \mu_q + s_q t_q.$$

Each scale is tested after conversion to the storage/model dtype. The check reads both the original float64 scale and the stored scale. A positive float64 value smaller than the least representable positive float32 value becomes zero when stored. Checking only before the cast would leave a division by zero in encode.

Correlated scalar outputs may use a covariance-aware geometry when the declared metric requires it. The same round-trip and physical-score rules apply, including minimum sample count and positive-definiteness. A one-row table cannot define a sample covariance and is refused with an explanation.

### C. Training and prediction

The network output has shape $(\mathtt{B}, \mathtt{Q})$ for $Q$ named quantities. A single-output run still retains its feature axis. Its shape is $(\mathtt{B}, 1)$. A $(\mathtt{B},)$ result would erase that axis and create batch-size-one squeeze ambiguity.

Prediction begins with a parameter dictionary. The predictor orders values by the artifact's saved input names, encodes, evaluates, decodes and returns a dictionary keyed by saved output names. Cobaya's scalar adapter uses those names to advertise what it can provide and inserts the calculated values into the current state.

An analytic smoke such as

$$\omega_m = \Omega_m \left(\frac{H_0}{100}\right)^2$$

is particularly strong. It supplies truth at off-center cosmologies without calling the production helper. The off-center requirement prevents the degenerate implementation that always returns $\mu_q$ from passing.

### D. What the scalar artifact persists

The artifact stores the shared model recipe together with ordered input names, ordered output names, centers/scales or covariance factors, dtype and family kind. The adapter refuses a non-scalar artifact even if its output width happens to equal $Q$. Family kind is an explicit semantic fact. Shape alone cannot infer it.

## XXI. CMB SPECTRA: MULTIPOLES, AMPLITUDE LAWS AND COVARIANCE

One artifact predicts one spectrum type. The geometry stores the exact multipole sequence, units, fiducial spectrum and diagonal uncertainty. The adapter may assemble several independently trained spectra, but it must prove their requested multipole layouts. Equal widths alone provide no axis identity.

### A. Four related but distinct spectra

An angular power coefficient $C_\ell^{XY}$ measures covariance between sky fields $X$ and $Y$ on angular scale indexed by the nonnegative integer multipole $\ell$: larger $\ell$ corresponds roughly to finer angular structure. The symbols $L$ and $\ell$ both denote such multipoles here. $L$ is conventionally used for the lensing spectrum. The generator writes four distinct blocks: $TT$ is the temperature auto-spectrum, $TE$ is the temperature to E-mode polarization cross-spectrum, $EE$ is the E-mode polarization auto-spectrum and $\phi\phi$ is the auto-spectrum of the CMB lensing-potential field $\phi$. Each array column belongs to one explicit multipole. The artifact stores that coordinate sequence and a convention describing whether the values are raw $C_\ell$ or a scaled representation.

Magnitude alone leaves the convention underdetermined. For lensing,

$$\frac{[L(L+1)]^2}{2\pi} C_L^{\phi\phi}$$

can differ from raw $C_L^{\phi\phi}$ by many orders of magnitude. A producer and consumer that both make the same wrong assumption can round-trip perfectly. Identity evidence therefore compares the production contraction with an independent known-answer calculation under fixtures where the two representations are genuinely distinct.

### B. From-fiducial diagonal geometry

For spectrum $s$ at multipole $\ell$, the current diagonal geometry is

$$t_{s\ell} = \frac{C'_{s\ell} - \overline{C}'_{s\ell}}{\sigma_{s\ell}}, \qquad \sigma_{s\ell} = C^{\mathrm{fid}}_{s\ell}\sqrt{\frac{2}{2\ell+1}}.$$

Here $C'$ is the training target after any amplitude law and $\overline{C}'$ is its training-row mean. The fiducial spectrum supplies the cosmic-variance scale. The training-row mean supplies the center. Mean and fiducial are both saved because they have different owners. A zero or non-representable $\sigma_{s\ell}$ is rejected or handled only through a physically declared zero-weight rule. A valid mask requires that explicit rule. The present constructor requires a strictly positive fiducial entry, which is an important explicit convention for a TE spectrum that can cross zero.

The diagonal loss becomes a sum of squared encoded residuals. Non-Gaussian covariance introduces band coupling and weights derived from raw spectra. A small banded fixture compares each accumulated contribution with explicit per-$L$ loops. Zero-band input provides a physical exact-zero control, while a width-three nonconstant band exercises the projection stencil.

### C. Amplitude and optical-depth law

CMB spectra contain a strong approximate dependence on primordial amplitude and optical depth. $A_s$ is the dimensionless amplitude of the primordial scalar-curvature power spectrum at its declared pivot scale. $\tau$ is the dimensionless Thomson-scattering optical depth from reionization. The family can factor a known law from the training target so the network learns a gentler residual. The executed `as_exp2tau_ref` law uses named finite reference values and forms

$$g(\boldsymbol{\theta}) = \frac{A_{s,\mathrm{ref}}}{A_s}\exp[2(\tau - \tau_{\mathrm{ref}})]\,,$$

so the factor is dimensionless and equals one at the reference cosmology. Training and decoding use inverse operations:

$$t = \mathcal{G}(gC)\,, \qquad \widehat{C} = \frac{\mathcal{G}^{-1}(\widehat{t})}{g},$$

where $\mathcal{G}$ is centering and diagonal whitening. The artifact persists the amplitude parameterization, parameter names and both reference values. The retired `as_exp2tau` law used $\exp(2\tau)/A_s$, a dimensionful factor of order $10^8$. Loading an artifact that names it raises with a retraining instruction because metadata cannot repair weights fitted in the old target space.

The physical metric must compare $\widehat{C}$ with $C$. Since encoded prediction and truth both carry $g$, their encoded difference also carries $g$. A common multiplicative factor remains in their difference. The loss therefore divides the residual by $g$ before summing squares. Omitting that step would report $g^2\Delta\chi^2$ and make model selection depend on $A_s$ and $\tau$ even at fixed physical error.

### D. Roughness penalty

An optional roughness term discourages high-frequency error across neighboring multipoles. It acts on the factor-corrected residual in the declared spectrum coordinates. A representation whose amplitude law changes its strength is outside this calculation. Let $r$ be a whitened residual row. The configured `period_cut` is rounded to an integer width $w \geq 5$ and increased by one when needed so $w$ is odd. One pass reflect-pads the ends by $(w-1)/2$ values and applies a normalized width-$w$ boxcar. Applying the same pass twice is a triangular smoother:

$$\begin{aligned} s(r) &= \mathrm{boxcar}_w[\mathrm{boxcar}_w(r)], \\ r_{\mathrm{short}} &= r - s(r), \\ c_{\mathrm{rough}} &= \sum_\ell r^2_{\mathrm{short},\ell}. \end{aligned}$$

Reflect padding mirrors nearby residual values at the spectrum boundary and introduces no zero jump. A constant residual is unchanged by both boxcars and therefore has zero roughness. An alternating residual is mostly removed by smoothing, so its remainder and penalty are large. The training score adds $\lambda_{\mathrm{rough}} c_{\mathrm{rough}}$ per sample. Turning the coefficient off must leave the original objective exactly unchanged.

Roughness is a training regularizer. The headline physical $\Delta\chi^2$ is a separate quantity, which validation reports after full reconstruction. This separation lets a sweep compare regularization while preserving one scientific score definition.

### E. Serving spectra to Cobaya

Cobaya first calls `must_provide` with requested spectra and multipole ranges. The adapter validates the complete sequence against each artifact. Later, `calculate` gathers current cosmological parameters, evaluates each predictor and inserts the physical spectra into state under the expected keys.

Several artifacts can contribute different spectrum blocks. Assembly proves that their coordinate layouts are compatible and destination blocks do not overlap. Blind concatenation would accept equal widths with shifted multipoles. Strict weight loading and artifact-axis validation occur before the first sampled calculation.

### F. Diagnostics and evidence

Family diagnostics plot residuals and error summaries as functions of multipole and spectrum type. They handle TE zero crossings without dividing unmasked truth by zero. When a fractional statistic is undefined at a crossing, the report uses an explicit availability status or a declared mask to prevent publication of NaN as an ordinary number.

The identity gate owns exact law, covariance, roughness, structured-head and save/rebuild claims. The smoke runs real CAMB, the covariance script, training, Cobaya lifecycle and diagnostics. Their separation prevents the real generator from becoming its own mathematical reference.

## XXII. BACKGROUND FUNCTIONS: TRAINED GRIDS AND DERIVED DISTANCES

The background network predicts a function on a stored redshift grid. For an $H(z)$ artifact, the shared physics module computes

$$\chi(z) = \int_0^z \frac{c}{H(z')}\, dz', \tag{38}$$

$$D_A(z) = \frac{\chi(z)}{1+z}, \qquad D_L(z) = \chi(z)(1+z). \tag{39}$$

for a flat model. Deterministic physics produces these derived quantities. The low-redshift artifact supplies $H(z)$. In a spatially flat model, transverse comoving distance equals radial comoving distance: $D_M(z) = \chi(z)$. The production background pair also includes a separate network that directly emulates recombination-era $D_M$ on its high-redshift window. Direct emulation on that window supplies the quantity. Low-redshift $H(z)$ extrapolation through the untrained desert is invalid. Queries are valid only inside the trained domain recorded by the relevant artifact.

When $H$ is stored in $\mathrm{km\,s^{-1}\,Mpc^{-1}}$ and the speed of light $c$ is in $\mathrm{km\,s^{-1}}$, the ratio $c/H$ has units of Mpc. Redshift is dimensionless, so the integral returns a comoving distance in Mpc. If the adapter exposes $H$ in inverse Mpc, it first applies the named unit conversion. Mixing these conventions can return a finite result wrong by a factor of $c$, so the quantity and unit pair is an artifact fact.

### A. Quantities and grid identity

A background artifact represents one named quantity such as $H(z)$ or transverse comoving distance $D_M(z)$ on a one-dimensional redshift grid. The configuration declares both quantity and units. The validator requires an allowed quantity and unit pair, and the adapter requires the exact pair it knows how to serve.

The grid is an ordered numerical sequence

$$0 \le z_0 < z_1 < \cdots < z_{N_z-1}.$$

It is saved with the geometry. Input arrays must have last-axis width $N_z$, but width is only the first check. The generated axis sidecar, geometry state, validation grid and requested adapter coordinates must agree in value and order.

### B. Target laws

A direct target law may center and scale the physical quantity. A `log_offset` law uses

$$u(z) = \log[y(z) + a]\,, \qquad t(z) = \frac{u(z) - \mu_u(z)}{s_u(z)}$$

where $u$ is the law-space value and $t$ is the standardized network target. The diagonal geometry fits one center $\mu_u(z)$ and scale $s_u(z)$ per stored redshift coordinate. Decoding reverses both stages:

$$y(z) = \exp[\mu_u(z) + s_u(z)t(z)] - a.$$

The offset $a$ is a finite numeric artifact fact. It must keep every training and validation value inside the logarithm's positive domain. A NaN offset can poison center and scale while bypassing ordinary ordered comparisons, so validation rejects nonfinite values before target construction.

The target law is owned by the geometry and reused at inference. The adapter reads that geometry-owned transform, keeping save/rebuild and direct prediction on the same formula.

*a. Flat-only use of the current background artifact.* The producer requests radial $\chi$ and labels it $D_M$, while the adapter's curvature check inspects only parameter names stored in the two artifacts. A fixed or separately consumed nonzero global Cobaya curvature can bypass that check. In curved geometry, $D_M$ is a sine or sinh mapping of $\chi$. Equality with $\chi$ applies only in flat geometry, so a returned value can be finite and scientifically mislabeled. Use this V1 background path only when the complete Cobaya model fixes $\Omega_k$ to exact zero. Use a curvature-aware provider for every nonflat model.

For positive curvature parameter $\Omega_k > 0$, the open-geometry mapping is

$$D_M = \frac{D_H}{\sqrt{\Omega_k}} \sinh\!\left(\sqrt{\Omega_k}\,\frac{\chi}{D_H}\right), \qquad D_H = \frac{c}{H_0}.$$

An independent reproduction at $H_0 = 70\,\mathrm{km\,s^{-1}\,Mpc^{-1}}$, $\Omega_k = 0.1$ and $z = 1100$ gives $\chi = 13296.826\,\mathrm{Mpc}$ but $D_M = 15538.408\,\mathrm{Mpc}$, a 16.858% difference. Both numbers are finite, positive and smooth. A check that asks only whether the returned distance looks numerical cannot detect the wrong physical label.

### C. Piecewise trained windows and the desert

Some background products are trained in separated redshift windows. The union of those windows is the trained domain. The gap between them is a *desert*: no training examples support interpolation there.

A numerical interpolation routine can return a smooth, finite value across the desert, but the value remains outside trained support. The public range checker first classifies the requested redshift by the persisted window set and raises for the gap. Boundary values have one explicit inclusion rule shared by startup validation and runtime getters.

### D. Distances derived from $H(z)$

When $H(z)$ is the emulated primitive, comoving distance is derived by integration. For a flat model,

$$\chi(z) = c \int_0^z \frac{dz'}{H(z')}.$$

Angular-diameter and luminosity distances follow

$$D_A(z) = \frac{\chi(z)}{1+z}, \qquad D_L(z) = (1+z)\chi(z).$$

The numerical quadrature uses the trained domain. Emulator support requires the stored grid to cover every integration interval. The load-time domain contract therefore checks every interval needed by the public distance getters, including the anchor at zero when the formula needs it.

*a. Safe use of the current positive-minimum grid.* The public generator requires its first redshift node to be positive. The adapter can then extrapolate both $H$ and the distance integrand from that node down to zero. Smooth, finite and monotonic output does not show that this interval was absent from training. Use the artifact for $H(z)$ only at or above its first stored node. Obtain $\chi$, $D_A$ and $D_L$ from a non-emulated provider because every one of those integrals needs the missing interval from zero to the first node.

An odd number of intervals requires the quadrature's explicit endpoint rule. The identity fixture uses low-degree polynomials whose integrals are known exactly, so a coefficient error cannot hide behind a loose percent tolerance.

### E. Serving and diagnostics

The Cobaya adapter advertises the background quantities it can provide, validates requested redshifts, evaluates the primitive predictor and calls the shared physics functions for derived quantities. One sampled cosmology owns one state update. A rejected provider result cannot expose arrays from the previous cosmology.

Diagnostics compare the primitive and derived functions with truth across redshift. They report absolute or well-defined relative errors, name unavailable quantities explicitly and mark trained-window boundaries. The identity gate uses closed-form flat-ΛCDM controls. The smoke compares the full generated, trained and served path with CAMB's own background.

## XXIII. MATTER POWER: TWO-DIMENSIONAL SURFACES AND ANALYTIC BASES

The matter power spectrum $P(k, z)$ quantifies the variance carried by Fourier modes of the dimensionless matter-density contrast at comoving wavenumber $k$ and redshift $z$. Wavenumber has inverse-length units: a larger $k$ describes a smaller spatial scale. When $k$ is measured in $\mathrm{Mpc}^{-1}$, $P$ has volume units $\mathrm{Mpc}^3$ so the Fourier-space variance is dimensionless. Linear power follows perturbations that have not strongly mode-coupled. Nonlinear power includes later gravitational coupling.

The matter-power target has shape $(N, N_z, N_k)$ and is flattened with redshift as the outer coordinate. A target law may divide by a known analytic base and take a logarithm. Staging performs this transformation in bounded row chunks and may thin the $k$ axis before materializing the final width.

At inference, the predictor reshapes the network row to $(N_z, N_k)$. The consumer multiplies the persisted law's analytic base back. Constant low-$k$ boost points may be pinned only when the quantity, coordinates and law make that constant physically valid.

### A. Two quantities with different meanings

The family can emulate linear power $P_{\mathrm{lin}}(k, z)$ or a nonlinear boost

$$B(k, z) = \frac{P_{\mathrm{nl}}(k, z)}{P_{\mathrm{lin}}(k, z)}.$$

Both are rectangular surfaces with different physical invariants. The boost $B$ is dimensionless. Linear power carries volume units fixed by the declared wavenumber convention: if $k$ is in $\mathrm{Mpc}^{-1}$, then $P$ is in $\mathrm{Mpc}^3$. An $h$-scaled convention changes both axes together. The artifact and request boundary therefore name units explicitly. Numerical magnitude has no authority to select a unit convention. For example, a boost may approach one in a declared low-$k$ region, while linear power follows a different limit. A constant-column pin is therefore permitted only when the artifact quantity, law, coordinates and observed center establish that identity.

The family kind and quantity are explicit state. A consumer cannot infer them from file name or array scale. Two artifacts assembled into a hybrid inference must

prove compatible parameter names and exactly matching $(z, k)$ grids.

### B. Flattening convention

The generated payload has shape $(\mathtt{N}, \mathtt{N_z}, \mathtt{N_k})$. Training flattens each surface to $(\mathtt{N}, \mathtt{N_z N_k})$ under one documented order: redshift is the outer coordinate and wavenumber is the inner coordinate. In index form,

$$j = i_z N_k + i_k.$$

The inverse reshape uses the same $N_z$, $N_k$ and order. A transposition preserves the total width and can preserve global statistics, so the artifact stores actual axis arrays and the identity gate uses a surface whose value depends differently on $z$ and $k$.

### C. Target laws and analytic-base placement

The no-law path learns the physical surface after ordinary centering and scaling. A base-law path may factor a fast analytic approximation:

$$u(k, z) = \log\left[\frac{P_{\mathrm{truth}}(k, z)}{P_{\mathrm{base}}(k, z)}\right],$$
$$t(k, z) = \frac{u(k, z) - \mu_u(k, z)}{s_u(k, z)}.$$

Inference reconstructs

$$\widehat{P}(k, z) = P_{\mathrm{base}}(k, z) \exp\left[\mu_u(k, z) + s_u(k, z)\widehat{t}(k, z)\right].$$

The base is evaluated at the same cosmology and coordinates as the target. Its implementation identity, input names, domain and version-independent facts are part of the artifact contract. A base from a different implementation can have the same file shape and a different scientific function.

For interpolation within an analytic base, both redshift and wavenumber grids are finite, strictly increasing and large enough for the declared spline order. $k > 0$ is required before $\log k$. Tail extrapolation parameters are explicit, finite and ordered. The code retains the vendored interpolation algorithm on valid inputs. Validation makes its previously implicit preconditions public.

### D. Bounded staging

Production surfaces are wide. Materializing every selected raw row in float64 before thinning can exceed host memory even when the final float32 training matrix would fit. Bounded staging therefore operates in chunks:

1. read a bounded set of selected disk rows,
2. cast to the payload dtype used by training,
3. apply the base and logarithmic law,
4. thin the $k$ axis according to the declared stride,
5. update stable streaming moments.
6. write the final compact rows to resident memory or an experiment-owned temporary memmap.

Streaming variance uses a Chan/Welford merge of chunk counts, means and second central moments. A sum/sum-of-squares formula suffers catastrophic cancellation when values have a large offset and small spread. The reference is computed from the stored float32 payload because pre-cast float64 values are not the data the geometry will encode.

Host-memory accounting includes both staged parameters and targets. A decision based only on target bytes can select a resident path that exceeds the promised RAM fraction after the parameter copy is added.

### E. Temporary-file ownership

A disk-backed staged target is a temporary resource owned by the experiment. Train and validation slots are independent because validation may remain live while successive training-size sweep points restage training rows. Restaging one slot unlinks the superseded file only after no live loader needs it. Explicit release and failure cleanup remove remaining files. Process-exit cleanup is only a last defense. A long sweep cannot retain one multi-gigabyte file per completed point.

### F. Hybrid inference

EMUL2 is the repository's name for the hybrid inference configuration that replaces several expensive provider products with coordinated emulator artifacts. In this EMUL2-shaped path, matter power, background history and scalar quantities can jointly replace expensive CAMB products inside a larger likelihood. The matter-power adapter validates requirements, rebuilds the linear and boost artifacts, evaluates the current cosmology, reconstructs each physical surface and assembles $P_{\mathrm{nl}} = P_{\mathrm{lin}} B$ when requested.

The derived quantity $\sigma_R$ is the root-mean-square linear density contrast after smoothing with a spherical top-hat of comoving radius $R$:

$$\sigma_R^2(z) = \frac{1}{2\pi^2}\int_0^\infty dk\, k^2 P_{\mathrm{lin}}(k, z) W^2(kR),$$
$$W(x) = 3\,\frac{\sin x - x\cos x}{x^3}.$$

The conventional $\sigma_8$ uses $R = 8\,h^{-1}\mathrm{Mpc}$, where $h = H_0/(100\,\mathrm{km\,s^{-1}\,Mpc^{-1}})$. If the stored wavenumber is in $\mathrm{Mpc}^{-1}$, the numerical radius entering $kR$ is therefore $8/h$ Mpc. A radius of 8 Mpc computes a different observable.

*a. Safe use of the current matter-power adapter.* The derived $\sigma_8$ helper receives $H_0$, forms $h = H_0/100$, and combines the stored $k$ in $\mathrm{Mpc}^{-1}$ with the conventional radius $R = 8/h$ Mpc. It requires one exact stored $z = 0$ row. Before taking the square root, it integrates in double precision and checks the positive contribution per unit $\log k$. Adjacent bands must show decay toward both missing tails, the estimated omitted variance must be small, interlaced lower-resolution integrals must agree, and no single integration panel may dominate the result. A short or under-resolved grid therefore refuses rather than returning a smooth but incomplete number.

Cobaya discovers matter-power grids and interpolators from their public getter methods. It treats $\sigma_8$ as a derived result. Requesting that result adds the $H_0$ dependency needed for the radius conversion; a saved emulator or a Syren formula may independently use $H_0$ as one of its own inputs.

The request boundary validates the full $k$ and $z$ sequences. It requires a complete requested grid and rejects unsupported holes. `must_provide` checks every grid and interpolator requirement while setup can still stop: only the total-matter density pair is accepted; the nonlinear choice must be a boolean; requested redshifts must lie inside the stored grid, because redshift is never extrapolated; and a requested maximum wavenumber must be servable — inside the stored grid for the raw grid product, beyond it for the interpolator only when the power-law extrapolation tails are enabled. Each refusal names the observed request, the stored bound and the corrective action, so an unservable likelihood fails during component assembly instead of inside a running chain. Every calculate call checks the external provider verdict before reading payload getters, so a rejected point cannot inherit a previous surface.

### G. Diagnostics and evidence

Matter-power diagnostics plot residual surfaces and slices on the persisted axes. They avoid constructing a validation-rows by neighbors by full-width cube at production scale. Bounded diagnostics sample or stream the required statistics.

The identity gate proves flatten/reshape, target laws, stable moments, constant-column permission, base assembly, structured-head behavior and temporary-file lifecycle on controlled fixtures. The smoke runs real CAMB, two trainings, public $P(k, z)$ getters and family pages under the no-law path. Together they establish exact internal algebra and real-provider connectivity without making the expensive provider its own reference.

## XXIV. DIAGNOSTICS AND PLOTTING: COMPUTE A FACT BEFORE DRAWING IT

A *diagnostic* calculates a statement about predictions. A *plot* turns already calculated values into marks, lines, colors and labels. The distinction is architectural: `emulator/diagnostics.py` owns estimators and their validity conditions, while `emulator/plotting.py` owns visual layout. A plotting function must not silently invent a replacement statistic when the diagnostic is undefined.

### A. The diagnostic record is a typed result

For validation rows $i = 1, \ldots, N_{\mathrm{val}}$, the shared evaluation path first obtains physical scores $q_i = \Delta\chi_i^2$. A coverage diagnostic may report fractions such as

$$f_{<t} = \frac{1}{N_{\mathrm{val}}} \sum_{i=1}^{N_{\mathrm{val}}} \mathbf{1}[q_i < t],$$

where $\mathbf{1}[\cdot]$ is one when its condition is true and zero otherwise. Here $N_{\mathrm{val}}$ is the number of held-out validation cosmologies. Training set and minibatch sizes use separate counts. The threshold $t$, sample count $N_{\mathrm{val}}$ and side of the inequality are part of the result. A bare number such as `0.93` cannot reveal them.

A diagnostic dictionary therefore carries values together with names, units, coordinate arrays, valid-row masks and a status. Status `available` means the estimator's mathematical preconditions held. Status `unavailable` carries a reason such as "no rows in this class" or "reference variance is zero." NaN is a floating-point value. A complete status record also carries availability and reason, so it can distinguish a deliberate undefined result from numerical corruption.

### B. Local-linear and hard-direction questions

The local-linear diagnostic asks whether a small neighborhood can already explain the validation target. It selects nearby training rows, fits a local linear approximation and evaluates that approximation with the same physical loss used for the emulator. The neighborhood count, distance definition, rank condition and memory bound are estimator inputs. Independent training data and the local fit supply the floor against which the neural model is compared.

The hard-direction diagnostic relates poor scores to movement along named parameter combinations. Correlation and $R^2$ require variation. If a feature is constant or a target class is empty, the estimator is unavailable on that sample. The result names that condition so the figure can annotate it without plotting a fabricated zero.

### C. Family diagnostics retain their physical axes

Scalar residuals are indexed by output name. CMB residuals are indexed by spectrum and multipole. Background residuals are indexed by redshift and may include distances derived through the real integration path. Matter-power residuals are two-dimensional surfaces over $(z, k)$. Cosmic-shear residuals retain correlation type, tomographic pair and angle. Reducing these objects to one aggregate score is useful for model selection but insufficient for finding a localized physical failure.

Fractional error, $(\widehat{y} - y)/y$, is undefined where $y = 0$. The CMB TE spectrum can cross zero, so a diagnostic uses an explicitly named mask or an absolute-error alternative at the crossing. A finite neural prediction does not make division by a zero truth valid.

### D. The plotting layer is a consumer

The page builder receives NumPy arrays on the CPU. Model evaluation occurs under `torch.no_grad()` so the report does not retain a backward graph. Tensor values are detached, moved to the CPU and converted before Matplotlib sees them. This is an eager data transfer. The figure does not own a live GPU tensor or a lazy loader.

Axis scale is chosen from the data domain. A logarithmic ordinate requires strictly positive finite values. A series containing zero remains linear or uses a separately defined signed transformation. Labels state estimator, units, mask and threshold. A multipage writer saves completed figure objects and closes them so long campaigns do not accumulate GUI and array state.

*a. Safe interpretation of current diagnostics.* Several estimators can publish NaN for an empty class, a zero-variance reference, a constant feature, an unguarded standard deviation or a fractional residual whose truth is zero. Some smoke fixtures exercise the plot layout with hand-built finite dictionaries and do not call these estimators. Treat every nonfinite diagnostic value as an unavailable or failed calculation, never as evidence that a model passed. Save-before-diagnostics preserves the trained artifact when reporting fails. For a production-width matter-power output, omit the generic local-neighbour diagnostic and use an explicitly bounded or subsampled analysis so the validation-rows by neighbours by output-width array does not exhaust host memory.

## XXV. SAVING, REBUILDING AND PREDICTING

Training ends with a live Python object on one machine. Inference needs a self-contained record that can rebuild the same computation later.

### A. The two-file artifact

The current format uses

A `state_dict` contains numerical state. The Hierarchical Data Format version 5 (HDF5) model recipe supplies the class name, dimensions and constructor options that give those tensors meaning.

The two files record different facts. The weight file stores the learned numbers. The HDF5 file defines the calculation that gives those numbers meaning. A matching tensor width is insufficient: the same-width array could represent different multipoles, redshifts, output names, units or target laws.

One ordinary artifact has the following logical tree. A slash denotes an HDF5 group, which is a directory-like container. A leaf such as `center` is a dataset, an array stored inside the file. A name after `@` is an attribute, a small scalar fact attached to a group or to the file itself.

```
root.emul
  pair_token -> shared random string
  state_dict
    tensor_name -> CPU tensor
root.h5
  @schema_version
  @pair_token
  @composition_mode
  @transfer_refined
  @rescale
  @git_commit
  model_recipe
    YAML constructor description
  config_yaml, config_resolved_yaml
    requested and consumed values
  param_geometry/
    @cls
    names, center, evecs, sqrt_ev
  dv_geometry/
    @cls
    center, coordinates
    covariance factors
  fixed_facts/, input_domain/
    the scientific record: the
    cosmology held fixed and the
    sampled parameter intervals
  facts_sidecar_yaml
    the generator's own record
    text, stored verbatim
  history/
    train_losses, val_medians
    val_means, val_fracs
```

Here, `tensor_name` stands for each registered parameter or buffer name, and `@cls` names the Python class whose `from_state` constructor interprets that group. The slash syntax above is explanatory. HDF5 stores these names as groups and datasets inside one file. The `fixed_facts/` and `input_domain/` groups are the artifact's memory of the science it was born under: the cosmology held fixed

| File | Contents |
|---|---|
| `root.emul` | The model `state_dict`: parameter and registered-buffer names mapped to CPU tensors. |
| `root.h5` | Input/output geometry state, model recipe, resolved configuration, histories, physical metadata and optional NPCE/transfer base. |

TABLE XXI. The two files that form one saved emulator. The `.emul` file owns learned PyTorch tensors. The HDF5 `.h5` file owns the constructor recipe and scientific coordinate facts needed to interpret them. Both are first written under temporary names and renamed into place only when complete, and both carry one shared random pair token, so a pair mixed from two different saves is refused when reopened.

while the dataset was generated, and the interval each sampled parameter was drawn from. The generator's own record text is stored verbatim beside them, and the reader checks the parsed groups against that text in both directions, so an edited block or a swapped text is refused rather than believed. The recursive writer walks a nested Python dictionary. A nested dictionary becomes a group, a numerical array becomes a dataset and selected scalar facts become attributes. The recursive reader performs the inverse walk and moves reconstructed tensors to the requested device.

Optional model compositions add owners while preserving the meaning of the ordinary fields:

*a. Safe handling of the two-file artifact.* Saving writes both members under temporary names and renames them into place only when both are complete, so an ordinary failure mid-save leaves no partial file under the public names, and a save refuses any root whose `.emul` or `.h5` already exists. Every save also draws one fresh random pair token and stores it in both members; reopening compares the two copies, so a `.emul` placed beside another run's `.h5` is refused instead of pairing weights with the wrong scientific record. Still keep both members under one root and copy or move them as a pair. After transport, rebuild the artifact and compare a known prediction with its source: the pair token proves the files were saved together, not that the bytes survived the copy.

### B. Moving tensors into storage

For a model tensor $w$, `w.detach()` returns a tensor disconnected from gradient history, but it normally shares the same numerical storage with $w$. Detachment changes what autograd records while preserving storage. Calling `w.cpu()` copies accelerator storage to host memory when necessary, but a tensor already on the CPU may be returned without a copy. Calling `w.numpy()` on a compatible CPU tensor presents that storage as a NumPy array and normally shares memory with it. An in-place write through either object can then change the other. Use an explicit `clone()` or NumPy `copy()` when the receiving caller must own independent mutable storage.

Serialization is a later boundary. Writing a tensor or NumPy array to a PyTorch or HDF5 file copies its current bytes into the file. Reading it later constructs new process-owned storage with no live pointer to the original process. Saving CPU tensors lets a machine without the training GPU open the artifact. In-process returned-array ownership still requires a separate copy-and-mutate check.

HDF5 uses groups, datasets and attributes. A group is a directory-like container. A dataset is an array payload. An attribute is scalar metadata attached to a group or file. Nested geometry dictionaries become nested groups through a recursive writer.

### C. Rebuilding the geometry class

Each geometry group stores a class string such as `emulator.geometries.parameter.ParamGeometry`. Rebuilding performs four steps:

1. split the string into module path and class name,
2. import the module,
3. retrieve the class object.
4. call that class's `from_state` constructor.

Using the persisted class prevents a factored or logarithmic geometry from silently rebuilding as its simpler base class.

### D. Rebuilding the model

The model recipe stores factory choices as names because a Python callable cannot be represented directly in YAML. Rebuilding translates those names back into activation and normalization factories, constructs the eager model and loads the weights with `strict=True`. Strict loading rejects both missing and unexpected keys.

A compiled model wraps the eager module and may prefix state keys with `_orig_mod.`. Saving removes that wrapper prefix. Rebuilding loads the plain state first and compiles afterward when requested and supported.

| Artifact kind | Main `.emul` tensors | Additional HDF5 owner |
|---|---|---|
| Ordinary | The complete predictor network | No optional model group. |
| NPCE | The neural residual or ratio refiner | `pce/` stores the frozen polynomial basis, coefficients, output modes, fitted box and composition form. |
| Transfer | The correction network | `transfer_base/` embeds the frozen base recipe, base tensors, both base geometries and the declared sum/gain composition facts. |
| Refined transfer | The correction network | The original base state remains as provenance. `drifted_state/` stores the jointly updated base. A native-boolean marker and group presence must agree before that state is selected. |

TABLE XXII. Tensor ownership in the four artifact compositions. Rebuilding does not infer an optional component from shape. Its named group and required facts must be present.

### E. Prediction

`EmulatorPredictor.predict` performs the five transitions in Table XXIII. The table separates the source of each fact from the object produced by the transition.

Evaluation mode disables training-only behavior. `torch.no_grad()` avoids constructing a backward graph and reduces memory and latency. The artifact's parameter names, axes, units and laws determine the result. The caller does not redeclare them.

Between step 1 and step 2, every prediction call checks the named point against the sampled intervals stored in the artifact's `input_domain` group, before any tensor is built or the model runs. A point outside any interval is refused, and the refusal names the parameter, its trained interval and the requested value. The refusal exists because a network does not fail outside its training region; it extrapolates and returns a confident number that is wrong. The same stored record feeds the pairing and fixed-value checks a chain runs at startup.

*a. Safe handling of returned arrays.* For a CPU background prediction, the returned NumPy `z` array can share storage with the geometry's cached PyTorch axis. Changing `result["z"]` in place can therefore change the predictor's saved axis and affect the next result. Other devices and derived tensors do not all share storage in the same way. Treat every array and container returned by prediction or diagnostics as read-only. Make an owned copy before an in-place unit conversion, mask, calibration or plotting transformation.

### F. Rebuild ledger: fact ownership by file

Loading an artifact reconstructs several Python objects from validated stored facts. The sequence is:

1. open the HDF5 record and validate its schema.
2. rebuild input and output geometries from their named classes and stored arrays.
3. resolve the model, activation, normalization and loss classes from the stored recipe.
4. instantiate an eager model with the persisted dimensions.
5. read the tensor mapping from `root.emul` onto the CPU.
6. load every expected key with `strict=True`.
7. move the complete predictor to the requested device.
8. compile only after eager reconstruction when compilation is requested and supported.

Each implemented step has a different refusal. An unknown geometry class fails before a network is built. A missing tensor key fails strict loading. A CMB axis that has the right width and the wrong multipoles fails coordinate validation. Each error names the first current boundary that can identify a particular inconsistency.

Current boundaries omit pair identity. Mixing two artifacts with the same expected tensor names and shapes can pass strict loading because the HDF5 file does not bind the exact `.emul` bytes. This remains a scientific-integrity defect even when both files load cleanly. Apply the pair-handling rules above before using a rebuilt prediction.

The reconstructed model is a new owner of its tensors. Moving it to CUDA or MPS copies CPU state to the accelerator. The HDF5 datasets and the NumPy objects used during loading remain separate from the live device parameters. The predictor retains the geometry and model objects it needs. A correctly rebuilt predictor therefore remains valid after the HDF5 file closes.

## XXVI. COBAYA ADAPTERS: PRESENTING THE EMULATOR AS A THEORY COMPONENT

Cobaya asks a theory component for named quantities at a sampled cosmology. The adapters in `cobaya_theory/` translate that lifecycle into an `EmulatorPredictor` call.

| Step | Input | Operation and owner | Output |
|---|---|---|---|
| 1 | Parameter dictionary | Order values by the parameter names stored in the artifact, refusing missing or unexpected names | Raw row $\Theta \in \mathbb{R}^{1\times P}$. |
| 2 | Raw parameter row | Parameter geometry centers, rotates and scales | Encoded row $X \in \mathbb{R}^{1\times P_{\mathrm{enc}}}$ on the model device. |
| 3 | Encoded row | Eager or compiled model in evaluation mode under `no_grad` | Network output $\widehat{T}$ in encoded coordinates. |
| 4 | Encoded network output plus encoded input row | The branch-specific decoder composes any PCE base, amplitude law, intrinsic-alignment templates or transfer correction and invokes the saved output geometry | A decoded kept row. It is physical for scalar, background, CMB and ordinary data-vector families. The grid2d family remains in its declared law space. |
| 5 | Decoded kept row | Family packaging restores a data-vector layout or attaches saved names and axes. The matter-power consumer performs the separately owned analytic-base multiplication | A scalar dictionary, a physical CMB/background/data-vector result or a grid2d law-space surface on the stored $(z, k)$ axes. |

TABLE XXIII. One prediction from a named cosmology. The caller supplies parameter values. The artifact supplies their order and every output-side scientific fact.

### A. Lifecycle methods

The adapter is a boundary between untyped configuration values and numerical code. Boolean, integer, float, path and list options require explicit value validation. Python truthiness and numeric coercion must not reinterpret quoted strings or NaN.

*a. Safe use of current adapter options.* The five adapters do not yet share a total value-schema boundary. Several options pass through `bool(value)`, a relative root can inherit an empty environment default and the CMB maximum multipole is coerced with `int(value)` before validation. Use native booleans, explicit absolute or `ROOTDIR`-resolved unique artifact roots, list or tuple collections and a non-boolean integer maximum multipole. In full-vector cosmic-shear mode, use one predictor unless block compatibility and destination disjointness have been checked independently.

### B. Requested coordinates are part of the contract

An output width omits axis identity. Two multipole arrays can have equal length and different values. Two redshift grids can have equal length and different sampling. `must_provide` validates the complete requested coordinate sequence against the persisted artifact sequence before any calculation can return a value.

*a. Safe use of current coordinate checks.* Current adapters implement only part of this exact-sequence contract. The CMB adapter checks a maximum multipole and can scatter a request with interior holes into a zero-filled array. Its training dump also lacks an authoritative multipole sidecar, so equal width can silently relabel shifted columns. Before serving CMB, independently verify that the dump columns, covariance and artifact all use the same contiguous sequence from $\ell = 2$ through the named maximum. Do not request interior holes or interpret their inserted zeros as physical values. Only the conventional storage positions at $\ell = 0, 1$ are expected zeros.

When several artifacts contribute blocks to one output vector, the adapter must prove that the layouts are compatible and the destination blocks do not overlap. Layout proof requires both checks before concatenation.

### C. State ownership

One `calculate` call corresponds to one sampled cosmology. The acceptance verdict is checked before any provider getter is called. Current state is then replaced only by payloads produced for the accepted current point, which prevents reuse of an earlier cosmology's cache.

*a. Safe use of provider-backed generation and references.* Provider-backed generator and reference paths do not all check the current provider verdict before reading a getter. Some paths discard the returned posterior object or infer rejection from text. A cached payload from a previous cosmology can therefore look successful. Treat a rejected or nonfinite provider verdict as a failed row, call no getters after it and publish no payload. A production run needs a two-call stale-cache check whose arrays identify the cosmology that produced them.

### D. Worked lifecycle: one CMB likelihood request

Assume the configuration loads one temperature-spectrum artifact whose stored axis is $\ell = 2, \ldots, 2500$.

| Method | Responsibility |
|---|---|
| `initialize` | Validate options, choose device, load artifacts and cache immutable metadata. |
| `get_requirements` | Declare sampled or provider-supplied inputs needed by the prediction. |
| `must_provide` | Validate the requested output coordinates and record what the likelihood asks for. |
| `calculate` | Build the parameter mapping, call the predictor, validate the returned payload and place it in the current state. |
| Getter methods | Return named arrays from the accepted current state. |

TABLE XXIV. Cobaya theory-component lifecycle in call order. Initialization constructs immutable machinery. Requirements describe needed inputs. `must_provide` records the requested output. Each `calculate` replaces per-cosmology state. Getters expose only that accepted current state.

Initialization opens the two artifact files, rebuilds the predictor, records that it provides `tt` and takes the union of the predictor's saved parameter names. That union becomes the dictionary returned by `get_requirements`. No sampled cosmology has been evaluated yet.

A likelihood can then declare a request equivalent to

```
Cl:
  tt: 2000
```

Cobaya forwards this declaration to `must_provide`. The method checks that a loaded artifact provides `tt` and enforces a requested maximum of 2500. This is request negotiation: it happens when Cobaya assembles the component graph, before the first Markov-chain point. A request for `ee` or for `tt` through 3000 receives a refusal during this step.

For one accepted sampled point, Cobaya calls `calculate` with a mutable `state` dictionary and named parameter values. The adapter passes those values to the predictor. It allocates a `float64` array covering $0, \ldots, 2500$, writes predicted values at the artifact's multipoles and stores the completed mapping under `state["Cl"]`. The zeros below $\ell = 2$ are conventional storage positions. Neural-network predictions begin at the artifact's declared multipoles.

Finally, the likelihood calls `get_Cl`. The getter reads `self.current_state["Cl"]`, which Cobaya associates with the current sample. It accepts only requested unit and plotting-factor conversions, so the training convention remains fixed. The getter does not call the network again.

The present CMB adapter negotiates by spectrum name and maximum multipole. It does not compare an arbitrary interior coordinate sequence with the artifact. The safe-use paragraph above therefore limits this worked example to a contiguous request on the independently verified stored axis.

## XXVII. GATES: EXECUTABLE EVIDENCE THAT THE PIPELINE STILL MEANS WHAT IT SAYS

A gate is a test whose failure blocks acceptance of a program unit. The board is the ordered list of gates. Each entry names its dependencies, required software or hardware, owning documentation and executable check.

### A. Identity checks and smoke checks

An identity check isolates mathematics and serialization with small synthetic inputs. A smoke check runs a short real external pipeline, such as CAMB or CosmoLike and proves that the components connect on the production path. Each check supplies a distinct evidence layer.

An identity check should contain four named pieces:

1. the production function or public boundary under test,
2. a small constructed fixture with known properties,
3. an independently calculated expected answer.
4. a deliberately broken form that the check must reject.

The deliberately broken form establishes catch power by distinguishing correct and incorrect implementations.

### B. Test doubles

A test double replaces an expensive or unavailable collaborator. A stub returns predetermined values. A fake implements a small working substitute. A monkeypatch temporarily replaces the referenced function or class during a test. Every double must state which behavior it reproduces and which external behavior it cannot prove.

An independently known answer cannot be computed by the same helper that produces the value under test. Shared code would reproduce the same defect on both sides of the comparison.

### C. Exit status and the board

Each check records failures and exits nonzero when any assertion fails. The board judges the subprocess exit status and required output. A skip requires a distinct board status, while a zero return otherwise records a pass.

| Lifecycle time | Method | State it may change | Failure meaning |
|---|---|---|---|
| Component construction | `initialize` | Immutable predictors, axes, requirements | Artifact or option cannot define a component. |
| Graph negotiation | `must_provide` | Recorded output request | A downstream likelihood asks for an unsupported product or coordinate. |
| One sampled point | `calculate` | The supplied current-point state | The named cosmology cannot produce a valid current payload. |
| Likelihood read | `get_Cl` or family getter | No numerical state. Reads the current payload | The consumer asks for an unsupported serving convention. |

TABLE XXV. Cobaya lifecycle time is part of ownership. Validation at one row cannot substitute for validation at component construction or request negotiation.

Board resume records are scientifically valid only for the executable surface that produced them. Every live gate declares an executable manifest. The runner resolves its declared roots, transitive repository imports, named check scripts, shared runner code, reviewed dynamic-import coverage and named input keys. It records per-member hashes, an input digest and a raw-log digest. A change to any declared member invalidates the stored PASS automatically.

Before a run, preflight also requires a clean executable surface. It watches `emulator/`, `ai/gates/`, `compute_data_vectors/`, `cobaya_theory/`, `syren/` and the top-level Python drivers. A dirty generator or adapter is therefore refused just like a dirty training module. This check establishes that the evidence can be tied to a committed tree. Each gate's executable manifest separately declares which committed files belong to that gate.

Manifest membership is revalidated on every runner invocation. A dynamic import assembled from a string appears in a reviewed waiver entry whose roots cover the imported code. A subprocess target that the import graph cannot find is a declared root. A stored pass that predates manifest membership is classified as `pre-manifest` and must run again under the declared surface.

Raw-log integrity uses one symmetric predicate. The resume decision and the display use the same predicate. A PASS whose cited log is absent, lacks a stored digest or has edited bytes becomes `stale-log`. This state forces the selected gate to rerun and blocks downstream dependencies. The board self-test drives the real resume function through deletion, truncation, edit and missing-digest cases and includes a tamper arm that flips a current PASS to `stale-log`.

Hardware-specific PyTorch evidence belongs in a board-listed gate. CPU-only checks can prove pure validation and algebra. CUDA compilation, GPU dtype and accelerator-specific behavior require execution on the corresponding workstation.

## XXVIII. ONE COMPLETE ROW THROUGH THE PROGRAM

The entire pipeline can now be read as one sequence. Consider generated row $i$. It carries cosmology $\boldsymbol{\theta}_i$ and target $\mathbf{d}_i$.

1. The path resolver converts YAML file names to absolute paths.
2. The experiment validates the selected family and records parameter names from the covariance header.
3. Staging opens the data-vector dump lazily, loads the parameter table, verifies row counts, applies physical cuts and selects row $i$ as part of a seeded subset.
4. The source dictionary records how to address row $i$ in resident or file-backed storage.
5. The parameter geometry transforms $\boldsymbol{\theta}_i$ into whitened input $x_i$.
6. The output geometry and any target law transform $\mathbf{d}_i$ into target $t_i$.
7. A loader returns $x_i$ and $t_i$ on the model device as `float32` tensors.
8. The model predicts $\widehat{t}_i$.
9. The loss reconstructs the final physical prediction, forms the residual and computes $\Delta\chi_i^2$.
10. Backward differentiation computes parameter gradients. The finite checks, optional clipping, optimizer, anchor and moving average act in their defined order.
11. Validation scores every held-out row and selects the best candidate — a trained epoch or the incoming weights — whose weights and metrics refer to the same state, published as a selection record in the resolved recipe.

12. Saving writes model tensors plus the resolved recipe and geometries.

13. Rebuilding reconstructs the eager model, loads weights strictly and restores physical metadata from the artifact.

14. The predictor accepts a named cosmology, repeats the saved encode, model and branch-specific decode chain. Most families return physical units. The grid2d family returns the declared law-space surface and its consumer applies the persisted analytic base to obtain physical $P(k, z)$.

15. A Cobaya adapter exposes that result under the requested theory-product name and coordinate grid.

16. Identity and smoke gates prove the mathematical, serialization and external-pipeline boundaries independently.

The network is only one step in this chain. Most silent scientific failures occur at the boundaries: a mismatched row, an unnamed axis, a scale that underflows in storage, a target law applied on only one side, a stale external provider result or a saved recipe that does not describe the weights. The architecture matters, but the program is trustworthy only when every boundary states and checks what its numbers mean.

### A. Object ledger for the same row

The symbol $i$ does not have one universal coordinate meaning throughout the program. Before staging, it can denote an original dump row. After a sorted unique RAM gather, the same physical cosmology has a local compact-array coordinate. During an epoch it has a position in a new seeded permutation. The row payload is unchanged only if the coordinate maps are applied correctly.

Take a cosmic-shear row as a concrete example. The original parameter table might store 17 cosmological and nuisance parameters, while a physical cut retains 12 as the model input and a likelihood mask retains 780 of a larger data vector. Staging first proves that the parameter row and target row share one original coordinate. The parameter geometry returns a $(1, 12)$ tensor. The output geometry returns a $(1, 780)$ target. The model preserves the batch dimension and predicts another $(1, 780)$ tensor. The output geometry and loss use the persisted mask and covariance to give one scalar $\Delta\chi^2_i$.

Shapes $(1, 12)$ and $(1, 780)$ omit the cosmology, mask and axis identities. The row-coordinate maps, parameter names, destination indices and covariance owner provide that meaning. Saving those facts is therefore part of saving the model, even though none is a learned weight.

### B. Rebuilding every vector figure

Every numbered scientific figure in this manuscript is generated by `documentation/make_figures.py`. The script has one named function per figure and writes vector PDF files under `documentation/figures/`. Vector output stores lines, text and curves as drawing instructions, so labels remain sharp when the paper is enlarged. No screenshot is the source of a manuscript figure.

From the repository root, rebuild the complete set with

```
python3 documentation/make_figures.py
```

and then rebuild the paper. The script prints one `wrote` line for each expected figure. The figure builder runs independently of the emulator package and PyTorch. The diagrams explain fixed program structure and the activation curves use their formulas directly. Keeping figure generation independent of training prevents an unavailable GPU from blocking documentation builds.

The compiled `documentation/emulator_code_guide.pdf` remains tracked so a reader can open the guide without installing LaTeX. That convenience creates a freshness obligation: after any change to this source or a numbered figure, the PDF must be rebuilt and visually inspected before the change lands. The board does not currently perform that comparison, so the clean rebuild and rendered-page inspection are manual acceptance steps. The `.tex` timestamp supplies no acceptance evidence.

The residual-block figure is tied to the executed operation order documented in Sec. VII: every internal dense layer maps $W$ to $W$, the skip is added after the final internal dense layer and the final normalization and activation follow the addition. The activation figure places each legend outside its data rectangle so no label hides a curve. Regenerating the figures after a structural code change is one acceptance step. Review must also compare each caption with the implementation.

## Appendix A: The gate board, one executable claim at a time

This appendix explains all forty gates currently registered in `ai/gates/board.py`, in exact board order. A first-time contributor needs to know the gate name, the production path it executes, what its fixture means, which value determines the verdict and which plausible defect it can catch.

### 1. How to read each gate description

Every description below answers five questions: the claim made credible by a pass. The production path executed. The controlled fixture. The numeric, artifact or loud-error verdict and the deliberately wrong behavior that establishes catch power.

| Boundary | Object before | Object after | Equality that should hold |
|---|---|---|---|
| Row staging | Original dump row coordinate | Resident-local or disk-global coordinate plus `dump_rows` | Parameter row and every sibling target come from the same original row. |
| Parameter geometry | $\theta_i$ in physical units | $x_i$ in dimensionless network coordinates | `decode(encode(theta_i))` returns $\theta_i$. |
| Output geometry | $d_i$ on a named physical axis | $t_i$ in target coordinates | Decode returns $d_i$ on the identical axis. |
| Model and loss | $(x_i, t_i)$ | $(\widehat{t_i}, \Delta\chi_i^2)$ | Whitened and direct physical contractions agree. |
| Artifact | Live classes and tensors | Weight payload plus HDF5 recipe | Rebuilt eager prediction matches the live eager prediction. |
| Adapter | Named cosmology and requested axis | Current Cobaya theory product | Getter readback equals the predictor output in the declared serving convention. |

TABLE XXVI. Six identities for one row. A shape check alone proves none of them.

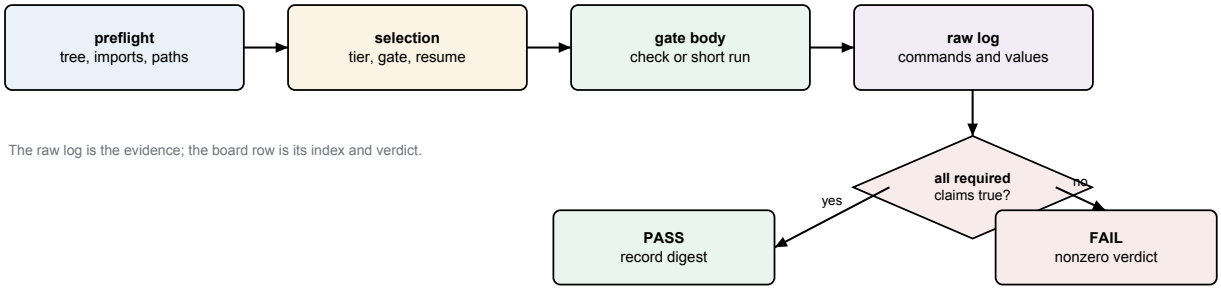



FIG. 10. The board's evidence flow. Preflight and selection determine what is eligible to run. The gate body produces evidence. The board converts all required claims into one explicit verdict.

An identity gate usually uses a small analytic fixture and asks whether a transformation, save/rebuild cycle or composition preserves a known value. A smoke gate runs a short production-shaped calculation and asks whether independently implemented components connect. A golden run compares selected output from the current tree with a pinned earlier tree. Golden equality is useful for an advertised no-op mode. Behavior absent from the pinned tree lies outside that comparison.

### 2. Gate vocabulary in plain language

**Fixture.:** A deliberately small input prepared for a check. Its coordinates and expected answer are known before the production function runs.

**Control.:** A nearby case known to be valid. A positive control must pass. A negative control must be refused. Controls prevent a gate from becoming red merely because its runner is broken.

**Mutation arm.:** A deliberately wrong implementation or input form inserted only in the test. It represents a plausible defect, such as a transposed axis or an inflated tolerance.

**Catch power.:** Evidence that the gate turns red for the mutation arm. Agreement on the correct fixture without a discriminating wrong arm can be a coincidence.

**Parity.:** Equality between two paths that are required to implement the same function, for example eager and compiled evaluation. An independent answer supplies the scientific truth beside this path-agreement check.

**Census.:** A mechanical enumeration of every member of a source-code class, such as every model constructor or every assertion site. A census prevents a repair from covering only the first instance found.

**Banner.:** Human-readable run text reporting a resolved setting. A banner is useful provenance, but construction or numerical behavior must prove that the reported setting was actually used.

**Sidecar.:** A small companion file that gives meaning to a larger numerical dump, for example column names or an axis array. Row-for-row and artifact-identity rules determine which sidecar belongs to which payload.

**Golden run.:** Output pinned from an earlier executable tree for a behavior promised to remain unchanged. It is a regression reference. New mathematics requires an independent derivation.

**Collapse bar.:** A threshold that a model which learned no useful dependence cannot beat. It turns a short smoke from "the program ran" into a minimal liveness claim.

**Provenance.:** Facts identifying how a result was produced: source digest, resolved configuration, coordinates, dtype, device and artifact identity. Provenance permits a reviewer to reconstruct the claim's scope.

### 3. The board runner as a small execution system

`ai/gates/run_board.py` is the command-line runner. `ai/gates/board.py` is the declarative registry. Keeping them separate means the list of scientific claims does not reimplement subprocess, path or resume handling. A board entry supplies an identifier, title, tier, home documentation, capability requirements, dependencies and a Python function that expresses the gate body through a constrained context object.

The context object is the gate's interface to the outside world. It can resolve a configured path, launch a check script, launch a driver, record a message and assert an expected condition. Every gate process launch goes through the context. This restriction gives the runner one place to capture command lines, standard output, standard error, return status and elapsed time.

Standard output is the normal text stream a process prints. Standard error is a separate stream intended for diagnostics. Exit status zero conventionally means success and nonzero means failure. The runner records all three. Checking only for a phrase is insufficient if the process later exits nonzero. Checking only exit zero is insufficient if the expected scientific assertion never ran.

### 4. Reconciling declared and executed claims

The registry is beginning to represent an acceptance leg with two strings: a plain identifier and an anchor in its home note. The identifier names the specific observation, such as `stage-ram.exact-fit-boundary`. The anchor points to the paragraph that defines what result is acceptable. Registry validation can prove that the anchor exists and that no two legs share an identifier.

Those checks establish declaration only. Execution requires a terminal record for each declared leg. Suppose a gate declares three legs but a conditional return skips the third. The remaining two can pass and the subprocess can exit zero unless the runner compares the declared and executed sets. The complete mechanism therefore needs one terminal record per declared leg:

$$D = E,$$

where $D$ is the set of declared identifiers and $E$ is the set of identifiers with an executed terminal result: `PASS`, `FAIL` or `UNAVAILABLE`. `UNAVAILABLE` is non-green. It states that a required hardware or software lane was unexecuted. The denominator retains that lane. A complete pass still requires its result. A crash before a check script emits its record is likewise a missing leg and makes the gate red.

A three-leg example makes the set equality concrete. Suppose the registry declares

$$D = \{\texttt{roundtrip}, \texttt{wrong-axis}, \texttt{real-provider}\}.$$

The check executes the first two legs and then returns before contacting the provider. Its executed set is

$$E = \{\texttt{roundtrip}, \texttt{wrong-axis}\}.$$

Both emitted results can say `PASS`, but $D \neq E$: the missing `real-provider` leg makes the gate red. Every emitted identifier must also occur in $D$ so it can be matched to a reviewed claim. `ctx.expect(label, condition)` is the in-process helper used by migrated gate bodies: `label` names the leg and `condition` is the boolean verdict. A false condition records failure and controls the gate result.

*a. Structured-evidence coverage.* For a gate that declares structured evidence, the runner checks that every human-readable `<gate-id>.<leg-name>` has one home-note anchor and exactly one terminal result. Unknown, duplicate and missing identifiers make that gate red. Eight gates still report one aggregate verdict without per-leg terminals: `finite-contract`, `finetune-identity`, `transfer-identity`, `save-rebuild-drift`, `cobaya-adapter`, `finetune-smoke`, `transfer-smoke` and `scalar-smoke`. For these gates, inspect the complete forced-rerun log and verify every advertised check before making a leg-specific claim.

### 5. Preflight protects expensive evidence

Preflight runs before GPU time. It checks the expected Git tip, watched-tree cleanliness, required imports and configured data paths. A workstation log is evidence about one executable surface. If the worktree contains unrecorded modifications, the log cannot be reproduced from the named commit.

Import checks establish capability availability. Gate execution remains a separate requirement. Importing

PyTorch proves that the module loads. CUDA kernels, compilation and mixed precision require their own executed checks. Opening a Cocoa data directory proves deployment availability. Adapter request correctness requires a separate executed check of the physical quantity.

*a. Watched-tree behavior.* The watched directory list now includes `emulator/`, `ai/gates/`, `compute_data_vectors/`, `cobaya_theory/`, `syren/` and the root drivers. The porcelain-status parser now preserves Git's leading two-column status field before examining individual lines. It excludes exactly `ai/gates/board_config.json`. An edit to neighboring `ai/gates/board.py` remains dirty. Self-test fixtures exercise the case in which the permitted config file is the first status line, because a whole- output `strip()` previously shifted that line alone. This clean-tree surface is shared by the report and the executed preflight decision.

*b. One project root for every child.* The runner resolves the project root from board configuration and injects it as `ROOTDIR` before launching a check or driver. It refuses an unresolved root and records the executed value in the raw log. The board self-test starts with one shell root and a different configured root, then verifies that the child opens files only below the configured root.

*c. A warning leg also checks process success.* The head-activation warning leg requires both a zero subprocess return status and the expected warning text. Its mutation fixture prints the correct warning and then raises, which makes the leg red. A printed phrase by itself is therefore insufficient evidence.

## 6. Selection, tiers, dependencies and resume

The runner can select gate identifiers, a tier or all gates at and after a named entry. Selection is validated against the registry before execution. A misspelled `-gate`, `-from` or `-force-rerun` identifier produces a nonzero usage error with nearby-name suggestions. The board self-test drives these public arguments and proves that an empty or dependency-skipped selection cannot report success.

A dependency states that one gate consumes an artifact or boundary produced by another. For example, adapter parity depends on successful save/rebuild. A dependency combines earlier list order with a confirmed verdict before the dependent body proceeds.

Resume skips a prior pass only when the executable and input digests, raw log and direct-dependency attempt lineage still match. It distinguishes stale code, stale inputs, stale dependencies, interruption, failure and dependency skip. Only current PASS is green. Force-rerun invalidates selected records without deleting unrelated logs.

*a. Executable manifests on the live board.* Every registered gate now declares an executable manifest. The runner derives membership from reviewed roots, persists per-file hashes and revalidates that membership on every invocation. Dynamic imports use reviewed waiver entries and subprocess targets outside the import graph appear as declared roots. A missing, truncated or digest-mismatched raw log becomes `stale-log`. A stored pass that predates its manifest becomes `pre-manifest`. Both states require a rerun.

## 7. Capabilities and where a claim is allowed to run

Each gate declares capabilities such as PyTorch, CosmoLike, Cobaya and GPU. The runner checks them before the body. Capability declaration prevents a missing dependency from being mistaken for a numerical failure. A missing capability produces the distinct non-green skip state described below.

Pure validation and NumPy algebra run on CPU. A claim about CUDA compilation must run on CUDA. A claim about Apple MPS float16 must run on Apple hardware. When a capability is optional, the board uses a distinct non-green skip state and states what remains unproven. Catching every exception and exiting zero would erase the difference between unsupported, broken and correct.

Hardware controls remain useful. A CPU or CUDA calculation that does not overflow records why that backend is only a control, while the affected MPS calculation supplies discrimination. The contract follows the executed dtype and device. Each accelerator needs its own evidence.

## 8. Anatomy of a pure numeric check

A pure check under `ai/gates/checks` is an executable Python file with a `main` function. It imports the production function, creates a bounded fixture, calculates an independent reference and accumulates named assertions. If any assertion fails, the process raises or exits nonzero.

The reference does not call the production helper under test. For a covariance contraction, the production side may use a banded implementation while the reference uses explicit loops and dense arithmetic on a tiny array. For a geometry, the reference may form the physical residual and direct inverse-covariance product. Agreement is valuable because the algorithms use distinct operations and failure modes.

Fixtures prioritize discrimination while retaining enough realism. A constant array can make correct and wrong axis orders coincide. An affine array can make several interpolation stencils exact. A strong fixture assigns distinct values to relevant coordinates and includes a control where the correct path is known to agree. The check prints actual and expected values so a red log carries both a diagnosis and a Boolean verdict.

| Stored situation | What it means | What the runner must do now |
|---|---|---|
| Current `PASS` | Code, declared inputs and raw-log digest still match | Reuse the pass unless the operator explicitly forces a rerun. |
| Stale code | An executable member changed | Rerun. The old log remains historical evidence for its old executable surface. |
| Stale input | A declared fixture or configuration input changed | Rerun with the new input identity. |
| Pre-manifest | The gate now declares an executable manifest but its stored PASS predates it | Rerun. The old record's narrower digest cannot certify the declared surface. |
| Stale log | The raw log is absent, truncated, edited or fails its digest | Rerun. A database status cannot substitute for its evidence. |
| Stale dependency | The stored pass consumed an older attempt of a direct prerequisite | Rerun the child after the current prerequisite is green. |
| Interrupted | The body began but did not write a terminal gate verdict | Rerun the interrupted gate. Earlier independent current passes may remain reusable. |
| Stored `FAIL` | The last complete execution was red | Rerun after a bounded repair. Never resume it as green. |
| Dependency skip | A prerequisite was not green | Run or repair the prerequisite before the dependent gate. |

TABLE XXVII. Resume states and their operational meaning. "Resume" means reusing evidence from an earlier invocation and skipping the gate body in the current invocation. A digest is a compact fingerprint of bytes. Matching digests support identity, while a mismatch proves that at least one byte changed.

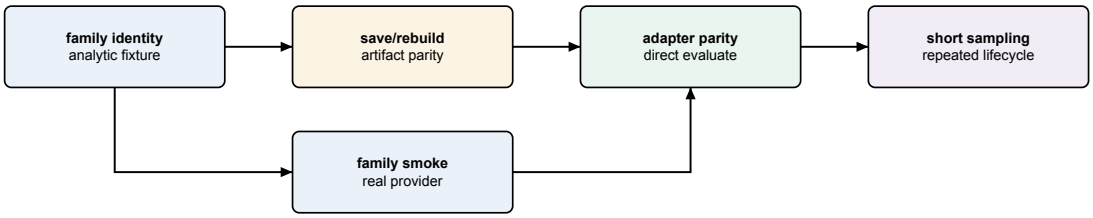



FIG. 11. A common evidence dependency. Identity establishes local mathematics, smoke establishes the real provider path, save/rebuild establishes persistence and adapter/sampling gates establish external use.

### 9. Anatomy of a training smoke

A smoke YAML is small but production-shaped. It uses the real driver, resolver, loaders, model, loss, optimizer, evaluation and saving path. Small means fewer rows or epochs while retaining the production trainer.

A collapse bar requires the network to learn more than a dead baseline. For an analytic scalar target, the baseline may always return the training mean. The acceptance bar belongs below that baseline. Merely requiring a finite final loss allows an unchanged or disconnected network to pass.

Banners reveal resolved facts that are hard to infer from one metric: activation, normalization, phase, learning rate, scheduler cadence and source artifact. They are readbacks. A behavioral leg supplies the truth source and accompanies a banner whenever the configuration might be printed but ignored.

### 10. Test doubles and monkeypatch scope

A stub returns predetermined values. A fake implements a small working collaborator. A monkeypatch temporarily replaces the object referenced by production code. Replacing CAMB with a fake can prove that an adapter requests and combines keys correctly. It cannot prove CAMB returns the assumed convention.

Patch scope must match the lookup site. Replacing a class definition in one module does not affect another module that already imported and bound the old class. A gate patches the name looked up by the production function and restores it before later control arms.

A convention-honest fake makes confusable representations genuinely different. For CMB lensing it assigns different arrays to raw $C_L^{\phi\phi}$ and the scaled representation. If the fake used equal arrays, the wrong convention would pass.

### 11. Mutation arms establish catch power

A mutation arm deliberately reinstates one plausible defect. The unchanged gate must fail under the mutation and pass under the production path. Examples include restoring width-squared chi-square tolerance, feeding scaled lensing spectra where raw spectra are required, using the old non-Gaussian weight and rebuilding under a drifted default.

The mutation is targeted. Changing many facts at once would not identify which assertion carries the catch power. Mutation evidence also detects a self-referential test: if actual and expected call the same helper, changing that helper changes both and the check remains green.

### 12. Tolerance is a derived part of the claim

Floating-point equality can be exact, absolute, relative or combined. Bitwise equality is appropriate when saving and loading the same tensor should perform no arithmetic. Relative tolerance is appropriate when the expected magnitude varies. Absolute tolerance supplies a floor near zero.

For a sum of $w$ float32 terms, the chi-square domain band scales with accumulation width $w$ and machine epsilon. A $w^2$ scale would admit materially negative scores. The band is computed from validated shape facts. The screened value has no authority to set it.

For example, float32 machine epsilon is approximately $\epsilon_{32} = 1.1921 \times 10^{-7}$. A kept residual width of $w = 780$ gives the declared domain band

$$\begin{aligned} b &= \max(10^{-6}, 32\epsilon_{32}w) \\ &\simeq 2.98 \times 10^{-3}. \end{aligned}$$

A computed score of $-10^{-3}$ lies inside this roundoff allowance and is normalized to the exact physical boundary zero. A score of $-10^{-2}$ lies outside it and is rejected before ranking. Replacing $w$ by $w^2$ would widen the same example to about 2.32 and allow an impossible negative score to look perfect. The numbers show why "some small negative tolerance" omits required inputs: dtype and contraction width are part of the claim.

Every widened tolerance needs a measured valid control and a recorded derivation. Adjusting a tolerance until a red result turns green would let the defect define its own acceptance region.

### 13. Raw logs and evidence readback

Each raw log begins with gate identity, home documentation, Git commit and capabilities. It records commands and complete output. Named lines state the value behind each check and the final line is the gate verdict.

A compact illustrative excerpt is

```
gate: scalar-identity
commit: 1a2b3c4
leg scalar-identity.roundtrip: PASS
  max_abs=2.1e-07 tolerance=1.0e-06
leg scalar-identity.bad-name: PASS
  refusal=unknown scalar column
gate verdict: PASS (2 declared, 2 executed)
```

The first leg's label, measured value and tolerance explain a numerical pass. The second leg proves a malformed name was refused. "PASS" refers to the refusal behaving correctly. The final counts establish completeness only when the runner has reconciled the declared and executed identifiers. This excerpt teaches the intended record shape. Quoted workstation evidence must come from a raw log.

`logs/BOARD.md` is an index. A reviewer must use the raw evidence. A reviewer opens the raw log, confirms every advertised leg executed and reads values near thresholds. A pass at 4.99 percent under a 5 percent bar carries different operational risk from a pass at 0.1 percent.

Readback comes from the real consumer surface. The gate reopens the artifact or reads the adapter's public result, so the evidence covers the saved HDF5 record as well as the in-memory configuration.

### 14. Backlog tier: training and data contracts

#### a. `ema-off-identity`: *the absent feature is a true no-op*

**Claim and path.** Disabling the exponential moving average, EMA, leaves pre-EMA training byte-identical. The normal driver runs on the current tree and in a temporary worktree at the pinned pre-EMA commit. A worktree is a second checkout sharing Git objects. It does not switch the user's active checkout.

**Fixture and verdict.** Both runs use the short gate YAML. The scanner extracts declared epoch lines and compares their bytes. The current-tree run also exercises a functional smoke so an empty comparison cannot pass.

**Catch power.** Creating an averaged copy in the off branch, changing selection, consuming randomness or changing optimizer order alters a line. The next gate owns the EMA-on behavior claim.

#### b. `ema-smoke`: *the averaged state participates in a run*

**Claim and path.** With EMA enabled, the epoch-based horizon is used and a plateau rewind restores coherent state. A production training run uses `ema.horizon_epochs=3`. The normal optimizer, scheduler, selection and rewind code execute.

**Fixture and verdict.** The YAML causes a learning-rate cut during the short run. Banners must identify the horizon and rewind and the process must finish with the expected metrics.

**Catch power.** Removing the EMA update, converting the horizon incorrectly or rewinding live weights without their averaged partner removes a required observation.

*c.* `production-diagnostic`*: one report crosses several boundaries*

**Claim and path.** The production diagnostic command can stage cut data, train and create the expected report while exposing accurate size and coverage facts. The cosmic-shear driver runs with `-diagnostic` and uses production plotting code.

**Fixture and verdict.** The gate checks the model-class census, rows surviving the density window, printed train/validation sizes and shaded coverage triangle. A file suffix alone is not acceptance.

**Catch power.** Dead model branches, cuts applied after selection, misreported sizes or a plot omitting the declared window breaks a leg.

*d.* `single-phase-demotion`*: phase keys are handled deliberately*

**Claim and path.** A single-phase `resmlp` can receive the documented phase-shaped configuration without a traceback, while a structured model retains two-phase behavior. Both pass through the real resolver and driver.

**Fixture and verdict.** The single-phase fixture contains head-shaped keys and must train under the declared demotion. A two-phase control must still execute separate phases.

**Catch power.** Forwarding a head-only option into `resmlp` reproduces the old failure. Globally deleting phase settings makes the two-phase control fail.

*e.* `head-scheduler-override`*: the head owns its cadence*

**Claim and path.** A head scheduler replaces the relevant top-level settings and cuts the head learning rate on its own patience. The real two-phase resolver constructs the scheduler and the epoch loop feeds its validation metric.

**Fixture and verdict.** The override uses patience ten. The resolved banner must name it and the cut must occur on the matching cadence.

**Catch power.** A shallow merge that retains top-level patience, a scheduler attached to the wrong optimizer or a metric-free step shifts or removes the cut.

*f.* `eval-batch-invariance`*: scores are invariant under partitioning*

**Claim and path.** Validation scores remain invariant when the same ordered rows are partitioned differently. The check calls real `eval_val` with several batch sizes, including its production final-batch padding path.

**Fixture and verdict.** Per-row scores and aggregate metrics agree at relative tolerance $10^{-6}$. Timing supplies a liveness signal outside the numerical verdict.

**Catch power.** Counting padded duplicates, leaking state between batches, batch-relative normalization or dropping a final short batch changes at least one per-row score.

*g.* `finite-contract`*: impossible scores never rank*

**Claim and path.** NaN, infinity and materially negative chi-square values are rejected at training, validation, diagnostics and warm-start parity boundaries. The PyTorch check calls the production reduction, `eval_val`, source diagnostic, safe square roots and parity helpers.

**Fixture and verdict.** Red fixtures include NaN in either parity arm, a nonfinite transfer surface, exact-fit zero, finite losses near the float32 maximum and finite negative chi-square. Valid controls include analytic positive square roots and tiny roundoff from an ill-conditioned positive-definite contraction. The allowed roundoff band scales with kept per-row width. Width-squared scaling is the rejected mutation.

**Catch power.** A mutation restoring the old width-squared band crowns a materially negative production-width score as perfect. Finite-only validation lets NaN comparisons evade threshold counts. On a supported compile lane, an exception produces failure. A green skip is forbidden.

*h.* `padded-head-identity`*: storage cells cannot become data*

**Claim and path.** Structured CNN and Transformer heads preserve the original physical coordinate of every kept output and keep artificial rectangle cells inert through every operation that could make them nonzero. The saved geometry and model checkpoint must describe the same map and mask. The writer verifies that agreement before staging either output file, and the reader verifies it independently before loading learned tensors.

**Fixture and verdict.** Small CPU fixtures use equal-count bins with different angular masks, a fully masked middle bin, nonzero-preserving activations, FiLM shifts, live convolutions and live attention. A second leg saves a model with a nonzero CNN correction, reopens it through the public reader and requires exact predictions and buffers. Authenticated checkpoints with a missing or disagreeing fixed layout buffer must be refused. A live

model paired with a different geometry must also be refused before staging, without changing a preceding valid artifact pair.

**Catch power.** Reconstructing positions from counts, masking only the initial scatter, applying only a token-level attention mask or trusting the checkpoint over the HDF5 geometry makes a focused witness fail while ordinary array shapes remain valid.

*i.* `board-selftest`*: the runner reports what actually ran*

**Claim and path.** The board's own selection, dependency, resume, atomic-status and evidence-map machinery cannot manufacture a green command from an empty, skipped, stale or interrupted run. The pure-Python check drives the real `run_board.main`, `select_gates` and resume helpers with small fake Gate objects.

**Fixture and verdict.** Fake gates pass, fail, skip through a failed dependency or raise an uncaught interruption. Unknown selectors and force targets must return nonzero with a suggestion. Mutating a gate body, referenced YAML, input config, evidence anchor or cited log must invalidate the corresponding record. For the currently migrated structured-evidence entries, every declared anchor must resolve to a real note location and every identifier must be globally unique. The live `-list` command reports the current registry size; a number copied into this guide would become stale. Most gates still lack individual leg declarations in the structured evidence map.

**Catch power.** A status-only resume rule, warning-only selector, log overwrite or board row whose note anchor does not exist makes one of the fake scenarios falsely green and is rejected. This gate tests runner control flow. Family gates supply the scientific evidence.

*j.* `generator-seed`*: sampling owns one recorded random stream*

**Claim and path.** Generator sampling is reproducible from one required integer seed. The seed constructs an owned NumPy `Generator` that is threaded through uniform draws, walker initialization, sampler moves and thinning. Process-global `np.random` state has no role in those draws.

**Fixture and verdict.** Two runs with the same seed produce identical draws and the chain header records that seed. Invalid seed types fail before sampling. A monkeypatch of global random functions must not affect the owned stream.

**Catch power.** One leftover global draw changes the same-seed result or triggers the monkeypatch. The pure gate proves ownership and local reproducibility. Append/restart replay and MPI worker-count invariance require their generator smoke legs.

*k.* `cli-strict`*: misspelled flags stop before expensive work*

**Claim and path.** Every public executable uses strict argument parsing. The check performs a source census and drives representative driver `main` functions through their real parsers while replacing the expensive training boundary with an instrumented stub.

**Fixture and verdict.** A valid command reaches the stub exactly once. The misspelling `-activaton` exits nonzero, names the unrecognized argument and reaches the stub zero times. All public entry points must use `parse_args`. The permissive `parse_known_args` form is forbidden.

**Catch power.** Silently retaining unknown arguments or manually discarding parser leftovers lets the misspelled option train a default model. The zero-call assertion exposes that behavior.

*l.* `family-first`*: each driver owns one data family*

**Claim and path.** A direct driver selects one physical data-block family before experiment construction. The cosmic-shear driver cannot become a CMB, scalar, background or matter-power run merely because the YAML contains that block, while each thin family wrapper accepts its own block.

**Fixture and verdict.** Wrong-family configurations fail at startup with both expected and received families. A clean cosmic-shear configuration reaches training and the four family wrappers reach their matching boundary. A census verifies that every cosmic-shear campaign pins the same family.

**Catch power.** Restoring automatic family dispatch makes a wrong-driver fixture run. Pinning only the single-run driver leaves a sweep or search census failure. The gate therefore covers the single-run, sweep and search drivers.

*m.* `stage-ram`*: the memory decision counts every copy*

**Claim and path.** `stage_source` budgets the compact parameter copy, compact target copy and row-reindex storage before choosing resident or file-backed staging. Parameter and target widths and dtypes are counted separately.

**Fixture and verdict.** A narrow target with a wide parameter table is chosen so target bytes alone fit but the coordinated copies exceed the budget. The real stage function must stay disk-backed. Resident and disk-backed controls return byte-identical selected rows, including an unequal-dtype case.

**Catch power.** A mutation restoring the old target-only estimate selects resident storage and fails the regime verdict even though its returned numbers remain correct. This separates resource truth from numerical truth.

*n.* `artifact-readback`*: stored values keep their types*

**Claim and path.** Artifact Boolean attributes are parsed by an explicit typed reader. Python truthiness is forbidden. The check exercises the shared reader and scans every artifact Boolean read site.

**Fixture and verdict.** A native Boolean is accepted, an absent key uses its named default. String or integer lookalikes, including the truthy string `"False"`, raise with file and schema context. The census requires all read sites to use the shared boundary.

**Catch power.** Replacing the reader with `bool(value)` turns `"False"` into true and can select drifted transfer weights. This pure gate proves type parsing and source coverage. The live save/forge/rebuild artifact leg remains a separate workstation obligation.

*o.* `diagnostics-domain`*: invalid scores are refused before reports*

**Claim and path.** Diagnostic producers apply the same finite, nonnegative chi-square boundary as training and evaluation before converting scores into coverage fractions or residual summaries. The check calls the real local-linear floor and CMB residual diagnostic and censes grid-family producer sites.

**Fixture and verdict.** Valid scores pass byte-identically. Negative roundoff inside the width-derived band becomes exact zero. Materially negative, NaN and infinite values raise with boundary, rows and band. A reachable one-coordinate negative floor is refused before `f_floor` is formed.

**Catch power.** Bypassing the screen recreates the former false `f_floor=0` "perfect" result. A valid diagnostic control prevents the repair from converting all small or difficult results into errors.

*p.* `berhu-loss`*: the objective matches its piecewise law*

**Claim and path.** The reverse-Huber objective has its declared value and derivative and a head can select it independently of a square-root trunk. A pure check calls the production reduction and autograd. A short two-phase run exercises schema resolution.

**Fixture and verdict.** Analytic points lie below, on and above the knot and cap. Required continuity is checked, while banners name square root for the trunk and capped BerHu for the head.

**Catch power.** Residual-norm units, a swapped branch inequality or a head resolved to the trunk loss change an analytic value or banner.

*q.* `loss-schema-equivalence`*: syntax migration preserves meaning*

**Claim and path.** Equivalent flat and nested loss settings resolve to the same run. The ordinary driver resolves and trains both forms.

**Fixture and verdict.** Selected numerical epoch lines reproduce the pre-migration result.

**Catch power.** Reading a key from the wrong nesting level, falling back to a new default or accepting conflicting spellings changes those lines. This equivalence check covers configuration migration. The analytic loss gate separately covers the numerical formula.

*r.* `berhu-anneal`*: endpoints have mathematical meaning*

**Claim and path.** BerHu annealing begins as square root, stays continuous at the hold boundary and reaches full BerHu at the declared endpoint. The shared production schedule supplies the blend coefficient.

**Fixture and verdict.** Values immediately around the hold boundary are compared, the epoch-15 fixture must have coefficient one and a no-anneal control preserves absent-key behavior.

**Catch power.** Reversing an increasing schedule, an off-by-one boundary or treating `const` as a generated ramp changes a sampled coefficient.

*s.* `ema-anneal`*: the average becomes live at the intended time*

**Claim and path.** EMA annealing excludes the deliberately poor early trajectory and evaluates averaged weights at the live point. The normal EMA update, shared schedule and model-selection path execute.

**Fixture and verdict.** A no-anneal control is compared with an annealed run. Output identifies the hold and metrics from the live averaged state.

**Catch power.** Updating during the hold, interpreting the horizon in the wrong unit or evaluating raw weights under an EMA banner breaks the transition.

*t.* `param-window-cuts`*: selection follows physical cuts*

**Claim and path.** A tight density window uses named parameter columns, shrinks the candidate pool by the independent count and then trains. The production cut resolver and staging selector operate on the real table.

**Fixture and verdict.** The expected surviving count is calculated independently and compared with the banner before the requested absolute number of rows is chosen.

**Catch power.** Cutting the wrong column, treating a missing side as zero, selecting before cutting or silently accepting too few rows changes the count or run verdict.

*u. `triangle-shading`: visualization states the same domain*

**Claim and path.** The optional coverage triangle shades all declared physical windows on their corresponding panels. The CPU check calls the production plotting helper and inspects its Matplotlib artists.

**Fixture and verdict.** Four nondegenerate synthetic windows have known panels and extents. The test counts fill objects and verifies coordinates.

**Catch power.** Shading only the last window, swapping axes or using untransformed bounds creates the wrong artist list.

## 15. New-features tier: model and family identities

*a. `joint-training`: a live trunk and head really train together*

**Claim and path.** With `freeze_trunk=false`, the second phase keeps both trunk and correction head trainable. The production phase transition sets `requires_grad`, creates optimizer groups and runs the joint graph.

**Fixture and verdict.** The banner states joint training and the check compares continuity across the phase boundary. Phase-two epoch time must sit above a frozen-trunk control because backward now traverses the trunk as well as the head. Timing supplies supporting liveness evidence. Parameter updates and gradient ownership carry the primary verdict.

**Catch power.** Leaving trunk parameters frozen, omitting them from the optimizer or evaluating a detached trunk can still let the head improve. Trainable-parameter counts and the control comparison expose those partial implementations.

*b. `head-activation-pin`: one head may own a different curve*

**Claim and path.** A structured head can pin a `multigate` activation while the trunk retains its model-wide family. Illegal phase combinations and head activations whose implemented derivative vanishes at zero are refused before training.

**Fixture and verdict.** The resolved model-spec banner names the head family and gate count. The parameter census must increase by the analytically expected activation tensors. A configuration that pins a head activation while requesting incompatible joint behavior must raise the teaching error.

**Catch power.** A resolver that prints the pin but still supplies the trunk factory to the head fails the parameter-count leg. Silently ignoring the illegal combination fails the error leg. This gate establishes construction. Separate behavioral head checks establish activation liveness.

*c. `relu-tanh-norm`: fixed baselines use the requested normalization*

**Claim and path.** Parameter-free ReLU and tanh activations can be constructed through the same factory interface and tanh runs with either shared affine or per-feature normalization as declared.

**Fixture and verdict.** Short tanh runs require the resolved norm in the banner and a descending loss. An absent-key control reproduces the default normalization behavior.

**Catch power.** Passing the feature width into `nn.Tanh`, sharing one mutable normalization instance across layers or silently selecting the default when `per_feature` was requested changes construction, parameter count or banner. Loss descent prevents a build-only false green, though separate head gates are needed for zero-initialization compatibility.

*d. `weight-decay-census`: decay follows module role*

**Claim and path.** AdamW decay applies exactly to weight matrices owned by `Linear`, `Conv1d` and `BinLinear`. It excludes biases, normalization gains and activation-shape parameters regardless of tensor shape.

**Fixture and verdict.** A model containing gated-power activations and structured layers is walked through the production optimizer factory. The check compares parameter object identities in the decay group with an independent owner-role census. A zero-decay golden control protects the off-mode behavior.

**Catch power.** The tempting shortcut `parameter.ndim > 1` incorrectly classifies some learned tensors and can miss custom weight owners. Because the expected set is named by owning modules, a shape-based mutation fails even if the total number of elements happens to match.

*e. `npce-training`: the analytic base and neural correction compose*

**Claim and path.** Neural polynomial-chaos emulation, NPCE, trains in both residual and ratio forms, persists enough state to rebuild, rejects mutually exclusive configurations and refits its base independently at each training-size sweep point.

**Fixture and verdict.** Short runs exercise both combination laws. Error fixtures request incompatible PCE/transfer or composition choices. A two-point $N_{\text{train}}$ sweep verifies that each point receives a base fit from its own selected rows. Reuse of the first point's coefficients fails the second fixture.

**Catch power.** A module-global PCE, a cached term selection shared across points or a rebuild that restores only the neural residual changes the second point or

round-trip. Successful training alone would not prove the base was refit.

*f.* `finetune-identity`*: warm start begins as the source function*

**Claim and path.** Fine-tuning validates the source artifact, extends input geometry only according to the declared rule, pins the output geometry and produces epoch-zero predictions equal to the source before any optimizer step.

**Fixture and verdict.** The pure PyTorch check covers equal parameter sets, allowed extensions, anchor masks, constant or degenerate columns and configuration refusals. It compares both eager source predictions and the newly constructed training-side path at epoch zero.

**Catch power.** Recomputing the output center from new targets, mapping an extended input into the wrong column, loading weights non-strictly or applying an anchor before parity changes the epoch-zero result. Loud-error legs ensure an unsupported mismatch cannot be mislabeled as numerical drift.

*g.* `transfer-identity`*: a zero correction is exactly the base*

**Claim and path.** Frozen-base transfer slices the inputs required by the source, evaluates the source in its own encoded space, initializes a zero correction and composes gain or sum forms without altering the epoch-zero function.

**Fixture and verdict.** The check exercises several combinations of base-space and final-space composition, target packing, parameter surgery and family-kind refusals. A grid-family arm passes through its logarithmic law and treats outputs according to their family geometry.

**Catch power.** Applying the correction in the wrong coordinate space, double-encoding base inputs, unpacking the staged target at the wrong offset or testing a family mismatch only after another invalid fixture condition changes parity or the expected error.

*h.* `scalar-identity`*: a named scalar survives every boundary*

**Claim and path.** Scalar geometry, model state, save/rebuild, predictor output and automatic Cobaya provision preserve named derived quantities exactly.

**Fixture and verdict.** The check uses analytic scalar targets and exercises subsets, duplicate names, constant columns, wrong artifact kinds, trunk-only architecture constraints and fine-tune parity. Bitwise equality is required where no floating-point recomputation is warranted.

**Catch power.** Width-only assembly can return the right number of scalars in the wrong order. The named subset and duplicate-name legs catch that class, while constant-column and wrong-kind legs prevent a finite-looking but undefined geometry from passing.

*i.* `cmb-identity`*: spectra, amplitude law and covariance agree*

**Claim and path.** The CMB family preserves spectrum conventions, applies its amplitude/optical-depth law on the correct side of the metric, rebuilds exactly, evaluates roughness as specified, supports its structured head and constructs non-Gaussian band covariance weights correctly.

**Fixture and verdict.** The check covers ruled per-$\ell$ constants, forward/inverse amplitude laws, physical-$\chi^2$ invariance, required parameters, roughness zero and high-frequency controls, fine-tune parity and a ResTRF identity-basis rebuild. Five covariance legs compare the production contraction with an independent known answer: exact contraction, deliberate old-weight miss, raw-versus-scaled lensing fixture integrity, width-three band projection and exact zero-band weight.

**Catch power.** Feeding scaled $[L(L+1)]^2 C_L^{\phi\phi}/(2\pi)$ where raw $C_L^{\phi\phi}$ is required creates a large known miss while the raw control matches. The round trip tests internal consistency, while the independent contraction exposes a consistently wrong convention.

*j.* `bsn-identity`*: background functions obey their trained domain*

**Claim and path.** Background-grid geometry, logarithmic-offset law, piecewise redshift windows, loud desert between windows, save/rebuild, adapter getters and fine-tune parity agree with closed-form flat-ΛCDM fixtures.

**Fixture and verdict.** Analytic $H(z)$ values span each trained window and approach its boundaries from both sides. Requests in the untrained gap must raise. Any interpolation across that gap fails the fixture. Derived distance relations are compared with independently integrated values.

**Catch power.** A generic interpolator can return smooth, positive and monotonic values in an untrained region. Those properties are insufficient. The explicit desert leg makes that plausible extrapolation red. The save/rebuild arm catches a law or window omitted from state.

*k.* `mps-identity`*: a two-dimensional surface keeps its coordinates*

**Claim and path.** Grid2d staging, geometry laws, constant-column pinning, matter-power base assembly, fine-tune parity, structured correction heads, stable streaming moments and staging-file lifecycle all preserve the declared $(z, k)$ surface.

**Fixture and verdict.** Stub bases make the assembly algebra exactly known. Law-space rows are compared with independently transformed values. Stable-moment fixtures contain large offsets and float32 payloads and several chunkings must reproduce the stored-payload population variance. Lifecycle legs restage train and validation slots, verify superseded temporary files are unlinked and check cleanup after failure and sweep-point release.

**Catch power.** The retired sum/sum-of-squares variance changes with chunk order under cancellation. A precast float64 reference disagrees with the actual float32 payload. A gate that checked only `isinstance(memmap)` would miss leaked multi-gigabyte files. Absence checks after restage provide the discriminating evidence.

*l.* `geo-paths`*: geometry classes have one import home*

**Claim and path.** New artifacts persist class paths under the `emulator.geometries` package, legacy flat paths fail loudly and no live source import retains the retired geometry homes.

**Fixture and verdict.** The check creates fresh states, inspects their class markers, attempts legacy reconstruction and performs an untruncated tree census for old imports.

**Catch power.** Compatibility shims can make a local rebuild pass while allowing new files to continue publishing obsolete paths. Requiring both the new marker and the loud old-path refusal prevents that half-migration.

### 16. Save-and-sample tier: artifacts and external execution

*a.* `save-rebuild-drift`*: an artifact is an executable recipe*

**Claim and path.** Saving and rebuilding produces a bitwise-equal function for plain, factored, NPCE and convolutional-head models and the artifact remains governed by its persisted recipe when source-code defaults later drift.

**Fixture and verdict.** The check trains or constructs each supported form, saves the weight/HDF5 pair, rebuilds through `rebuild_emulator` and compares predictions before and after. For a structured head, persisted bin splits must reconstruct the same scatter layout. The check temporarily perturbs a code default and proves that the stored resolved value wins. Version-one and pre-persistence head artifacts must be refused with migration guidance.

**Catch power.** A weight-only test can pass while rebuilding a different geometry or layout. A current-default test can pass until the default changes. The deliberate drift arm establishes artifact facts as the owner of reconstruction. It perturbs ambient source defaults.

*b.* `cobaya-adapter`*: inference repeats the training-side function*

**Claim and path.** The Cobaya theory adapter reads named sampled parameters, calls the rebuilt predictor and returns the same physical prediction as the training-side model, including factored recombination.

**Fixture and verdict.** A parity check evaluates the same off-center cosmology through both paths and requires relative agreement at $10^{-6}$. A Cobaya `evaluate` call then verifies the public lifecycle and a short MCMC smoke confirms repeated calls under the sampler.

**Catch power.** Reordered parameters, a missing decode, a factor law applied only in training or stale adapter state changes parity. The MCMC arm adds lifecycle evidence that one direct call cannot provide, while the direct known-input comparison supplies the precise numeric verdict.

*c.* `finetune-smoke`*: a warm start can improve and republish*

**Claim and path.** A real fine-tune begins at the source function, continues training on new data, writes source provenance and saves an artifact that rebuilds to the final fine-tuned function.

**Fixture and verdict.** The run consumes the board's own previously saved emulator. Before the first update, it prints and passes epoch-zero parity. It then completes, improves the declared metric, writes `finetuned_from` and survives save/rebuild.

**Catch power.** Loading source weights but recomputing incompatible geometry can still lead to eventual loss descent. The pre-update parity makes that path red immediately. Writing provenance without embedding the facts needed for reconstruction fails the final round-trip.

*d.* `transfer-smoke`*: a frozen base and correction complete a run*

**Claim and path.** A real transfer configuration loads the board's base artifact, composes a zero-start correction in the declared gain form, preserves epoch-zero parity, trains and saves a self-contained transfer artifact.

**Fixture and verdict.** The banner names the base path and composition form. The pre-train parity line

must pass, training must complete under its collapse bar and the saved HDF5 record must contain transfer provenance plus the embedded base facts needed for rebuild.

**Catch power.** Depending on the external base file after publication, packing the base and truth in the wrong order or composing in the wrong space may still produce a trainable correction. Parity and isolated rebuild separate those defects from ordinary optimization error.

#### e. `scalar-smoke`: *a complete analytic scalar example*

**Claim and path.** The scalar pipeline can learn a derived quantity whose formula is independently known, predict away from the training center, serve it through the scalar Cobaya adapter and generate its diagnostics.

**Fixture and verdict.** The target is analytic $\Omega_m h^2$. A short training must beat a collapse bar chosen below the mean-predictor baseline. An off-center cosmology avoids the degenerate case where every implementation agrees at the center. Cobaya `evaluate` readback is compared with the formula and expected diagnostic pages must be created.

**Catch power.** Returning the center, swapping parameter columns or advertising the scalar without inserting its calculated value can look plausible on a centered fixture. The off-center analytic readback makes each wrong path visible.

#### f. `cmb-smoke`: *real CAMB reaches training and inference*

**Claim and path.** The CMB generator, covariance script, training driver, artifact rebuild, Cobaya $C_\ell$ lifecycle and family diagnostics connect on real CAMB output. The non-Gaussian covariance path also executes with its normalization facts persisted.

**Fixture and verdict.** A smoke-scale CAMB run generates spectra. The covariance output is checked structurally, including non-diagonal blocks and provenance keys. Training must cross a relative collapse bar, the adapter must return the requested spectra and diagnostic pages must complete.

**Catch power.** This gate detects interface and convention failures that a synthetic fake cannot: CAMB key names, raw versus scaled spectra, actual coordinate ranges, covariance file shape and adapter lifecycle. The independent `cmb-identity` contraction supplies the numerical truth.

#### g. `bsn-smoke`: *background emulators agree with CAMB*

**Claim and path.** The background generator, two family trainings, saved predictors, Cobaya getters, derived-distance calculations and grid diagnostics work end to end against CAMB's own background solution.

**Fixture and verdict.** Real CAMB supplies $H(z)$ and reference distances at smoke-scale redshifts. A stale-cache tripwire ensures each generated row belongs to the current cosmology. The served values must agree within the declared two-percent smoke tolerance and the grid diagnostic pages must render.

**Catch power.** A generator can return a finite array cached from the previous point and a smooth interpolator can make it look reasonable. The generation token/tripwire and independent CAMB comparison jointly expose that class. Exact formula and desert-boundary behavior remain in the identity gate.

#### h. `mps-smoke`: *the matter-power surface completes EMUL2-shaped service*

**Claim and path.** The matter-power generator, two trainings, grid2d artifacts, Cobaya `get_Pk` lifecycle and residual-surface diagnostics connect on real CAMB data under the no-analytic-law path.

**Fixture and verdict.** CAMB produces a smoke-scale $P(k, z)$ surface on declared coordinates. Both trained pieces save and rebuild, the adapter returns surfaces on the requested grid and values are compared with the real reference within the smoke contract. The family pages must finish without materializing the production-size diagnostic cube.

**Catch power.** Transposing $z$ and $k$, relabeling an equal-width axis, using the training axes for validation or assembling two artifacts in the wrong law space changes the surface. Stubbed identity legs prove exact assembly mathematics. This smoke proves the real CAMB and Cobaya interfaces.

### 17. Gate-by-gate failure triage

The first response to a red gate is to locate the smallest executed claim that failed. A tolerance change requires a numerical derivation. A rerun follows diagnosis. A gate with structured evidence exposes stable leg identifiers and the runner reconciles them with its declared set. For an aggregate-only gate, the check-script labels and raw log are the available evidence, so inspect the whole log before attributing a pass to one internal leg. The following guide identifies the first evidence to inspect for every board entry.

*a.* `ema-off-identity`. Diff the first unequal epoch line and determine whether the difference begins before model construction, at optimizer setup or at evaluation. A changed random draw suggests the off path consumed state. A changed metric with identical early banners suggests an off-mode branch entered the training or selection calculation.

*b.* `ema-smoke`. Read the resolved horizon and executed steps per epoch, then locate the learning-rate cut and rewind lines. If the cut exists but rewind is absent,

inspect scheduler-to-rewind control flow. If rewind exists but later metrics jump, compare which of live weights, optimizer moments and averaged weights were restored.

*c.* *production-diagnostic.* Separate a training failure from a report failure. Verify the cut-pool and size banners before opening the plotting traceback. If training succeeded but a page is absent, inspect the first diagnostic function that did not emit its completion marker. That missing marker identifies the first local report failure.

*d.* *single-phase-demotion.* Inspect the resolved architecture before the exception. A constructor receiving head keys indicates demotion happened too late. If the single-phase arm passes and the structured control fails, the resolver likely deleted phase information for every architecture.

*e.* *head-scheduler-override.* Compare the printed head patience with the epoch of the first cut. Correct printing with wrong cadence points to scheduler construction or metric stepping. Wrong printing points earlier, to precedence resolution. Confirm that the scheduler holds the head optimizer object. The trunk optimizer has a separate owner.

*f.* *eval-batch-invariance.* Find the first row whose score changes and whether it lies in the final batch. A difference only at the end implicates padding or trimming. Differences at every boundary suggest batch-relative state or reduction. Compare per-row arrays before comparing rounded medians.

*g.* *finite-contract.* Use the failed part label to distinguish producer, boundary, accumulation, tolerance, parity and compile arms. Print the offending row count and minimum. Do not weaken the predicate until the valid ill-conditioned control and the production-width negative mutation have been evaluated together.

*h.* *padded-head-identity.* Read the failed leg first. A layout failure points to scatter coordinates, per-operation masking or a fully masked row. An artifact failure points to geometry persistence or fixed model buffers. If saving fails, compare the live model with the resolved recipe before looking for a file error. If rebuilding fails, inspect the independent comparison before strict state loading. Do not replace the physical map with bin counts merely because the counts make the rectangle shape agree.

*i.* *board-selftest.* Classify the failure as selection, dependency, resume digest, interruption, log atomicity or evidence-map integrity. Inspect the fake gate's call count before the stored status: a body that did not run can never supply scientific evidence. For digest failures, change one input at a time and confirm which stale category the runner reports.

*j.* *generator-seed.* Compare the first differing draw and identify which sampling stage owns it. A difference before the sampler implicates uniform or walker initialization. A later difference implicates moves or thinning. Search for process-global random calls only after confirming the seed reached the owned generator.

*k.* *cli-strict.* Record parser exit status and expensive-boundary call count together. A nonzero traceback after the stub ran is too late. The typo must be rejected by argument parsing. A source-census failure names the executable that still accepts unknown flags.

*l.* *family-first.* Read the requested driver family beside the YAML's detected data block. If a wrong-family run reaches experiment construction, the driver omitted its pin. If a correct thin wrapper fails, inspect only its explicit family argument before changing shared dispatch.

*m.* *stage-ram.* Recalculate parameter bytes, target bytes, index bytes and budget separately. Agreement in values plus a regime mismatch locates the defect in accounting. The staging algebra has already matched. Compare resident and disk row coordinates before comparing payloads.

*n.* *artifact-readback.* Print the stored HDF5 type and value before conversion. A string that looks like a Boolean remains an invalid stored value. The reader must reject it without coercion. If the pure reader passes but a live rebuild fails, locate the read site that bypassed it.

*o.* *diagnostics-domain.* Identify the producer and its declared reduction width. Compare the raw score with the derived band before any plotting or availability logic. A within- band negative should normalize to zero. A material negative or nonfinite value should never reach a statistic.

*p.* *berhu-loss.* Identify the analytic region containing the first mismatch. A value mismatch at a knot suggests branch selection. A value match with gradient mismatch suggests a non-differentiable expression or misplaced detach. If only the training smoke fails, the analytic formula has passed and loss schema resolution is the next boundary to inspect.

*q.* *loss-schema-equivalence.* Diff resolved configurations before diffing epochs. The first differing leaf usually identifies an inherited default or wrong nesting level. If resolved records agree but epochs differ, look for a code path that still reads raw configuration after resolution.

*r.* *berhu-anneal.* Print the coefficient on the last hold epoch, first anneal epoch and final epoch. This distinguishes direction, boundary and endpoint errors. Then verify the loss uses that coefficient. A correct banner with unchanged analytic values means the schedule is disconnected.

*s.* *ema-anneal.* Inspect whether an averaged copy was updated during hold and which weights evaluation used at the live point. If schedule values are correct but metrics match raw weights, inspect the evaluation model selection. The schedule-value check has already cleared the horizon formula.

*t.* *param-window-cuts.* Resolve parameter names to column indices and independently recount each bound. A disagreement before combination indicates one formula or column. A disagreement only after combination indi-

cates mask composition. Confirm the absolute row count is applied to the surviving pool.

*u.* `triangle-shading`. List artist types, panel coordinates and bounds. A correct count with wrong panels is an axis-map failure. A missing fill with correct numeric masks is a plot ownership failure and does not invalidate the staging cut itself.

*v.* `joint-training`. Print trainable names and optimizer membership at phase entry. A parameter can have `requires_grad=True` yet remain absent from the optimizer. After one batch, compare trunk and head gradients and actual parameter deltas. Treat epoch loss as supporting evidence.

*w.* `head-activation-pin`. Compare the resolved head factory, instantiated class names and activation parameter count. If the banner is correct but count is not, the pin was recorded without construction. If the illegal configuration trains, the validator was bypassed or ran after allocation.

*x.* `relu-tanh-norm`. Inspect every residual block. The model's first norm cannot establish the remaining blocks. Shared object identity across layers means a factory returned one instance. For a finite but flat tanh run, inspect input range and normalization gains before assuming optimizer failure.

*y.* `weight-decay-census`. Report missing and unexpected parameter names with owning module types. A rank-based pattern appears as activation or normalization tensors in the unexpected set. A custom linear owner in the missing set means the role registry is incomplete.

*z.* `npce-training`. First identify residual versus ratio and the training-size point. If only the second point fails, compare PCE coefficient identities and selected term sets between points. If rebuild fails, inspect persisted domain, multi-indices, coefficients and combination law separately.

*aa.* `finetune-identity`. Compare source and new named parameter orders, then locate the first unequal stage: encoded input, first projection, network output or decode. An output difference with equal network coordinates implicates geometry. A difference at the first projection implicates extension or weight surgery.

*bb.* `transfer-identity`. Print base input slices, base-space output, correction and final composition in that order. This exposes double encoding and wrong-space composition. For an unexpected error message, verify the fixture is invalid only in the dimension under test.

*cc.* `scalar-identity`. Read named outputs beside their columns. Equal width is not enough. For a constant-column failure, inspect the stored dtype after scale construction. A positive float64 scale may underflow to zero in float32.

*dd.* `cmb-identity`. Identify spectrum, multipole and representation before examining tolerance. For covariance failures, compare raw fixture arrays, independent per-band accumulation and production weights. For amplitude failures, divide the reported metric by the expected physical factor to expose an accidental multiplicative law.

*ee.* `bsn-identity`. Classify the red point as trained window, exact boundary or desert. A finite desert return is a domain-policy failure even if monotonic. A trained-window numeric mismatch should be traced through output law, interpolation and distance conversion in that order.

*ff.* `mps-identity`. Separate coordinate, law, moment, assembly and temporary-file legs. For moment failure, record chunk order and compare against the stored float32 payload in its stored representation order. Precast data uses the wrong reference. For cleanup failure, list live train and validation slots and which restaging event should have unlinked each path.

*gg.* `geo-paths`. Inspect the class marker in a newly written state before chasing imports. If the marker is current but the census is red, a live source still depends on a retired home. If a legacy path loads, locate the compatibility shim that made the required loud refusal impossible.

*hh.* `save-rebuild-drift`. Compare predictions at four boundaries: pre-save eager, rebuilt eager, rebuilt under drifted defaults and family public predictor. Then compare state keys, shapes and resolved construction facts. Non-strict loading is never a repair for an unexplained missing key.

*ii.* `cobaya-adapter`. Print named sampled values in artifact order and compare the predictor result before adapter assembly. If direct parity passes but repeated sampling fails, inspect per-call state reset and provider verdict order. If parity itself fails, inspect units, axes and decode before MCMC.

*jj.* `finetune-smoke`. Treat failed epoch-zero parity as a construction failure and stop before interpreting later improvement. If parity passes but save/rebuild fails, compare fine-tune provenance and inherited architecture facts. If training does not improve, inspect learning rate and trainable census.

*kk.* `transfer-smoke`. Verify the base digest and family identity, then epoch-zero parity, then correction learning. A missing embedded base is a publication failure even when the run succeeds locally with the external source still present.

*ll.* `scalar-smoke`. Evaluate the analytic formula at the exact off-center input printed in the log. If direct prediction agrees but Cobaya readback does not, inspect automatic provides and state insertion. If both return the center, inspect input ordering and encoding.

*mm.* `cmb-smoke`. Follow generator, covariance, training, rebuild, adapter and diagnostics markers in order. The first absent marker locates the interface boundary. Use `cmb-identity` to judge a normalization disagreement. The smoke's generated covariance serves its connectivity fixture.

*nn.* `bsn-smoke`. Check the stale-cache tripwire before comparing CAMB values. A tripwire failure invali-

dates every later numeric agreement. For a getter mismatch, compare requested redshift sequence with artifact axes before inspecting interpolation.

*oo.* `mps-smoke`. Confirm the stored $z$ and $k$ sequences on both artifacts and the request before comparing flattened arrays. A transpose can preserve shape and summary statistics. If coordinates agree, inspect lawspace assembly and only then the real-provider tolerance.

### 18. Worked example: a finite-contract gate

Consider the claim that validation must refuse a finite negative $\Delta\chi^2$. First identify the public boundary. The relevant function is `eval_val`, because it creates per-row validation scores and ranks epochs. The gate calls `eval_val` directly and therefore proves that the public evaluator uses the predicate.

The fixture uses a small test double returning a score vector with a negative value well outside the derived roundoff band. All other inputs are finite and validly shaped. This isolates one invalid property: an exception cannot be misattributed to NaN input, shape mismatch or an absent model parameter.

The expected result is a typed exception naming the validation side, affected row count, minimum and allowed band. The gate requires its own typed classification before low-level square-root work. A warning afterward arrives too late.

Next comes a near-zero valid control. A small positive-definite covariance produces a tiny negative roundoff under finite precision. The shared predicate normalizes a value inside the derived band to exact zero. This valid control proves that the repaired gate still accepts legitimate roundoff.

The mutation restores the retired width-squared band. At realistic output width, the chosen negative score then falls inside the wrong band and becomes zero. The mutation must make the gate red. Finally, the same predicate is exercised through training reduction and source diagnostics. One helper owns the rule, while integration legs prove every public producer calls it:

$$\begin{aligned}\text{public boundary} &\to \text{red fixture} \to \text{typed verdict}\\ &\to \text{valid control} \to \text{mutation}\\ &\to \text{boundary census}.\end{aligned}$$

### 19. Worked example: identity and smoke for the CMB family

The CMB family needs both an end-to-end smoke and an identity check. The smoke can run CAMB, write spectra, construct a covariance, train and serve $C_\ell$. If every stage uses the same wrong lensing convention, the path can be internally consistent and scientifically wrong.

The identity fixture makes raw and scaled lensing spectra different. Let the fake raw spectrum be nonconstant $C_L^{\phi\phi}$. Independently form

$$\widetilde{C}_L^{\phi\phi} = \frac{[L(L+1)]^2}{2\pi} C_L^{\phi\phi}.$$

The fake returns each only under its proper request. Production receives the raw array. A separate explicit loop computes the expected band contribution. Agreement establishes the correct arm.

For catch power, feed the scaled array into the raw slot without changing the reference. The band weight must miss by a large recorded margin. This is stronger than asserting that the arrays differ: it proves the final contraction is sensitive to the convention.

The real-CAMB smoke checks that the generator requests available keys, lengths and multipole sidecars agree, training consumes the covariance, the artifact answers Cobaya's requested sequence and the non-Gaussian configuration executes its declared path.

The logs meet through persisted provenance. The smoke asserts which path ran. The identity asserts that this path has the right mathematics. Each log uses an independent numeric truth source.

### 20. Worked example: artifact drift

An artifact combines tensors with instructions for interpreting them. A drift test changes an ambient code default after writing the artifact. Suppose a model was saved with activation $H$, three gates and affine normalization. The check records an off-center prediction $y_{\text{before}}$.

The source default is monkeypatched to another allowed setting. Public rebuild must read the persisted resolved configuration and produce $y_{\text{after}}$. Because this fixture requires no new arithmetic, the comparison can be bitwise:

$$y_{\text{after}} = y_{\text{before}}.$$

The off-center input matters. At zero input or identity initialization, several architectures coincide and a drifted build can pass accidentally. The check also inspects class, parameter names, output geometry and head layout. Prediction equality for one row can be coincidental. State identity plus several discriminating predictions localizes failure.

An old artifact missing a required construction fact is refused with a migration instruction. Guessing from current defaults makes the file load today and change meaning tomorrow. Loud refusal is correct when reconstruction cannot be proven.

### 21. How to add a gate while preventing a false green

Start with one sentence that can be false. For example: *the rebuilt predictor returns the same named spectrum on*

*the same multipole sequence.* Avoid a broad goal such as *test CMB*. A narrow claim determines the public boundary, fixture and verdict.

Then follow this construction order.

1. Name the production owner and call its public function. If a private helper also needs a unit check, retain the public integration leg.
2. Build the smallest fixture preserving the distinction. Use nonconstant, nonsymmetric values when order matters.
3. Derive the expected answer independently. Write down units, axis, dtype, device and shape before calculating it.
4. Add a valid edge control matched to the claim: exact zero, one row, final short batch or an ill-conditioned positive-definite matrix.
5. Add a mutation or deliberately wrong form and verify the unchanged assertion fails it.
6. Give every failure a typed or named verdict. A red leg requires the warning and a nonzero exit.
7. Declare hardware and external capabilities. Do not hide an unavailable required lane inside a broad exception handler.
8. Register the gate with home documentation and actual artifact dependencies.
9. Run the check directly and through the board. Direct success plus board failure usually indicates deployment, path, capture or capability metadata.
10. Temporarily break the intended production line and observe the board turn red before accepting the new gate.

### 22. Common false-green patterns

*a. Expected and actual share one helper.* The test repeats the defect. Replace the expected side with explicit algebra, a closed-form solution or a differently structured reference.

*b. The fixture erases the distinction.* Zeros, constants, identity matrices and centered cosmologies make wrong paths agree. Keep them as edge controls and include a discriminating fixture.

*c. Any exception counts as success.* An always-raising implementation passes. Require the documented exception type and message facts, then include a healthy in-range control.

*d. A banner substitutes for behavior.* The resolver can print a setting that construction ignores. Pair the banner with parameter counts, state readback, a numerical transition or a changed weight.

*e. Only aggregates are compared.* Permuted or transposed rows can preserve a mean. Compare per-row and per-coordinate arrays before reducing.

*f. A skip exits zero.* The board records a pass without evidence. Use a distinct skip status that cannot contribute to a full-board denominator.

*g. Tolerance follows the failure.* The defect defines its own acceptance. Derive tolerance from dtype, operation count and valid controls before looking at the red value.

*h. The smoke is its own reference.* Internal consistency replaces truth. Pair the smoke with an identity or an independent external implementation.

### 23. A compact evidence checklist

For each raw log, a reviewer checks:

1. Confirm that the header identifies the exact commit and watched surface.
2. Confirm that every promised leg prints a start and verdict.
3. Find the production public boundary in the execution record.
4. Record actual, expected, tolerance, dtype, device, shape and coordinates.
5. Confirm that the valid control is close for the stated reason.
6. Confirm that the mutation fails through the intended assertion.
7. Exclude skips from the pass count.
8. Use artifact or adapter readback to show that the consumer sees the same resolved fact.
9. Record a thin margin to threshold explicitly.
10. Reproduce the evidence without untracked workstation state.

### 24. Reading one registry entry field by field

A `Gate` entry is data describing execution. Its `id` is the stable command-line name and log-file stem. Changing an identifier breaks resume continuity and external run instructions, so titles may improve while identifiers remain stable.

The `title` is human-facing. It states the claim in ordinary language. The `tier` controls grouping and scheduling. Executed checks determine pass semantics. The `home` points to the maintained design note that owns the acceptance contract. The optional map text gives a reviewer a more precise route into that note, but executable assertions remain the verdict.

The `run` field holds a function object and registers work for later. The runner calls it with the context for the current log, configuration, capabilities and dry-run state. This delayed call is why constructing the board at import time need not launch tests.

`needs` is a tuple of capability names. A tuple is an immutable ordered Python sequence. The runner iterates it before the gate body. `deps` is a separate tuple of gate identifiers whose accepted products or boundaries are required. Capability availability and scientific dependency are separate relationships.

The optional golden-worktree commit identifies a comparison tree for a no-change claim. Its expected-answer scope is limited to unchanged behavior. If a feature is new, an older commit lacks the behavior and cannot establish its mathematics. The gate uses analytic, mutation or real-provider evidence for that case.

Inside the body, `ctx.run_check(path)` launches a repository check script and returns its exit status and captured output as ordinary values. The body passes the status into `ctx.expect`. `ctx.expect` records a named check and changes the gate verdict when its boolean condition is false. The label names the scientific claim represented by the Python comparison.

Dry-run mode follows the same registry and dependency traversal but prints commands without executing them. A gate body must not interpret absent dry-run output as failed science. Conversely, dry-run success is never stored as a gate pass because no assertion values were produced.

### 25. Choosing the right evidence form

*a. Use exact identity when no arithmetic should occur.* Examples include loading the same tensor bytes, an off mode that bypasses a branch and a zero correction whose composition is algebraically the base. Exact comparison gives the narrowest contract and catches accidental casts or reordered operations.

*b. Use a tolerance when the correct path performs floating-point work.* The tolerance is derived from dtype and operation structure. Record absolute and relative parts explicitly. Compare in the dtype in which the verdict is defined. Upcasting after a float32 computation cannot recover lost information.

*c. Use a golden run for a promised no-op migration.* Schema rearrangement and disabled optional features are good candidates. Pin the base commit and select stable, science-bearing output lines. Do not use a golden run to freeze a known defect or to replace a new feature's reference.

*d. Use an analytic fixture for a mathematical law.* Closed-form scalar outputs, polynomial surfaces, diagonal covariance and simple distance relations make good fixtures. Exercise off-center and nonconstant values so competing formulas do not coincide.

*e. Use a fake for an external protocol.* A fake can record requested keys, enforce call order and return generation-tagged arrays. It proves the code handles the protocol it was given. Follow with a real-provider smoke to show the protocol matches the installed collaborator.

*f. Use a smoke for production connectivity and liveness.* A smoke should cross the same public entry points used in production and beat a baseline that a dead component cannot beat. Reduce rows and epochs to keep it short. Retain the component under test.

*g. Use mutation when a plausible wrong path also looks reasonable.* Axis relabeling, stale cache, raw/scaled convention, tolerance inflation and configuration printing without construction can all yield finite outputs. A targeted mutation proves which assertion distinguishes them.

### 26. Why gate names pair identity and smoke

The family pairs are an intentional reading aid: `scalar-identity`/`scalar-smoke`, `cmb-identity`/`cmb-smoke`, `bsn-identity`/`bsn-smoke` and `mps-identity`/`mps-smoke`. The shared prefix says the gates refer to one physical family. The suffix says which evidence layer they own.

Warm-start and transfer use the same pattern. Their identity gates establish epoch-zero composition and error paths without external training cost. Their smokes consume a saved artifact and prove improvement, provenance and final rebuild. If the identity is red, running the smoke spends accelerator time on an undefined starting function. If identity is green but smoke is red, the failure lies in training, persistence, deployment or the real data path while the core algebra has already passed its isolated check.

Some cross-cutting features use an off-identity/on-smoke pair. EMA first proves that absence preserves old behavior, then proves the new averaged state is live. The name states the two distinct claims that the feature needs.

The board order places cheap, pure identities before dependent expensive smokes where practical. This is fail-fast scheduling: discover a local algebra or schema failure before allocating a workstation for an end-to-end run. Order never allows a smoke pass to override a failed identity. Both remain separate verdicts in the final board.

### 27. From a red log to a bounded repair

A repair begins at the first failed invariant. The final traceback supplies path evidence. Reproduce the smallest failing gate directly. Preserve the original raw log, then instrument values without changing the acceptance predicate. Once the mechanism is understood, state a bounded implementation contract and add the counterexample that current code gets wrong.

The implementer changes the production owner and the gate together only when the existing gate lacks the required distinction. A gate must not be relaxed to match the implementation. If a previously accepted behavior was itself wrong, the record states that the acceptance is reopened and replaces its reference with independently justified truth.

After the repair, run the direct gate, its mutation arm, dependent family identity and the relevant smoke. Then run the complete board when the changed surface can affect other families. Record the exact source commit beside the raw logs. A reviewer can then reconstruct this chain:

$$\begin{aligned}\text{counterexample} &\to \text{root cause}\\ &\to \text{bounded change}\\ &\to \text{discriminating leg}\\ &\to \text{regression evidence.}\end{aligned}$$

This discipline keeps a red result useful. The goal is a true scientific claim with executable evidence that catches the same failure class later.

### 28. Mapping gates to the pipeline boundaries

The board can also be read horizontally, by the boundary each gate protects. At the configuration boundary, `single-phase-demotion`, `head-scheduler-override`, `loss-schema-equivalence`, `head-activation-pin`, `relu-tanh-norm`, `cli-strict` and `family-first` prove that commands and text become the intended constructible objects for the intended physical family. Their common risk is a value that is accepted or printed but does not control construction.

At the generation and data-selection boundary, `generator-seed`, `stage-ram`, `param-window-cuts`, `triangle-shading` and `production-diagnostic` connect reproducible cosmology draws, physical support, staged rows, resource policy and the human coverage report. The plot gate checks the domain shown in the report. The selection gates check the enforced row set.

At the objective boundary, `berhu-loss`, `berhu-anneal`, `finite-contract` and `eval-batch-invariance` protect the meaning of a score. `diagnostics-domain` extends the same boundary to post-training estimators. They distinguish the per-row physical metric from its training transform and they ensure batching, invalid values or schedule position cannot redefine which epoch looks best.

At the optimizer-state boundary, `ema-off-identity`, `ema-smoke`, `ema-anneal`, `weight-decay-census`, `joint-training` and `head-scheduler-override` establish which parameters move, which auxiliary state advances and which metric controls the learning rate. This group covers state transitions and tensor formulas.

At the representation boundary, the scalar, CMB, background and matter-power identity gates protect names, axes, units, geometry laws and family-specific assembly. Their smoke partners then carry the verified representation into the real physics provider. `geo-paths` protects the Python class identity that reconstructs those representations.

At the reuse boundary, NPCE, fine-tune and transfer gates establish how an existing function participates in a new model. Identity gates prove the initial composition. Smokes prove that the composed model trains and publishes. Their central invariant is that reuse starts from a known function whose identity is established before its tensors are loaded.

At the publication boundary, `save-rebuild-drift` proves that the saved recipe owns reconstruction, `artifact-readback` protects typed metadata and `cobaya-adapter` proves that the external consumer sees the same function. Family smokes add repeated lifecycle calls. These gates close the final gap between an in-memory result and a theory product used by inference.

Finally, `board-selftest` protects the evidence-control boundary itself: selection, dependency, status, digest, log and note-map machinery. Scientific truth comes from the family gates. The self-test prevents the runner from reporting a claim as executed when it was empty, skipped, stale or interrupted.

This horizontal reading reveals coverage gaps. A new component that changes configuration, representation, training state and publication needs evidence at each affected boundary. One end-to-end smoke covers its executed path, while the earlier boundaries need their own claims. A construction identity and a public-driver reachability check also cover separate questions. The gate plan is complete when every changed arrow in the ownership chain has a named executable claim.

### 29. Running the board and preserving the evidence

The workstation operator begins in an environment where Cocoa, PyTorch and the relevant external packages are importable. The first command is

```
python ai/gates/run_board.py --check
```

This performs preflight without launching GPU work. A clean preflight states which repository commit, deployment configuration, watched paths, imports and data roots will govern the run. If any fact is wrong, fix the deployment and run preflight again. Do not treat an intended different tree as close enough.

Next,

```
python ai/gates/run_board.py --dry-run
```

prints the selected order, dependencies, capabilities and commands. Dry run is the place to catch a wrong YAML path or accidentally broad selection. Because it produces no scientific results, it never writes pass verdicts.

The full command

```
python ai/gates/run_board.py
```

streams each command into its per-gate raw log. The terminal output is useful for monitoring, but the log file is the durable record. If the process is interrupted, completed matching passes remain eligible for resume and the in-flight gate runs again.

For a bounded investigation, `-gate` selects named gates and `-from` selects the suffix beginning at one gate. `-tier` selects a registry tier. Force-rerun applies only to explicitly named matching gates unless the all-selected form is requested. Before trusting the plan, the operator reads the printed selected count and identifiers. Zero selected gates returns an error. It can never pass as a no-op.

After the run, begin with the summary to locate red or skipped entries, then open every corresponding raw log. For a full acceptance run, sample green logs as well, especially thin-margin and hardware-specific gates. Confirm the last gate verdict, but also confirm every advertised leg appears above it. A gate body that returned early could otherwise produce an incomplete log whose last executed check happened to be green.

Logs are committed only after the operator verifies that they name the intended commit and contain no unrelated workstation secrets. The log commit adds evidence metadata while leaving the tested source commit fixed. The evidence note names both commits. If source changes after the run, the old logs remain historical evidence and a new run is required for the new executable identity.

When a gate needs CUDA or another workstation-only capability, the repository maintainer supplies the check under `ai/gates/checks`, registers it in the board and gives the operator an exact selection command. The operator is not asked to paste an untracked Python snippet into a GPU shell. Keeping the check in the repository makes its source part of the watched evidence surface and lets later board runs repeat it.

The final handoff reports counts only after excluding skips and unexecuted gates. On the board described by this appendix, a phrase such as 40/40 means forty registered required gates executed and passed on the recorded surface. If the environment lacks a required lane, the honest result is fewer certified claims plus an explicit outstanding capability. A smaller denominator would falsely present a complete board.

### 30. Scope of a full green board

A green board means that every registered executable claim passed on the recorded source tree, configuration, dependencies and hardware. Unregistered states remain outside that evidence. The board therefore persists the Git commit and should bind resume records to a digest of the complete executable surface: production modules, gate bodies, fixtures and relevant configuration.

The strongest pattern is paired evidence:

1. an identity fixture proves an exact mathematical or serialization statement.
2. a mutation or deliberately wrong form proves the assertion can fail.
3. a smoke run proves the real external components reach the verified boundary.
4. artifact readback proves the published object carries the facts the executed path used.

When only one layer is available, the log should say exactly what remains unproven. A CPU identity cannot certify CUDA compilation. A CUDA smoke cannot establish an independent analytic truth if its reference repeats the production helper. A PDF existing cannot prove its plotted coordinates. The board is useful because those claims stay separate and named.

## Appendix B: A file-by-file route for studying the code

This appendix is a reading itinerary arranged in execution order. Each file is placed after the concepts needed to understand it and before the files that consume it. The reader should keep one example YAML, one parameter table and one small artifact in view while following the route.

### 1. How to use the route

For each file, complete four checks before moving on:

1. Identify the object entering the file's public function.
2. Identify the object leaving, including shape, dtype, device and ownership.
3. Name the physical or lifecycle invariant owned by the file.
4. Name the later file that consumes the returned object.

Do not begin by reading every helper from top to bottom. Locate the public entry named below, follow one call path and return for variants after the baseline path is clear.

#### *a. Prepare one concrete reading packet*

Keep four artifacts open beside the source code:

1. one shipped YAML recipe, because it supplies the public configuration for the study.

2. its `.paramnames` file, because that file assigns scientific names to sampled and derived parameter columns while excluding chain-weight/log-posterior bookkeeping.

3. the first two rows of the parameter table and data-vector dump, because a named number is easier to trace than an abstract array.

4. one small saved artifact produced by the same family, because the reader can compare training-time objects with their persisted recipe.

Here `.paramnames` is a plain-text sidecar used by CosmoLike/CoCoA conventions: each non-comment line declares a parameter name and may carry a display label or status fields. The numerical parameter table itself contains only numbers, so the sidecar supplies column identity.

Select one row number, for example original disk row 17 and write it at the top of a worksheet. Do not silently replace it with "some row" later. If staging turns row 17 into local row 3, record both coordinates. If batching later places local row 3 at minibatch position 1, record that third coordinate too. The numerical sample is unchanged. Only the coordinate system used to address it has changed.

#### *b. Read every file in five passes*

The five passes below prevent a large module from becoming a wall of Python.

1. Locate imports, public classes/functions, their arguments and return values during the boundary pass. Defer helper bodies.

2. Annotate every array crossing the public boundary with shape, dtype, device and coordinate meaning during the shape pass.

3. Mark which object validates, transforms, stores, releases or persists each value during the ownership pass. A caller that passes a value onward has a different responsibility from its scientific owner.

4. Follow one invalid value from the public boundary to the first typed exception during the failure pass. Confirm that refusal precedes the expensive model or provider.

5. Find the identity, smoke or readback gate that executes the same public boundary during the evidence pass. Write what the gate proves and what remains outside its fixture.

On the first pass, defer a private helper deliberately. A reader first learns why the helper exists and which contract calls it. The second visit can then answer how it implements that bounded responsibility.

### 2. Stage 0: orientation before Python

*a. 1. Root `README.md`.* Read the pipeline diagram, one-run command, YAML anatomy and the definitions of data vector, whitening and chi-square. Stop when the four path concepts (shell `ROOTDIR` plus the three path flags) have distinct meanings and when the required `data`/`train_args` blocks are distinguishable from optional top-level `pce`, `transfer` and `sweep` blocks.

*b. 2. `emulator/README.md`.* Read the layout and five-family table. Use its function census as an index. Begin the explanation with the physical family. Stop when each family can be mapped to one geometry, one loss and one Cobaya adapter.

*c. 3. One file under `example_yamls/`.* Choose the simplest `resmlp` training example for the family of interest. Trace indentation and write the dotted path of each model/loss control. This concrete recipe will anchor later configuration code.

### 3. Stage 1: the shortest complete training path

Stage 1 contains many files because it crosses the whole lifetime of one model. They divide into four smaller reading packets:

*a. 4. `emulator/cocoa.py`.* Start with the path resolver. Follow how `-root`, `-fileroot` and `-yaml` become concrete locations. The file owns project layout. Numerical cosmology belongs to later scientific modules. Next read the experiment constructor that consumes those paths.

*b. 5. `emulator/experiment.py`, first visit.* Read only `EmulatorExperiment.from_yaml` and `EmulatorExperiment.from_config`. Follow family dispatch until the cosmic-shear or selected-family branch has named its source files. Defer the large validators and grid2d staging details. Stop when the order *validate, stage, build geometry, build specs, train* is visible.

*c. 6. `emulator/data_staging.py`.* Read `load_source`, `stage_source` and the source-dictionary docstrings. Draw resident and memmap cases separately. Track `idx` and `dump_rows`. One is the coordinate used to index the staged object and the other retains original disk-row identity for sibling files. Then read `phys_cut_idx`, streamed statistics and scalar named-column loading.

*d. 7. `emulator/geometries/parameter.py`.* Read `ParamGeometry.from_covmat`, `encode`, `decode` and `state`. Write the center, rotation and scale operations as equations. Confirm which arrays are NumPy during fitting and which become device tensors during loading. Defer log and amplitude-factor subclasses until the baseline round trip is clear.

*e. 8. `emulator/geometries/output.py`.* Read `DataVectorGeometry` in the order squeeze, center, whiten, unwhiten, unsqueeze. Identify the mask and covariance as physical owners. Then

| Object or name | Shape | Dtype | Device | Owner/lifetime | Next consumer |
|---|---|---|---|---|---|
| Parameter table `C` | $(N, P)$ | NumPy float32 | CPU/file | `load_source`. Full source lifetime | parameter geometry or batch gather |
| Selected coordinates `idx` | $(N_{\text{sel}},)$ | NumPy integer | CPU | staging. Valid while source dictionary lives | loader's row gather |
| Encoded batch `X` | $(B, P_{\text{enc}})$ | torch float32 | selected device | parameter geometry. One minibatch | `model(X)` |
| Encoded target `T` | $(B, K)$ | torch float32 | selected device | output geometry/loss. One minibatch | residual and loss |
| Physical residual `R` | $(B, K)$ | torch compute dtype | selected device | loss wrapper. One forward evaluation | covariance contraction |
| Saved recipe | typed HDF5 groups | bytes plus typed arrays | disk | `save_emulator`. Artifact lifetime | rebuild and adapter |

TABLE XXVIII. A worksheet to fill while reading one executable path. "Owner" is the function or object responsible for creating and validating a value. "lifetime" states how long that value remains valid. $N_{\text{sel}}$ is the number of rows selected for one training or validation set. Filling the worksheet exposes hidden transitions: a dtype cast, a device copy, a disk-row to local-row remap or a temporary object that is incorrectly reused.

| Packet | Files | Fact established before continuing |
|---|---|---|
| Control path | `cocoa.py`, first visit to `experiment.py` | Validated recipe and selected family branch |
| Data and coordinates | `data_staging.py`, `geometries/parameter.py`, `geometries/output.py`, `losses/core.py` | Rows and physical coordinates used for the model input, target and scalar objective |
| Network and optimization | `designs/blocks.py`, `designs/plain.py`, `activations.py`, `batching.py`, `training.py` | Registered parameters updated during one minibatch and data location |
| Persistence and use | `results.py`, `inference.py` | Facts written, reconstructed and exposed as a physical prediction |

TABLE XXIX. Stage-1 reading packets. Establish the fact in the right column before entering the next packet. This ordering follows the executed ownership chain. Alphabetical file names and module length play no role.

read `DiagonalGeometry`, `BlockDiagonalGeometry` and `build_shear_angle_map` as alternative coordinate structures.

*f. 9. emulator/losses/core.py.* Read `CosmolikeChi2.chi2`, `encode`, `decode` and `_reduce`. Separate the physical per-row statistic from the training transform. Then read `anneal_value` and `make_chi2`. Stop when a loss object can be described as geometry plus composition plus reduction.

*g. 10. emulator/designs/blocks.py.* Read `Affine`, normalization construction and one `ResBlock` forward pass. Name every registered parameter and buffer. Then read `BinLinear`, `TRFBlock` and `FiLMGenerator`. These are components used by structured heads. Complete emulators assemble them elsewhere.

*h. 11. emulator/designs/plain.py.* Begin with `ResMLP`: entry projection, residual-block list, exit projection and final affine map. Follow tensor shapes through `forward`. Only then read `ResCNN` and `ResTRF`. Focus on scatter, validity mask, correction, gate and inverse gather.

*i. 12. emulator/activations.py.* Now the `act(dim)` factory has context. Read baseline $H$, then multi-gate, power and gated-power forwards. For multi-gate, write the shape created by `unsqueeze(-2)` and the axis removed by `sum(-2)`. Finish with `make_activation`. Its factories create fresh instances at each activation site.

*j. 13. emulator/batching.py.* Read memory estimators before loader construction. Follow `_build_loaders_one` once for device-resident, host-stream and memmap-stream regimes. The returned closures have the same interface while retaining different storage objects.

*k. 14. emulator/training.py, first visit.* Read `make_model`, `make_optimizer` and `make_scheduler`. Note which computed values the factories inject. Then follow one epoch through `training_loop_batched`: clear gradients, load, forward, loss, finite checks, backward, unscale/clip, step, anchor, EMA and metric accumulation. Finish with `eval_val` and best-state selection.

*l. 15. emulator/results.py.* Read `save_emulator` beside `rebuild_emulator`. For ev-

ery written field, find its readback. Distinguish weight state from the constructor recipe and geometry state. Two successful writes establish two files. Transactional pair integrity also requires digest binding, which the current pair lacks.

*m. 16.* *emulator/inference.py.* Read `EmulatorPredictor.__init__`, then one `predict` call. Verify that saved parameter names define input order and that the training-side decoder is reused. The baseline path is now complete from YAML to physical prediction.

*a. Carry original disk row 3 through Stage 1*

Return to the running example's seeded order $[3, 0, 2]$. The table below follows original disk row 3 consistently through every file.

### 4. Stage 2: learn the other geometry families

*a. 17.* *emulator/geometries/scalar.py.* Study named output columns, per-output center/scale, batch-by-output shape and the constant-column guard. Pair it immediately with `emulator/losses/scalar.py`, whose `ScalarChi2` also wraps the grid families because their physical law lives inside their geometry.

*b. 18.* *emulator/geometries/cmb.py.* Track spectrum name, multipole sequence, units, fiducial values and per-multipole scales. Read `attach_head_coords` to see how one spectrum becomes a structured-head coordinate block. Next read `emulator/losses/cmb.py`: amplitude-law registry, factored target, roughness and the plain/factored loss factory.

*c. 19.* *emulator/geometries/grid.py.* Read the target-law registry before `GridGeometry`. Follow `log_offset` through encode and decode and identify the persisted redshift/quantity/unit facts. Then read `emulator/background.py` in the order cumulative integration, comoving distance and interpolators.

*d. 20.* *emulator/geometries/grid2d.py.* Write down the flattening rule $j = i_z N_k + i_k$. Follow law-space standardization, stored thinned $k$, constant-mask handling and head coordinates. Then read `emulator/syren_base.py`, which maps resolved cosmological parameters into the analytic base consumed by generator and adapter.

*e. 21.* *emulator/analytics.py.* Read this optional cosmic-shear preprocessing only after ordinary geometry and loss are understood. Track exactly which broadband analytic factor is divided out and where it is multiplied back.

### 5. Stage 3: design variants and warm starts

*a. 22.* *emulator/designs/ia.py.* Start with the template count and output shape. Verify that intrinsic-alignment amplitudes do not enter the template network. Pair the design with `emulator/losses/ia.py`. Read NLA/TATT coefficient builders before the template-combine loss.

*b. 23.* *emulator/designs/pce.py.* Read `pce_multi_index`, `pce_design` and `select_lars_loo` in that order. Then follow `PCEEmulator.from_training`: box map, Legendre matrix, target SVD, per-mode selection and frozen buffers. Finally read `emulator/losses/pce.py` to see cached base output packed beside truth and combined with the neural refiner.

*c. 24.* *emulator/warmstart.py.* Read source resolution and validation first. Then read recipe reconstruction, input-geometry extension, weight transfer, output pinning and epoch-zero parity. Read anchor-mask machinery as a mapping of named source/new parameters. The public fine-tune anchor key is refused by the current validator, so examples using that key stop before training.

*d. 25.* *emulator/losses/transfer.py.* Follow frozen base, correction, composition form and composition space. Read cosmic-shear and diagonal-family variants separately. Identify which tensor blocks the packed target contains and which owner defines the split.

*e. 26.* *emulator/experiment.py*, *second visit.* Return for `build_geometry`, `build_specs`, fine-tune/transfer branches, grid2d law staging, two-phase resolution and every family validator. The earlier files now make this large orchestration module readable.

### 6. Stage 4: campaigns, reports and publication

*a. 27.* *emulator/family_drivers.py.* Read sweep-block parsing and dotted-path assignment. Then inspect one root-level train driver, one $N_{\text{train}}$ sweep, one single-knob sweep and the Optuna driver. Family-specific driver files are thin wrappers. Verify what they pin before reading duplicated command-line setup.

*b. 28.* *emulator/scheduling.py.* Read assignment functions, worker entry/exit protocol, then VRAM-token packing. Distinguish job scheduling from model-distributed training: one job still owns one complete model and device.

*c. 29.* *emulator/diagnostics.py.* Start with coverage, local-linear floor and hard-direction functions. For each, list return keys and undefined-data policy. Then read the CMB, scalar, grid and grid2d diagnostics that produce physical arrays for plotting.

*d. 30.* *emulator/plotting.py.* Read public plotting functions after diagnostics so data preparation and rendering do not blur together. Trace one returned diagnostic dictionary into one page builder and the multipage

| Boundary | Name used there | Object/shape | What the reader verifies |
|---|---|---|---|
| `cocoa.py` → `experiment.py` | no row loaded yet | resolved path strings and validated recipe | Paths name the intended source. No array or GPU allocation has occurred. |
| `data_staging.py`: selection | original disk row 3 | one member of `idx=[3,0,2]` | The row remains first in seeded training order. |
| `stage_source`: resident branch | local row 2 | compact storage rows are sorted originals $[0, 2, 3]$. `idx_src=[2,0,1]` | `searchsorted` maps original 3 to compact 2. Plain `arange` would lose seeded order. |
| `stage_source`: disk branch | global row 3 | memmap plus `idx_src=[3,0,2]` | No local remap is invented. Both regimes emit the same cosmology sequence. |
| `parameter.py` | physical parameter row | $(1, P)$ then $(1, P_{\mathrm{enc}})$ | Center/rotation/scale come from the fitted training geometry. Dtype/device change is named. |
| `output.py` and loss | physical target row | $(1, D)$ gathered to $(1, K)$ and encoded | The mask, mean, covariance and inverse map retain their owners. |
| `batching.py` | one position in `rows` | device tensors $(B, P_{\mathrm{enc}})$ and $(B, K)$ | Loader closures preserve order while changing storage regime and device. |
| `training.py` | minibatch position $b$ | one row of prediction and per-row $\Delta\chi^2_b$ | The row is reduced only after its physical score and finite verdict exist. |
| `results.py` | no training-row coordinate | weights plus typed recipe | Artifacts persist the function and geometry. A transient minibatch pointer expires with the batch. |
| `inference.py` | named cosmology mapping | one reconstructed physical prediction | Saved parameter order, model and decoder reproduce the same function without access to the training source dictionary. |

TABLE XXX. One row through the shortest complete path. Original row identity, compact local storage coordinate and minibatch position are different addresses for the same cosmology. The table also marks the point at which row identity ceases to be persisted: the final artifact represents a function, while training rows remain in source storage.

writer.

*e. 31.* ***`emulator/results.py`****, second visit.* Return for learning-curve and sweep tables, histories, provenance, PCE/transfer groups and family flags. Compare every persisted fact with the campaign or adapter that later relies on it.

### 7. Stage 5: generation and external inference

*a. 32. Shared generator core.* Open `compute_data_vectors/generator_core.py`. Read command parsing, parameter sampling, master/worker distribution, checkpoint load/save and failure masks. Identify which checkpoint members and failure verdicts are revalidated before treating a dump as training-ready.

*b. 33. Family generator files.* Read `dataset_generator_lensing.py`, `dataset_generator_cmb.py`, `dataset_generator_background.py` and `dataset_generator_mps.py` as thin physics adapters over the shared core. For each, record provider requirements, payload shape, dtype, axis sidecars and output file names. Read `compute_cmb_covariance.py` after the CMB generator. It produces the separate scale/covariance input consumed by CMB geometry.

*c. 34. Cobaya theory adapters.* Read `emul_cosmic_shear.py` first for the simplest predictor wrapper. Then read `emul_scalars.py`, `emul_cmb.py`, `emul_baosn.py` and `emul_mps.py`. In each, trace `initialize`, `get_requirements`, `must_provide`, `calculate` and getters. Compare requested coordinates with artifact coordinates and identify per-call state replacement.

### 8. Stage 6: executable evidence

*a. 35.* ***`ai/gates/README.md`****.* Learn runner vocabulary, tiers, logs, resume and workstation commands. Treat its gate table as navigation into the detailed Appendix A.

*b. 36.* ***`ai/gates/board.py`****.* Read the `Gate` record and shared helper functions, then one pure identity gate and one smoke gate. Follow capability/dependency metadata into the ordered board registry.

*c. 37.* ***`ai/gates/checks/*.py`****.* Begin with `ai/gates/checks/scalar_identity.py`: its named

scalar outputs make shape, round-trip, refusal and mutation legs easier to see than in a full survey vector. Then read the check matching the feature selected in the preceding stages. Locate its production import, fixture, independent reference, valid control, mutation/catch-power arm and nonzero failure exit. Family identity checks precede family smokes.

*d. 38.* `ai/gates/run_board.py`. Read this last. The scientific claims already live in `board.py`. The runner owns preflight, selection, subprocess capture, logs and resume. Compare its branches with the structured-evidence coverage paragraph, the resume-state table and the safe-use warnings in Appendix A.

### 9. Three shorter routes

| Goal | Read in this order | Write down at each boundary | First concrete evidence |
|---|---|---|---|
| Change a model | `designs/blocks.py`. Then `designs/plain.py` or `designs/ia.py`. `activations.py`. Model construction in `training.py`. Save/rebuild in `results.py` | Input/output shape, registered parameter owner, initialization function, constructor spec, persisted class/state | `ai/gates/checks/gsv_bitwise_drift.py`, plus the relevant `head-activation-pin` or `relu-tanh-norm` board gate |
| Change a physical family | Family generator. Family geometry. Family loss. Family branch in `experiment.py`. Branch in `inference.py`. Corresponding `cobaya_theory/emul_*.py` adapter | Quantity, units, exact axis, target law, metric, artifact fact, requested Cobaya product | For the scalar worked route, `ai/gates/checks/scalar_identity.py` followed by `scalar-smoke`. Substitute the matching family pair |
| Change training behavior | Validation and factory in `training.py`. Executed minibatch/epoch loop. History writer in `results.py`. `family_drivers.py` and `scheduling.py` when a campaign is parallel | Validated control, state before/after one step, reported metric, selection record, behavior under a different storage/device lane | `ai/gates/checks/finite_contract.py` and the `eval-batch-invariance` board gate |

TABLE XXXI. Three bounded study routes. Periods separate successive files or boundaries. These entries describe reading boundaries. The third column is the reader's worksheet and the final column names a concrete executable starting point for each route.